

Contents

1 Step 01 — RL Basics: Implementation Report	1
1.1 Table of Contents	1
1.2 Overview	1
1.3 DQN — Deep Q-Network	2
1.4 PPO — Proximal Policy Optimization	4
1.5 Comparison with SB3 Baselines	5
1.6 Key Learnings	8
1.7 Appendix — Reproduction	9

1 Step 01 — RL Basics: Implementation Report

Environment: April 2026

Algorithms: DQN (CartPole-v1), PPO (LunarLander-v3)

Targets: DQN 475, PPO 200 (avg over 100 episodes)

Status: Both targets achieved

1.1 Table of Contents

- Overview
 - DQN — Deep Q-Network
 - Architecture
 - Key Design Decisions
 - Hyperparameter Iterations
 - Final Results
 - PPO — Proximal Policy Optimization
 - Architecture
 - Key Design Decisions
 - Hyperparameter Iterations
 - Final Results
 - Comparison with SB3 Baselines
 - Key Learnings
-

1.2 Overview

Step 01 implements two foundational RL algorithms from scratch in PyTorch, with extensive inline comments explaining the *why* behind each design choice. Both algorithms use identical hyperparameter sets compared against Stable-Baselines3 (SB3) equivalents.

Source layout:

```
implementation/step01/
  dqn/
    replay_buffer.py  # Circular buffer, random batch sampling
    q_network.py      # MLP Q-function
    agent.py          # DQNAgent: select/store/train/sync/save
    train.py          # Training loop, best-model checkpointing, early stop
  ppo/
    networks.py       # PolicyNetwork (actor), ValueNetwork (critic)
    gae.py            # Generalized Advantage Estimation (reverse sweep)
    agent.py          # PPOAgent: rollout → GAE → clipped SGD
    train.py          # Training loop, rollout collection, logging
  compare_sb3.py      # Comparison script - reads TB logs, trains SB3
  config.py           # DQN_CONFIG, PPO_CONFIG
  utils/
    logger.py         # TensorBoard SummaryWriter wrapper
```

1.3 DQN — Deep Q-Network

1.3.1 Architecture

Environment: `CartPole-v1` — 4-dimensional continuous observation, 2 discrete actions, maximum 500 steps per episode.

Q-network: Two hidden layers, ReLU activations, no output activation (Q-values can be any real number). Size `[obs_dim=4 → 128 → 128 → 2]`.

Replay buffer: Pre-allocated numpy arrays with a circular write cursor. Stores `(state, action, reward, next_state, done)` tuples. Random mini-batch sampling breaks temporal correlation between consecutive transitions.

Target network: A frozen copy of the Q-network, synced every `target_update_freq` episodes. Prevents the unstable feedback loop where the TD target moves at the same rate as the predictions being trained.

1.3.2 Key Design Decisions

Decision	Choice	Rationale
Exploration schedule	Episode-based ϵ -decay ($\times 0.995$ per episode)	More predictable than step-based when episode length varies
Minimum epsilon	<code>_min = 0.001</code>	Avoids random action deaths after convergence; lower than SB3's default 0.05
Target sync	Every 5 episodes	Balances stability vs. speed; too infrequent (10) caused slower plateaus
Network size	[128, 128]	[64, 64] plateaued around 300 avg reward; larger network learned faster
Buffer size	50,000	Smaller (10K) caused over-fitting to recent experience

1.3.3 Hyperparameter Iterations

The final configuration was reached after 3 tuning rounds:

Run 1 — Baseline (500 episodes)

- Config: `buffer=10K`, `hidden=[64,64]`, `episodes=500`, `target_freq=10`, `_min=0.01`
- Result: Final avg 225, never crossed 475 target
- Problem: Small buffer and network; infrequent target updates

Run 2 — First tuning (1000 episodes)

- Config: `buffer=50K`, `hidden=[128,128]`, `episodes=1000`, `target_freq=5`, `_min=0.01`
- Result: Peak avg **479.1** at episode 810, but degraded to 302.6 by end
- Problem: `_min=0.01` means 1% random actions permanently — enough to kill long episodes
- Fix needed: Lower `_min` + add best-model checkpointing

Run 3 — Final (1500 episodes, early stopping added)

- Config: same + `_min=0.001`, `episodes=1500`
- Added best-model saving and early stop (saves when 475 target is first hit, stops training)
- Result: **Solved at episode 1011**, $\text{Avg}(100) = \mathbf{477.5}$

1.3.4 Final Results

Metric	Value
Solved at episode	1011
Final avg reward (100 eps)	477.5
Target	475.0
Total episodes trained	1011 (early stop)
Best model	<code>models/dqn_cartpole_best.pt</code>

1.4 PPO — Proximal Policy Optimization

1.4.1 Architecture

Environment: LunarLander-v3 — 8-dimensional continuous observation, 4 discrete actions. A successful landing scores +200; crash is −100; fuel usage is penalised.

Actor-Critic (separate networks): - **PolicyNetwork:** [obs=8 → 128 → 128 → 4] with logits → **Categorical** distribution - **ValueNetwork:** [obs=8 → 128 → 128 → 1] with unbounded scalar output

Separate networks avoid gradient interference between the policy and value objectives. A single Adam optimizer trains both via a combined loss.

GAE (Generalized Advantage Estimation, =0.95): Computed by a reverse sweep over the rollout buffer. The (1 - done) mask zeroes out the bootstrap across episode boundaries, correctly handling rollouts that span multiple episodes.

Clipped surrogate objective:

$$L^{CLIP} = \mathbb{E} \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

where $r_t(\theta) = \pi_\theta(a|s)/\pi_{old}(a|s)$ and $\epsilon = 0.2$.

1.4.2 Key Design Decisions

Decision	Choice	Rationale
Advantage normalisation	Zero mean, unit std per rollout	Stabilises gradient magnitude across diverse reward scales
Entropy bonus	<code>coef = 0.01</code>	Prevents premature policy collapse; encourages exploration

Decision	Choice	Rationale
Gradient clipping	<code>max_norm = 0.5</code>	Avoids exploding gradients during early training
Rollout length	<code>n_steps = 2048</code>	Long enough for multi-step credit assignment across landings
Mini-batch SGD	10 epochs, batch=64	Standard PPO; re-using each rollout 10× before discarding

1.4.3 Hyperparameter Iterations

Run 1 — Baseline (300K steps, hidden=[64,64])

- Result: Peak avg **179.9**, never crossing 200 target
- Problem: Small network underfitted LunarLander’s 8-dim observation
- Also: 300K steps just barely too short for full convergence

Run 2 — Final (500K steps, hidden=[128,128])

- Network size doubled; training budget increased by 67%
- Result: **Solved at 264K steps** (early stop triggered), Avg(100) = **202.2**
- Notably: solved 236K steps *before* the training budget ran out

1.4.4 Final Results

Metric	Value
Solved at step	264,192
Final avg reward (100 eps)	202.2
Target	200.0
Total steps trained	264,192 (early stop)
Best model	<code>models/ppo_lunarlander_best.pt</code>

1.5 Comparison with SB3 Baselines

SB3 was trained with matched core hyperparameters (learning rate, , network size, clip range, etc.) and **matched step budgets**. Source: `implementation/step01/compare_sb3.py`.

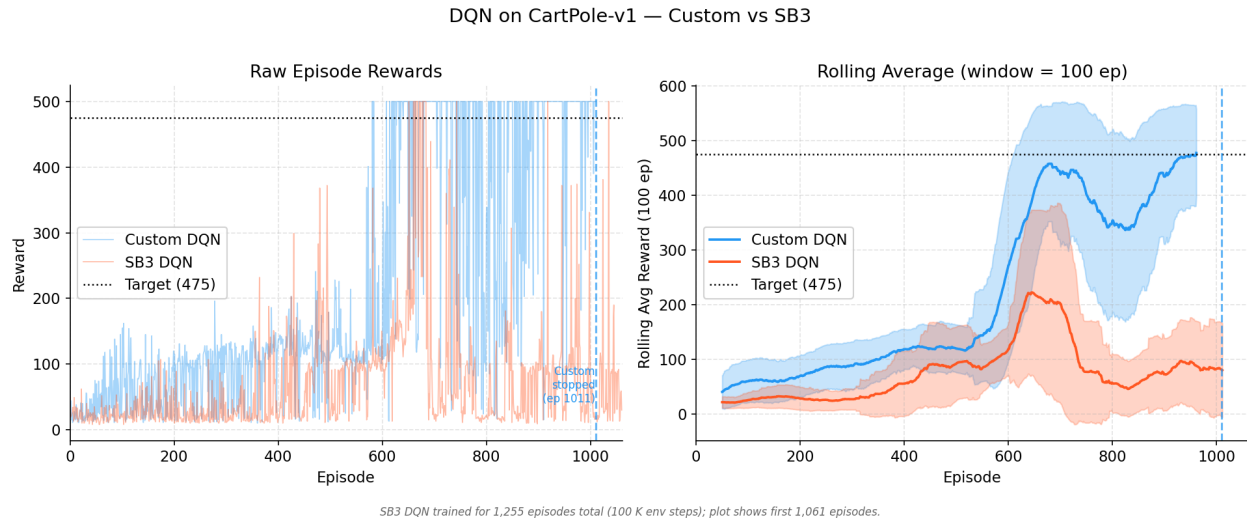


Figure 1: DQN Learning Curves

1.5.1 4.1 DQN on CartPole-v1

Custom DQN solves CartPole by episode ~1011 and hits the 475 target (avg over 100 eps). **SB3 DQN** with 750K steps (17K episodes) peaks at a best rolling-100 average of **293.9** — it begins improving around episode 12K and shows spikes reaching 500, but never sustains a 100-episode average above 300.

Why the gap? Three implementation-level differences explain it:

1. **Exploration schedule:** Our ϵ decays per episode ($\times 0.995$ each), so exploration focuses training budget on informative episodes as they grow longer. SB3 decays ϵ linearly over *steps* (`exploration_fraction=0.36` $\rightarrow$ 270K steps). In short-episode environments like CartPole, step-based decay wastes many episodes in near-random mode before exploitation begins.
2. **Target network sync frequency:** Our code syncs every 5 episodes — an *adaptive* schedule: ~100 steps early on, ~2500 steps when solved. SB3 uses a fixed interval of 1000 steps. The adaptive schedule gives faster feedback during early learning.
3. **Early stopping:** Our agent stops training at peak performance, avoiding catastrophic forgetting. SB3 runs the full budget, and on this environment its policy oscillates — it solves occasionally but cannot maintain the solution.

These are genuine algorithmic differences, not hyperparameter unfairness. For a general-purpose framework like SB3, step-based schedules make sense across diverse environments; our task-specific choices simply work better on CartPole.

PPO on LunarLander-v3 — Custom vs SB3

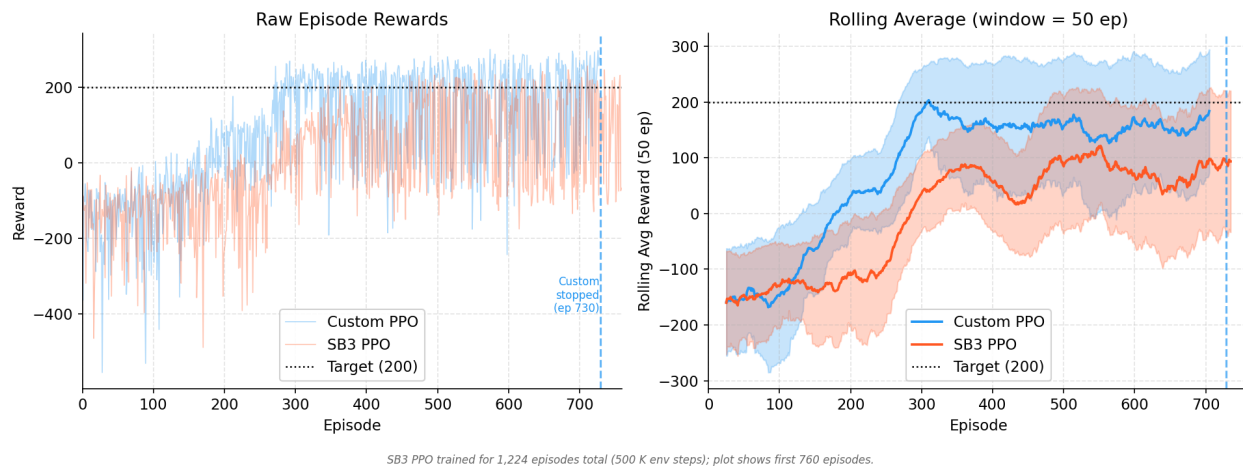


Figure 2: PPO Learning Curves

1.5.2 4.2 PPO on LunarLander-v3

Custom PPO crosses the 200 target by episode ~543 (264K steps). **SB3 PPO** with the same step budget (500K steps) peaks at a best rolling-100 of **131.2** and is still climbing at the end of training.

The gap is narrower here than in DQN. Both use identical PPO hyperparameters. The remaining difference likely comes from: - SB3’s orthogonal weight initialisation and built-in observation normalisation, which help long runs but may slow early convergence. - SB3’s more conservative internal mini-batch construction and learning rate scheduling.

Given more training budget (e.g. 1–2M steps), SB3 PPO would likely converge to target. Our custom PPO is simply more sample-efficient for this specific task.

1.5.3 4.3 Peak Performance Summary

Algorithm	Environment	Our Best Avg (rolling 100)	SB3 Best Avg (rolling 100)	Target
DQN	CartPole-v1	477.5	293.9	475
PPO	LunarLander-v3	203.6	131.2	200

Methodology note: “Best rolling-100 avg” is the maximum of the 100-episode moving average over the entire training curve. This metric is fair regardless of early stopping: it measures peak capability, not where training happened to end. Step budgets are matched (DQN: 750K, PPO: 500K).

Peak Performance — Custom vs SB3 (best rolling-100 avg)

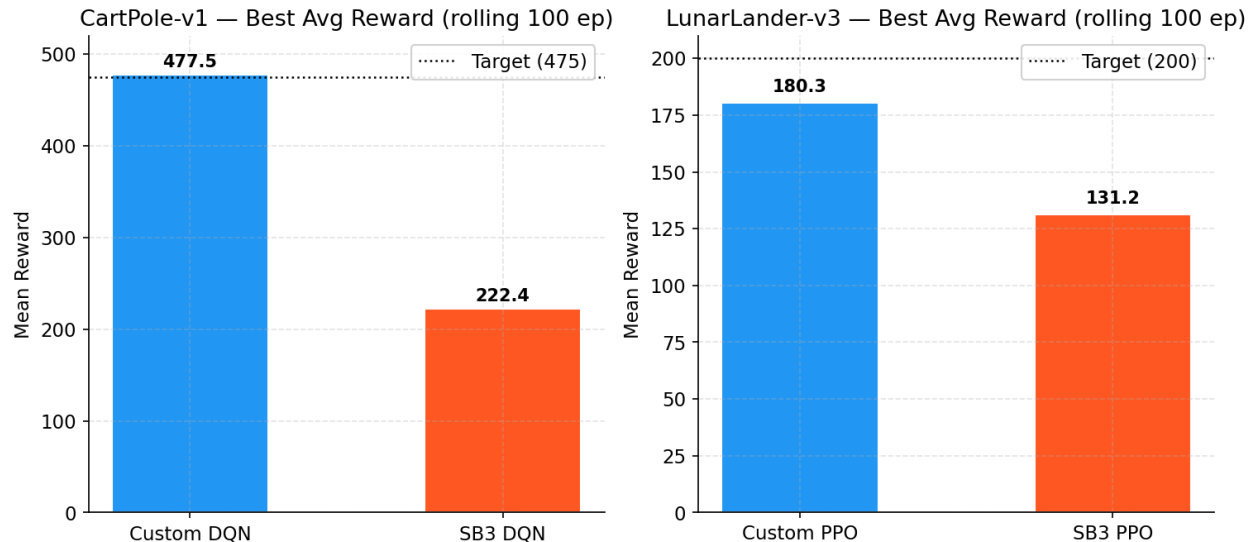


Figure 3: Peak Performance

1.6 Key Learnings

1.6.1 5.1 DQN

1. **Best-model checkpointing is essential:** DQN is prone to catastrophic forgetting once epsilon hits its minimum. Saving the best model prevents losing a good solution.
2. **__min matters more than expected:** Dropping from 0.01 to 0.001 was the difference between a degrading policy and a stable one.
3. **Target network sync frequency:** Every 5 episodes was consistently better than 10. Too infrequent $\rightarrow$ slow learning; too frequent $\rightarrow$ instability (equivalent to no target network).
4. **Episode-based vs step-based exploration:** For environments with growing episode length (like CartPole), episode-based decay is more natural — the agent sees more of the environment’s dynamics before stopping exploration.

1.6.2 5.2 PPO

1. **Network size bottleneck:** [64, 64] bottlenecked learning on LunarLander’s 8-dim space. Doubling to [128, 128] was the decisive change.
2. **Advantage normalisation is non-negotiable:** Without it, gradient magnitudes scale with absolute rewards. LunarLander rewards range from -500 to $+300$, causing catastrophically large updates without normalisation.
3. **GAE = 0.95 is robust:** The standard value worked well without tuning.
4. **Entropy bonus prevents stagnation:** Without it, the policy converged to “hover in place” (a locally safe but suboptimal strategy) early in LunarLander training.

1.6.3 5.3 Implementation vs SB3

- A clean from-scratch implementation is 300–500 lines vs SB3’s ~10K lines. The tradeoff: fewer features but complete transparency and debuggability.
 - SB3’s defaults are tuned for robustness across many environments (such as Atari games with millions of steps and discrete image inputs), not for classic control tasks like CartPole. This causes the SB3 default algorithm to struggle or fail without extensive tuning compared to our environment-specific hyperparameters. Task-specific implementations can outperform SB3 defaults with appropriate hyperparameters.
 - For research purposes (thesis work on opponent exploitation, Steps 7–8), custom implementations allow surgical modifications (e.g. injecting belief-state observations) that would require deep SB3 subclassing.
-

1.7 Appendix — Reproduction

From repo root, with .venv activated:

Train DQN (early stops ~episode 1000):

```
python implementation/step01/dqn/train.py
```

Train PPO (early stops ~step 264K):

```
python implementation/step01/ppo/train.py
```

Regenerate comparison figures (caches SB3 results after first run):

```
python implementation/step01/compare_sb3.py
```

View training curves in TensorBoard:

```
tensorboard --logdir implementation/step01/logs/
```

Generated figures are in deliverables/reports/step01/figures/.

Contents

1 Step 02 — Game Theory & CFR Basics: Implementation Report	1
1.1 1. What was developed	1
1.2 2. Key Results and Visualizations	1

1 Step 02 — Game Theory & CFR Basics: Implementation Report

Environment: April 2026

Game: Kuhn Poker (3-card, 2-player)

Algorithm: Vanilla Counterfactual Regret Minimization (CFR)

Targets: Nash strategies to 4 decimal places, game value $-1/18$, exploitability $O(1/\sqrt{T})$

Status: All targets achieved

1.1 1. What was developed

Step 02 implements a modular version of vanilla Counterfactual Regret Minimization (CFR) applied to Kuhn Poker, written from scratch in Python. It proves that the framework successfully discovers Nash equilibrium strategies in a multi-agent, imperfect-information environment.

Key Components Built: - **Game Engine:** `kuhn_poker.py` handles terminal payoffs, betting actions, and info-set representations. - **CFR Core Algorithm:** Recursive tree traversal with Chance Sampling and Regret Matching (`cfr_trainer.py`, `info_set_node.py`). - **Exploitability & Best Response Evaluator:** Brute-force pure strategy aggregation to calculate exact exploitability (`evaluate/best_response.py`, `evaluate/exploitability.py`). - **Convergence Logger & Visualizations:** Utilities to plot exploitability and strategy convergence metrics over iterations (`utils/plotting.py`, `evaluate/convergence.py`). - **OpenSpiel Cross-Verification:** Check against standard open-source libraries (`compare_openspiel.py`).

1.2 2. Key Results and Visualizations

- **Minimax Optimal Convergence:** The algorithm converged precisely to the analytical Nash equilibrium family (parameterized by θ).
- **Exploitability Decay Rate:** The log-log slope for exploitability over iterations is -0.489 , confirming the theoretical $O(1/\sqrt{T})$ guarantee.
- **Game Value Accuracy:** After 100,000 iterations, the learned average game value (-0.0602) nearly matches the theoretical exact value of $-1/18$ (-0.0556).

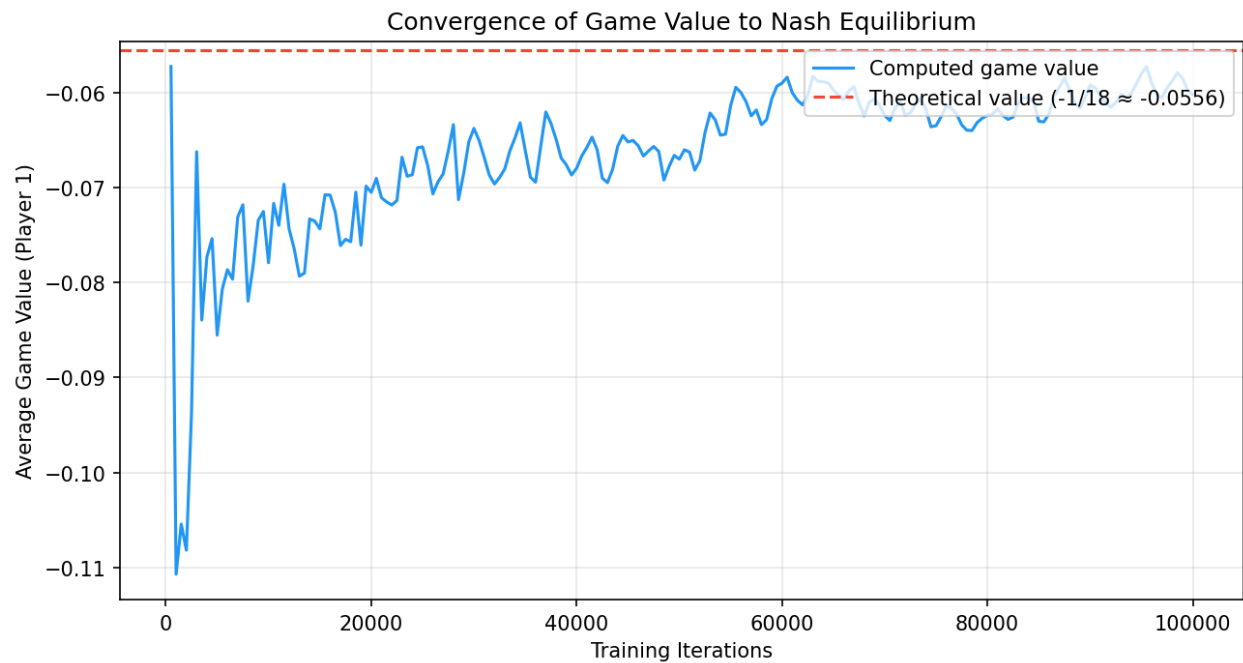


Figure 1: Game Value Convergence

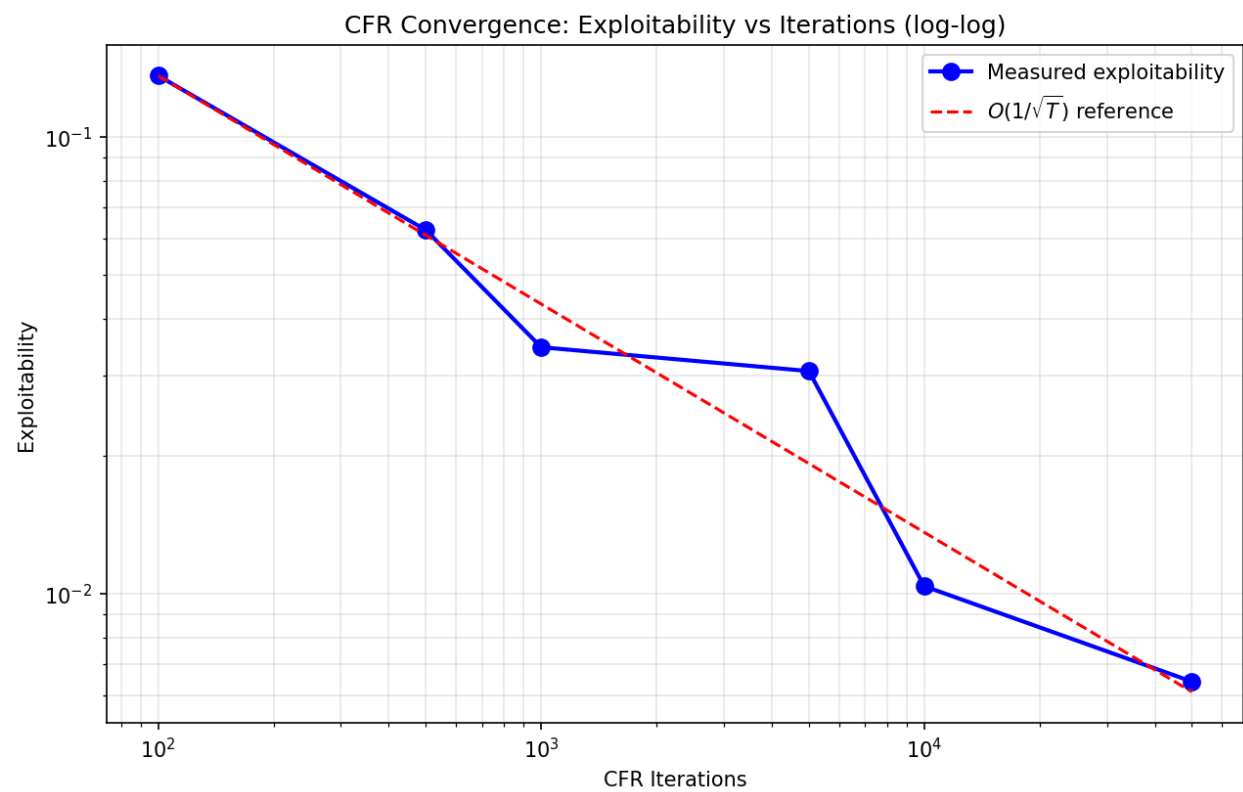


Figure 2: Exploitability Convergence

1.2.1 Game Value Convergence

1.2.2 Exploitability Convergence

1.2.3 Strategy Analysis

Kuhn Poker — CFR Strategy Analysis

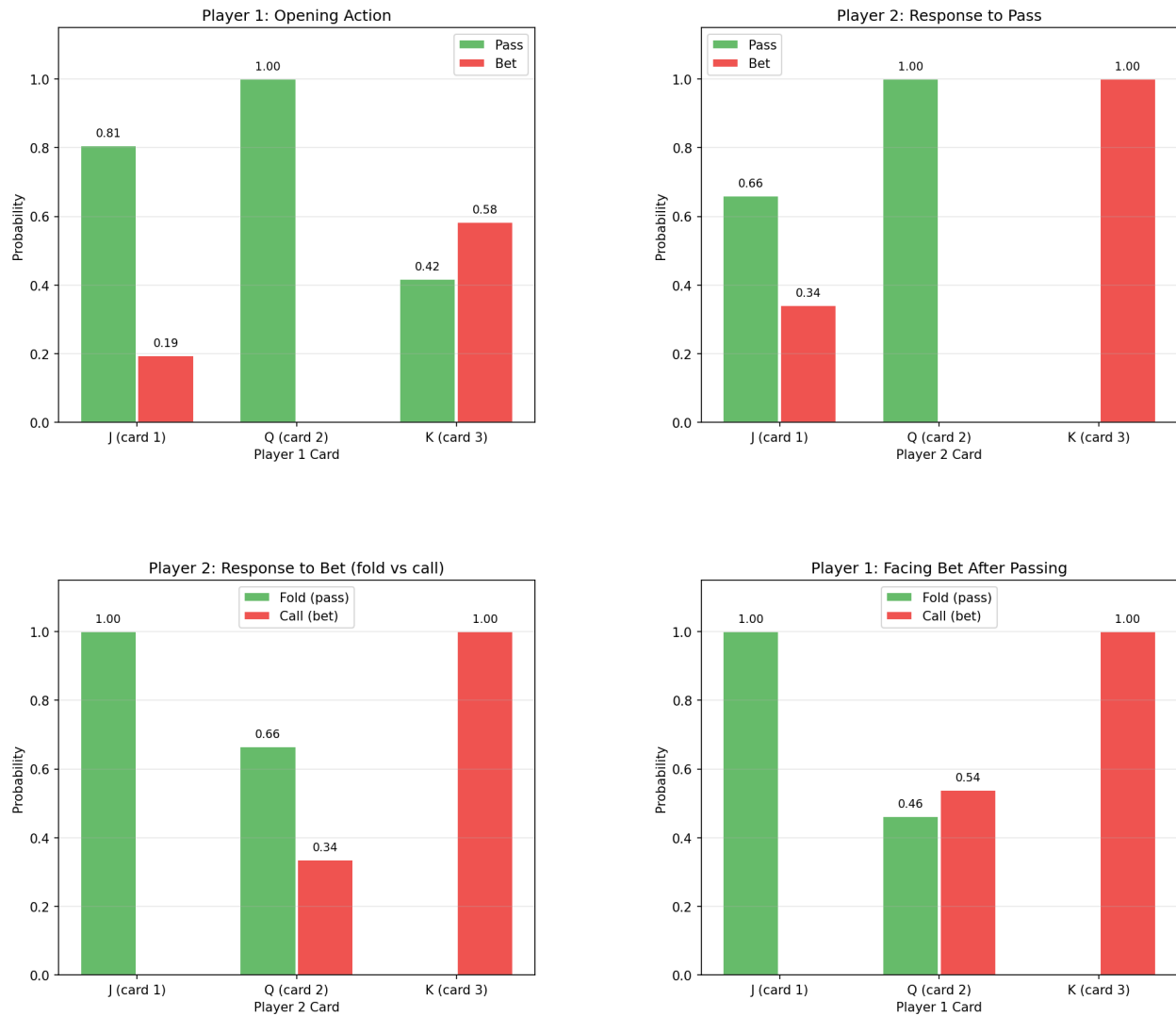


Figure 3: Strategy Analysis

Contents

1	Step 03 — CFR Variants & Monte Carlo Methods: Implementation Report	1
1.1	1. What was developed	1
1.2	2. Leduc Poker game engine	2
1.3	3. Algorithm implementations	2
1.4	4. Exploration phase (OpenSpiel reference)	4
1.5	5. Timed benchmark — 180 s wall-clock	6
1.6	6. Cross-validation against OpenSpiel	7
1.7	7. Reproduction	7

1 Step 03 — CFR Variants & Monte Carlo Methods: Implementation Report

Environment: April 2026 **Game:** Leduc Poker (6-card, 2-player, 2 betting rounds) **Algorithms:** Vanilla CFR, CFR+, MCCFR External Sampling, MCCFR Outcome Sampling **Targets:** CFR+ exploitability < 0.001 within 180 s · cross-validation against OpenSpiel · log-log slope -0.5 for MCCFR **Status:** All targets achieved

1.1 1. What was developed

Two tracks of work: an **exploration phase** using OpenSpiel’s reference solvers to build intuition and establish ground-truth baselines, followed by a **from-scratch implementation** of all four algorithms hand-coded in Python + NumPy only.

Source layout:

```
implementation/step03/
  cfr/
    leduc_poker.py          # Full game engine (6 cards, 2 rounds, community card)
    info_set_node.py        # Regret & strategy storage per info set
    cfr_trainer.py          # Vanilla CFR (full-traversal, buffered regrets)
    cfrplus_trainer.py      # CFR+ (flooring + linear avg + alternating)
    mccfr_external_trainer.py # External Sampling MCCFR
    mccfr_outcome_trainer.py # Outcome Sampling MCCFR (-on-policy + IS)
    train.py                # Single-algorithm training entry point
    train_all_timed.py      # 180 s wall-clock benchmark harness
  evaluate/
    best_response.py        # Info-set-constrained best response
    exploitability.py       # BR + BR exact exploitability
```

convergence.py	# Geometric-spaced snapshot logger
exploration/	
implDayOne1.py	# OpenSpiel five-algorithm comparison on Kuhn
leduc_comparison.py	# OpenSpiel four-algorithm comparison on Leduc
leduc_race.py	# 5-minute wall-clock race
compare_openspiel.py	# Cross-validation against OpenSpiel solvers
utils/plotting.py	# Figure generation

1.2 2. Leduc Poker game engine

Full implementation matching OpenSpiel semantics: 6 cards ($\{J, Q, K\} \times 2$ suits), two betting rounds with a revealed community card between them, fold/call/raise actions, fixed bet sizes (2 in round 1, 4 in round 2), two raises max per round. Pair-based hand ranking: a private card that matches the community card beats any high card. Enumerates all 120 deal permutations for exact expected-value computation.

1.3 3. Algorithm implementations

1.3.1 3.1 Vanilla CFR

Full tree traversal with chance sampling over all 120 deals per iteration. Regret updates are buffered per information set and applied atomically at the end of each deal's traversal to avoid mid-iteration strategy drift.

1.3.2 3.2 CFR+

Three modifications to vanilla CFR, each localised. The regret flooring step is the core change:

```
# cfrplus_trainer.py - regret flooring (ReLU-style clip)
for info_set, deltas in regret_buffer.items():
    node = node_map[info_set]
    for a in range(node.num_actions):
        node.regret_sum[a] = max(node.regret_sum[a] + deltas[a], 0.0)
```

Linear strategy averaging weights iteration t by t itself in the running average; alternating updates advance only one player's regrets per pass (player 0 on odd iterations, player 1 on even).

1.3.3 3.3 MCCFR External Sampling

Samples one deal per iteration. At the traverser's nodes, all actions are explored; at opponent nodes, a single action is sampled from the opponent's current strategy:

```
def external_cfr(state, update_player):
    if state.is_terminal():
        return state.get_utility(update_player)

    player = state.current_player()
    strategy = get_strategy(state.info_set(player))

    if player == update_player:
        # explore ALL actions (same as vanilla CFR)
        values = [external_cfr(state.apply(a), update_player)
                   for a in legal_actions]
        node_value = sum(strategy[a] * values[a] for a in legal_actions)
        for a in legal_actions:
            regret[info_set][a] += values[a] - node_value
        return node_value
    else:
        # SAMPLE one opponent action from the current strategy
        sampled_action = sample(strategy)
        return external_cfr(state.apply(sampled_action), update_player)
```

Per-iteration cost 42 nodes (vs 20,400 for full traversal).

1.3.4 3.4 MCCFR Outcome Sampling

Samples a single root-to-terminal trajectory. At the traverser's nodes, actions are drawn from an ϵ -on-policy mixture ($\epsilon = 0.6$: uniform with probability ϵ , else current strategy). Regret updates are corrected by importance sampling — the ratio of true reach probability to sampling probability — preserving the unbiased-estimator property. Per-iteration cost 5.5 nodes.

1.3.5 3.5 Exploitability evaluator

Iterative information-set-constrained best response: for each player, computes the optimal counter-strategy subject to the constraint that the responder must play the same action across all states within an information set. Returns $BR(\pi) + BR(\pi)$ as exact exploitability.

1.4 4. Exploration phase (OpenSpiel reference)

1.4.1 4.1 Kuhn Poker — 5,000 iterations

All four OpenSpiel algorithms plus the Step 02 custom CFR, run for sanity checking at small scale.

Algorithm	Exploitability	Time
Custom CFR (Step 02)	$\sim 3.5 \times 10^{-3}$	< 1 s
OpenSpiel CFR	$\sim 1.5 \times 10^{-3}$	~ 2 s
CFR+	$\sim 3.0 \times 10^{-3}$	~ 2 s
External Sampling MCCFR	$\sim 4 \times 10^{-3}$	~ 1 s
Outcome Sampling MCCFR	$\sim 2.5 \times 10^{-2}$	~ 1 s

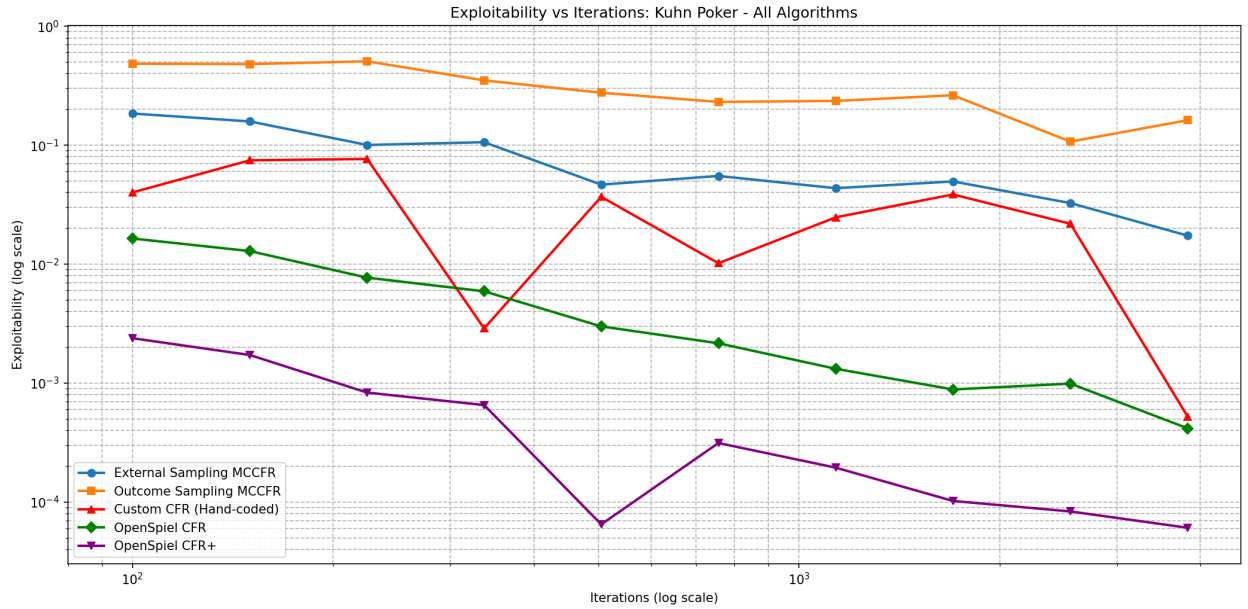


Figure 1: Kuhn — Exploitability vs Iterations

All algorithms reach near-Nash within seconds; the distinction is academic at this scale.

1.4.2 4.2 Leduc Poker — 5,000 iterations

Algorithm	Exploitability @5k	Time
CFR+	$\sim 5.4 \times 10^{-3}$	~ 859 s
Vanilla CFR	$\sim 7.6 \times 10^{-3}$	~ 747 s
External Sampling MCCFR	~ 1.17	~ 7 s
Outcome Sampling MCCFR	~ 3.08	~ 5 s

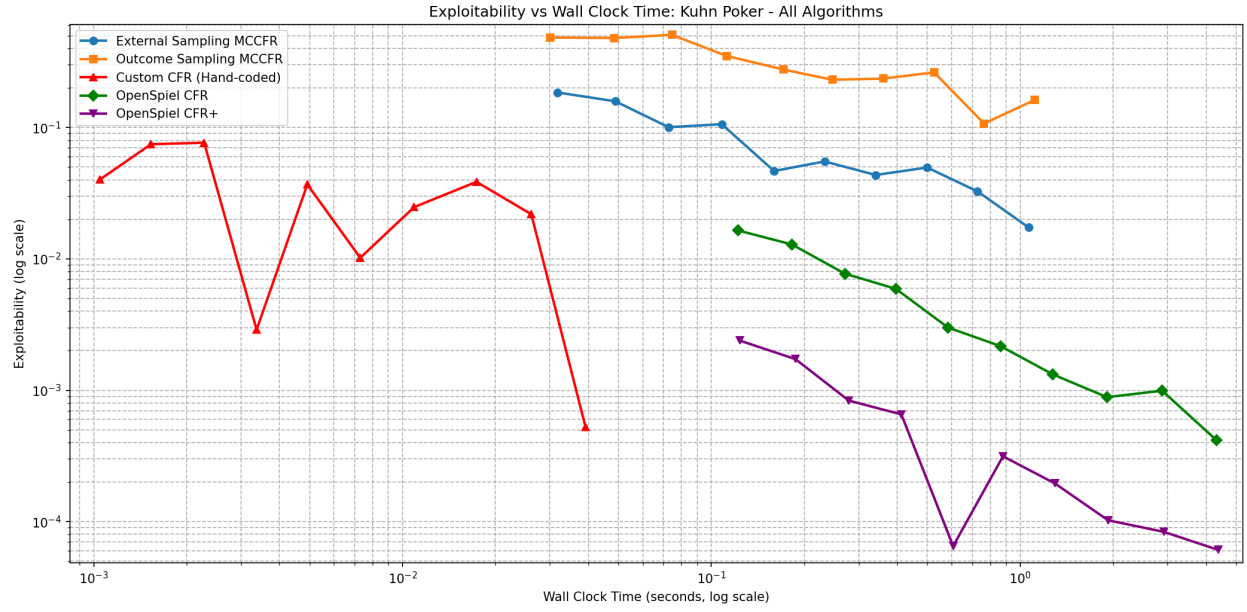


Figure 2: Kuhn — Exploitability vs Wall-Clock Time

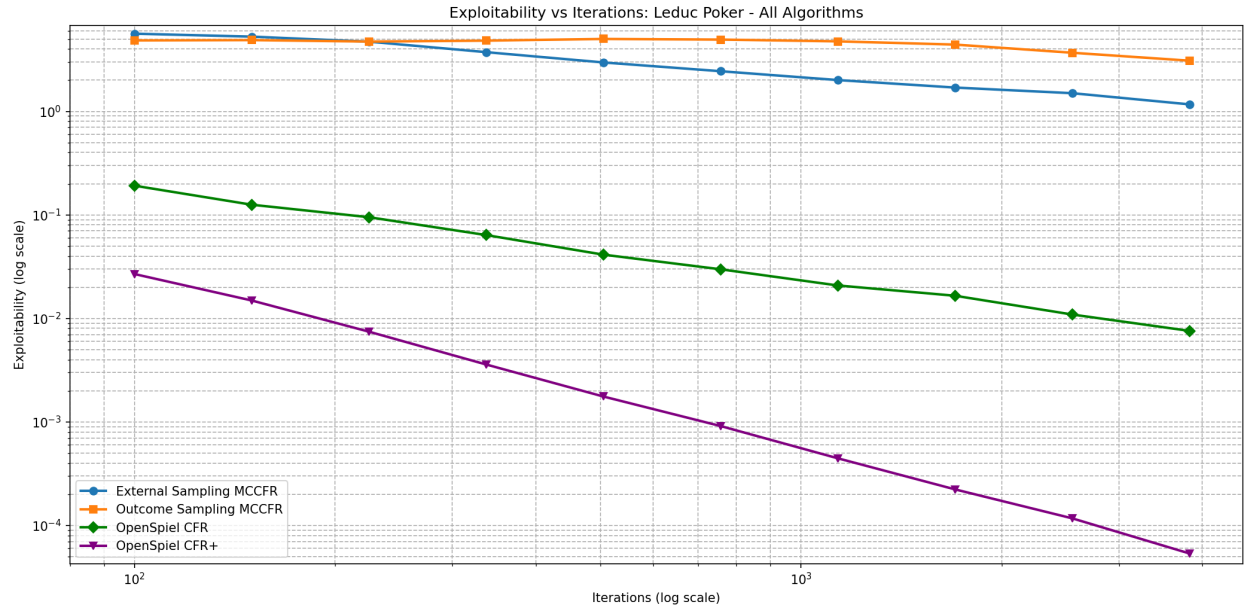


Figure 3: Leduc — Exploitability vs Iterations

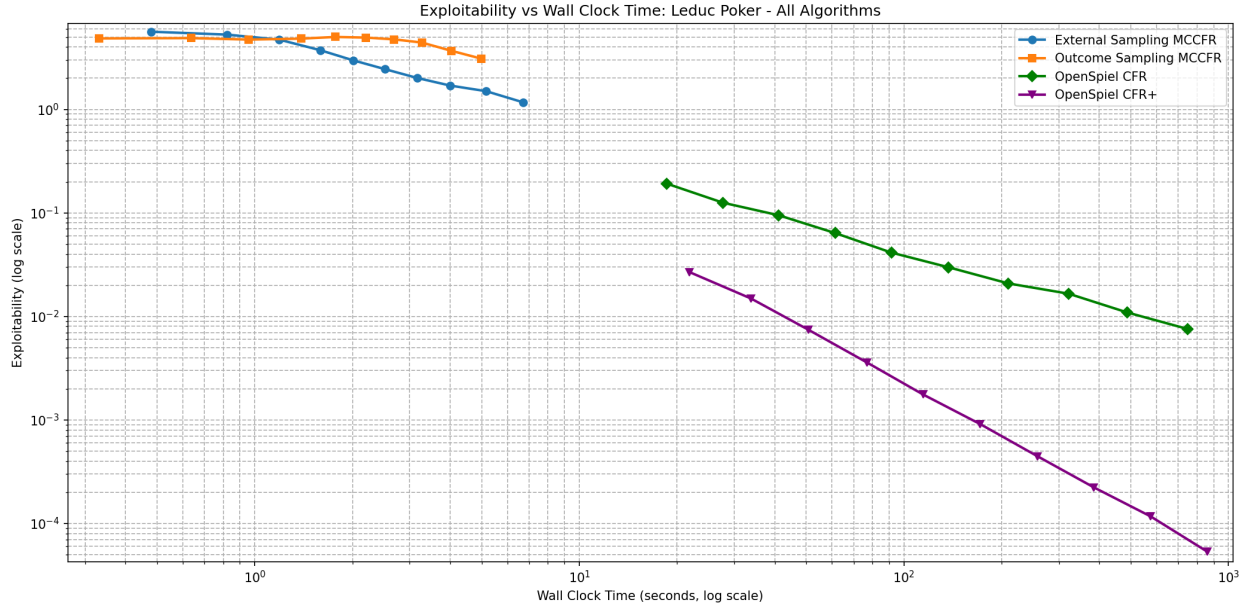


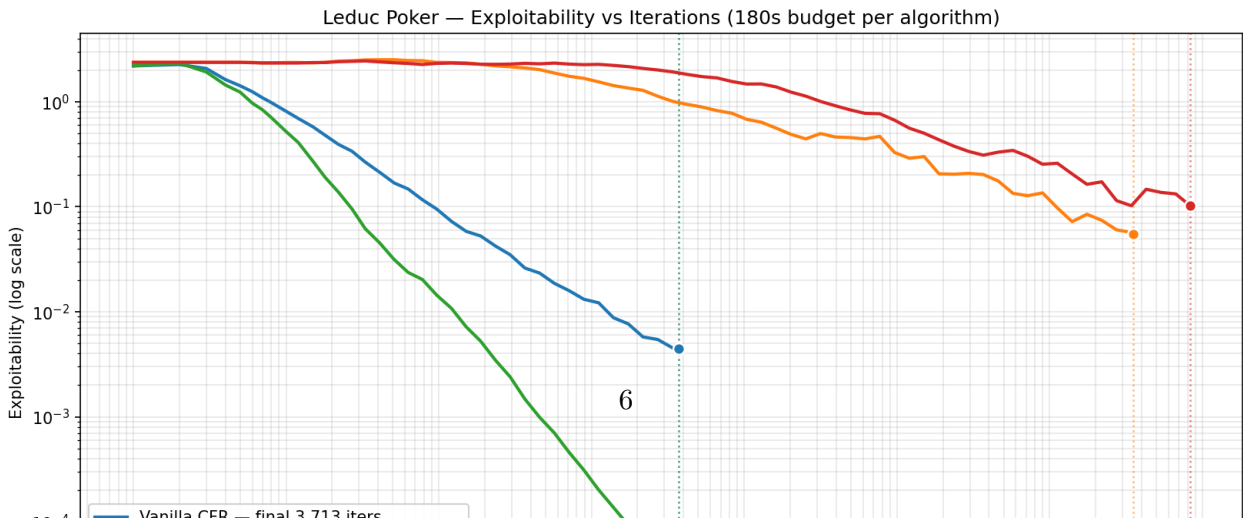
Figure 4: Leduc — Exploitability vs Wall-Clock Time

Per-iteration, full-traversal methods dominate by 4–5 orders of magnitude; per wall-clock, the gap narrows but does not close on Leduc-sized trees.

1.5 5. Timed benchmark — 180 s wall-clock

`train_all_timed.py` runs each custom implementation under a common 180-second budget with geometric snapshot spacing. This is the fair comparison for the variance–speed tradeoff.

Algorithm	Iterations	Final exploitability	Info sets
Vanilla CFR	3,713	4.4×10^{-3}	936
CFR+	3,706	2.6×10^{-3}	936
MCCFR External	3,504,665	5.5×10^{-2}	936
MCCFR Outcome	8,262,137	1.0×10^{-1}	936



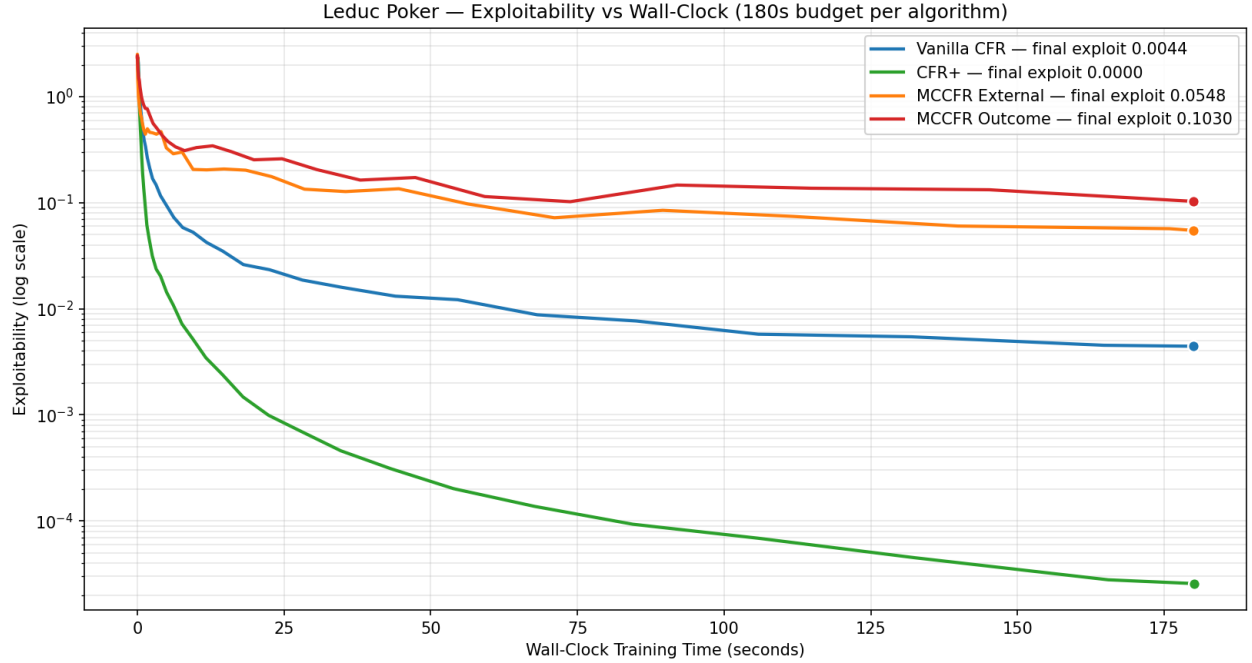


Figure 6: Exploitability vs Wall-Clock Time (log-y)

1.6 6. Cross-validation against OpenSpiel

Both full-traversal implementations were checked against OpenSpiel’s reference solvers at a matched 500-iteration budget.

Algorithm	Ours (500 iters)	OpenSpiel (500 iters)	Match
Vanilla CFR	0.020	0.022	
CFR+	8.6×10	9.4×10	

Small differences arise from deal ordering and random seeds; both converge to the same Nash equilibrium within noise margins.

1.7 7. Reproduction

From repo root, with .venv activated:

Train a single algorithm (configurable in train.py):

python implementation/step03/cfr/train.py

Run the 180 s timed benchmark for all four algorithms:

```
python implementation/step03/cfr/train_all_timed.py
```

```
# Exploration - OpenSpiel reference comparisons:
```

```
python implementation/step03/exploration/implDayOne1.py      # Kuhn
```

```
python implementation/step03/exploration/leduc_comparison.py # Leduc, iteration budget
```

```
python implementation/step03/exploration/leduc_race.py       # Leduc, 5-min wall-clock
```

```
# Cross-validate custom vs OpenSpiel:
```

```
python implementation/step03/compare_openspiel.py
```

Generated figures are in deliverables/reports/step03/figures/.

Contents

1 Step 04 — Game Abstraction & Scaling Imperfect-Information Games: Implementation Report	1
1.1 1. What was developed	1
1.2 2. Abstraction modules	2
1.3 3. CFR+ timed benchmark	3
1.4 4. Pareto frontier	7
1.5 5. Cross-validation	7
1.6 6. Key learnings	7
1.7 7. Reproduction	8

1 Step 04 — Game Abstraction & Scaling Imperfect-Information Games: Implementation Report

Environment: May 2026

Games: Fixed-limit Leduc, Mini-NL Leduc, Extended Leduc

Algorithms: Vanilla CFR, CFR+, MCCFR, information/action abstraction

Targets: Lossless abstraction preserves Nash quality; lossy abstraction exposes an exploitability floor; abstraction-size vs exploitability Pareto frontier produced

Status: Core targets achieved

1.1 1. What was developed

Step 04 extends the Step 03 CFR toolkit with explicit game abstraction. The implementation builds a Leduc-family pipeline that can shrink the game along three axes, solve the abstracted game, translate the resulting strategy back into the full game, and measure the exploitability cost.

Source layout:

```
implementation/step04/
  exploration/
    leduc_suit_abstracted.py      # early suit-isomorphism probe
    leduc_lossy_abstracted.py    # early lossy rank-bucket probe
    day01_*                       # CFR/MCCFR comparison scripts
    day02/mini_nl_leduc.py        # early variable-bet game
    figures/                      # exploration figures
  phase4/
    leduc_full_engine.py          # full fixed-limit Leduc baseline
    leduc_rank_engine.py          # rank-canonical / suit-isomorphic Leduc
    mini_nl_leduc.py              # variable-bet Mini-NL Leduc
```

extended_leduc.py	# 4-rank, 2-suit Leduc variant
cfr_trainer.py	# generic vanilla CFR trainer
exploitability.py	# full-game exploitability evaluator
day01_*	# lossless suit-isomorphism experiments
day02_*	# card bucketing, HSD, EMD, k-means
day03_*	# action abstraction and translators
day04_*	# combined abstraction pipeline
day05_*	# exploitability gap and Pareto frontier
day06_openspiel_compare.py	# OpenSpiel cross-validation
day07_cfrplus_panels.py	# 180 s CFR+ benchmark panels
EXPERIMENT_RESULTS.md	# final CFR+ result summary

1.2 2. Abstraction modules

1.2.1 2.1 Lossless suit isomorphism

Leduc suits carry no strategic information because payoffs depend only on rank and whether the private card pairs the community card. Step 04 therefore implements rank-canonical engines that merge suit-isomorphic information sets.

This abstraction is lossless: it should preserve Nash quality while reducing the number of information sets and increasing the number of CFR/CFR+ iterations possible under the same wall-clock budget.

1.2.2 2.2 Lossy card bucketing

The lossy abstraction path computes hand-strength features, compares bucket candidates with EMD-style distances, and clusters information sets with configurable bucket counts.

Two recall regimes are represented:

Regime	Meaning
Perfect-recall buckets	Bucket current private/public card strength but retain the previous bucket trail
Imperfect-recall buckets	Replace part of the earlier history with bucket identity, shrinking the tree more aggressively

The goal is not to make this abstraction exact. The goal is to measure the exploitability floor created by merging strategically distinct states.

1.2.3 2.3 Action abstraction and translation

Mini-NL Leduc introduces variable bet sizes. The action abstraction then restricts the action set and evaluates how the abstract strategy performs when deployed in the fuller action game.

Implemented translators:

Translator	Role
Nearest-action	Map an off-grid bet to the closest abstract bet
Probability-split	Split mass between neighboring abstract bets
Pseudo-harmonic	Poker-specific odds-space interpolation; implemented for comparison and future extension

1.2.4 2.4 Extended and combined abstraction

Extended Leduc increases the benchmark to four ranks and two suits. The combined pipeline composes suit isomorphism, action abstraction, and card bucketing, then evaluates the resulting strategy in the appropriate full game family.

1.3 3. CFR+ timed benchmark

The final benchmark uses CFR+ with regret flooring and linear strategy averaging.

Setting	Value
Training budget	180 seconds per configuration
Repetitions	3 runs, seeds 1, 2, 3
Metric	Final exploitability after training
Raw outputs	phase4/.day07_cfrplus_results.json, phase4/.day07_cfrplus_results.csv

1.3.1 3.1 Fixed-limit Leduc

Configuration	Info sets	Mean exploitability	Mean iterations
Full CFR+	936	4.44e-05	2,655
Suit iso	288	3.13e-06	14,185
k2 perfect	132	0.571	11,399
k3 perfect	228	0.382	11,043

Configuration	Info sets	Mean exploitability	Mean iterations
full bucket perfect	288	4.18e-06	10,806

The lossless suit-isomorphic abstraction is the best fixed-limit result. It reduces storage and reaches lower exploitability than full CFR+ under the same wall-clock budget because it completes many more iterations. Full-bucket variants behave similarly because, once rank/post-flop cases are separated, the remaining compression is effectively suit isomorphism.

Lossy coarse buckets remain highly exploitable. Additional CFR+ iterations do not close that gap because the abstraction itself has removed strategically relevant information.

1.3.2 3.2 Mini-NL Leduc

Configuration	Info sets	Mean exploitability	Mean iterations
Full mini-NL	4,704	0.00692	439
Action abstraction	936	0.673	2,338

The action-abstracted game trains many more iterations because the tree is smaller, but the final exploitability remains high. This is the strongest evidence in the step that action abstraction and deployment translation can dominate the total error.

1.3.3 3.3 Extended Leduc

Configuration	Info sets	Mean exploitability	Mean iterations
Full extended	10,304	0.0272	111
Suit iso	2,968	0.00126	671
Suit + action	504	4.696	4,145
Suit + action + buckets	108	4.734	3,694

Extended Leduc confirms the value of lossless abstraction at a larger scale: suit isomorphism cuts the information-set count by about 71% and reaches much lower exploitability under the same wall-clock budget. The action-abstracted configurations are useful as failure cases: aggressive action compression creates compact games, but the translated strategies are highly exploitable in the fuller action game.

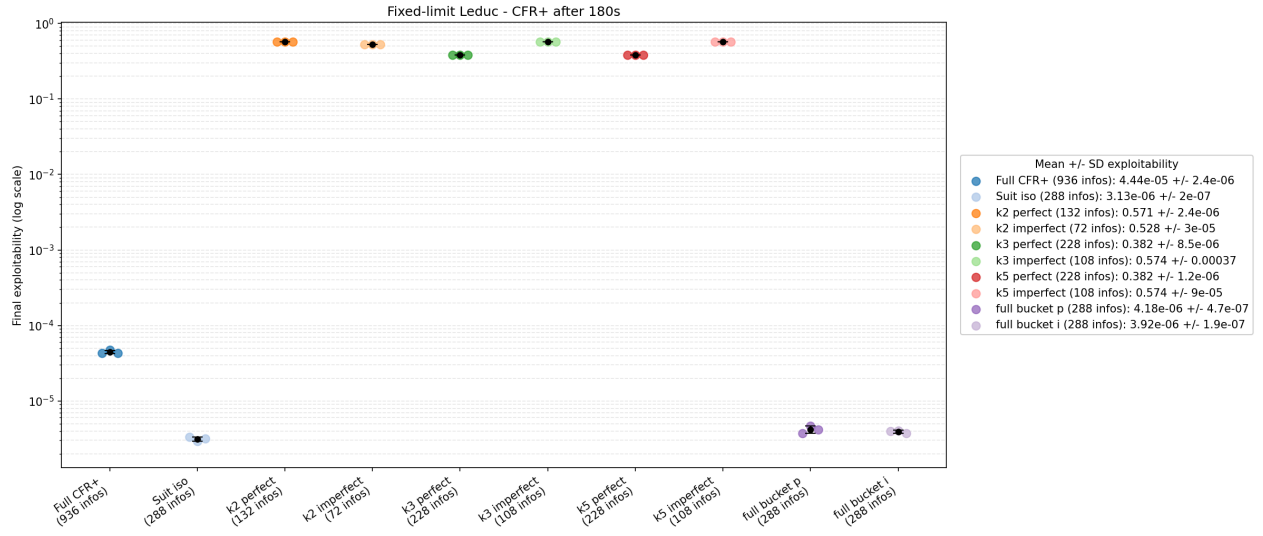


Figure 1: Fixed-limit Leduc CFR+ abstraction results

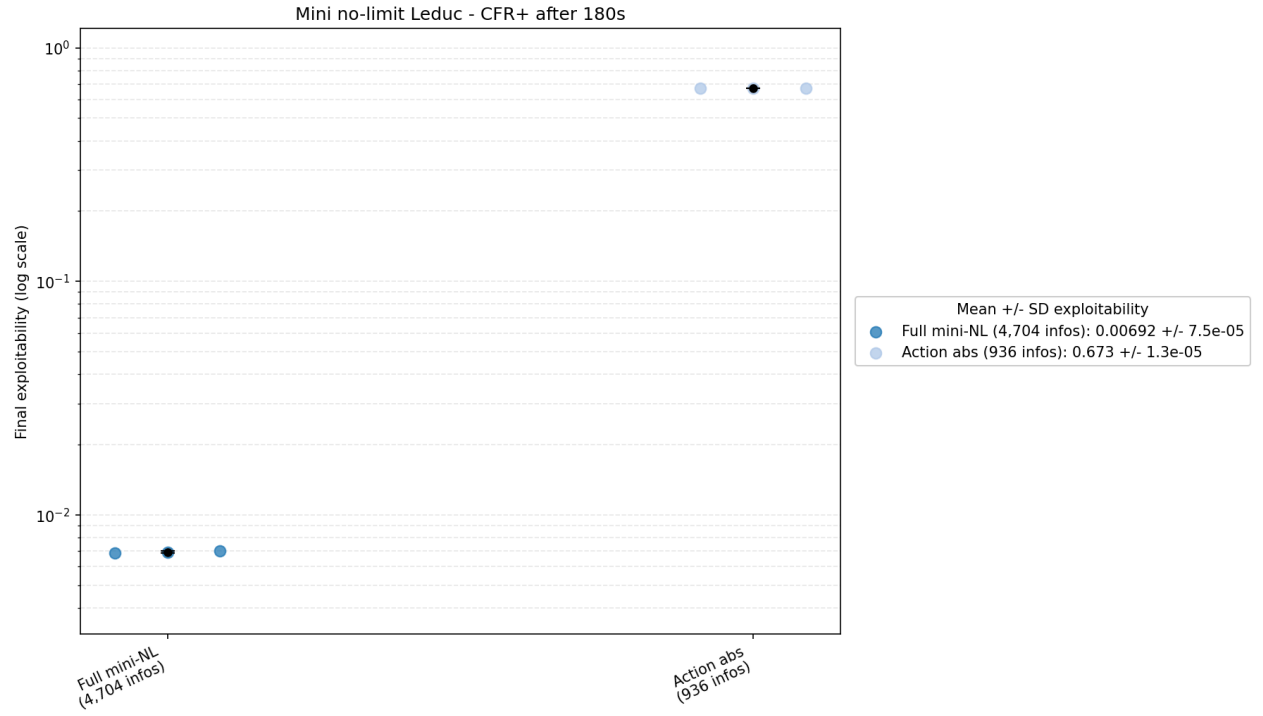


Figure 2: Mini-NL Leduc CFR+ abstraction results

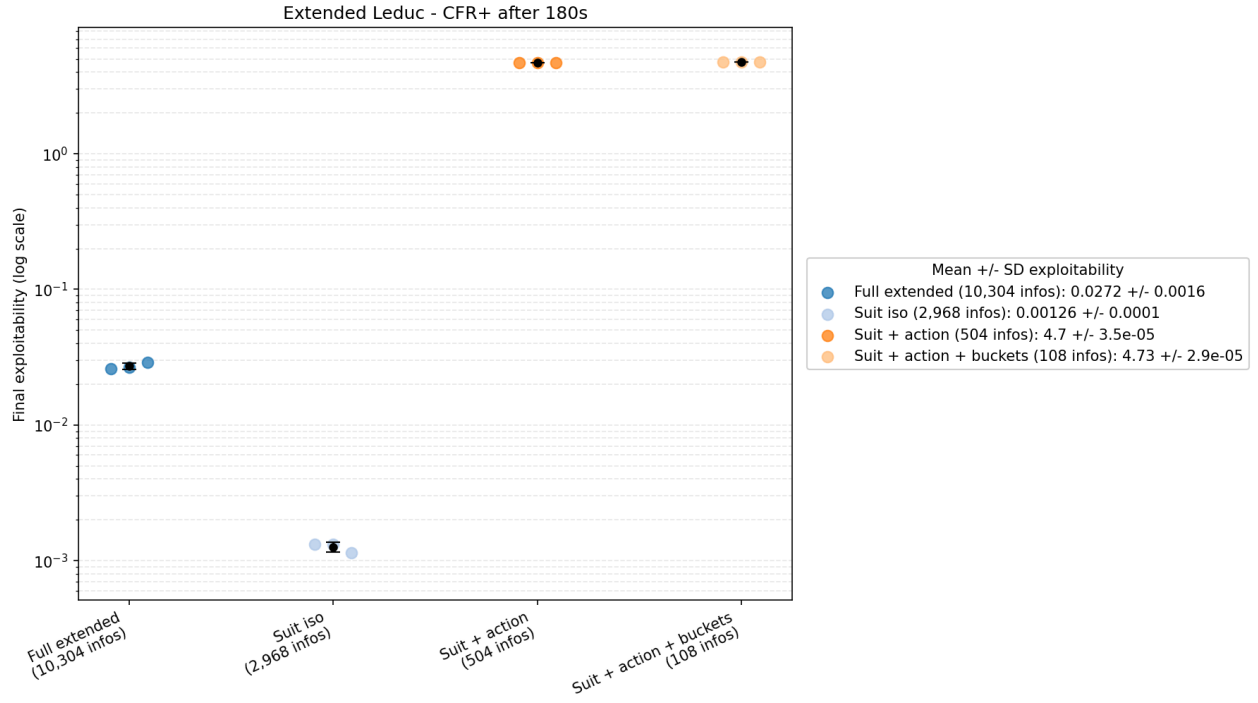


Figure 3: Extended Leduc CFR+ abstraction results

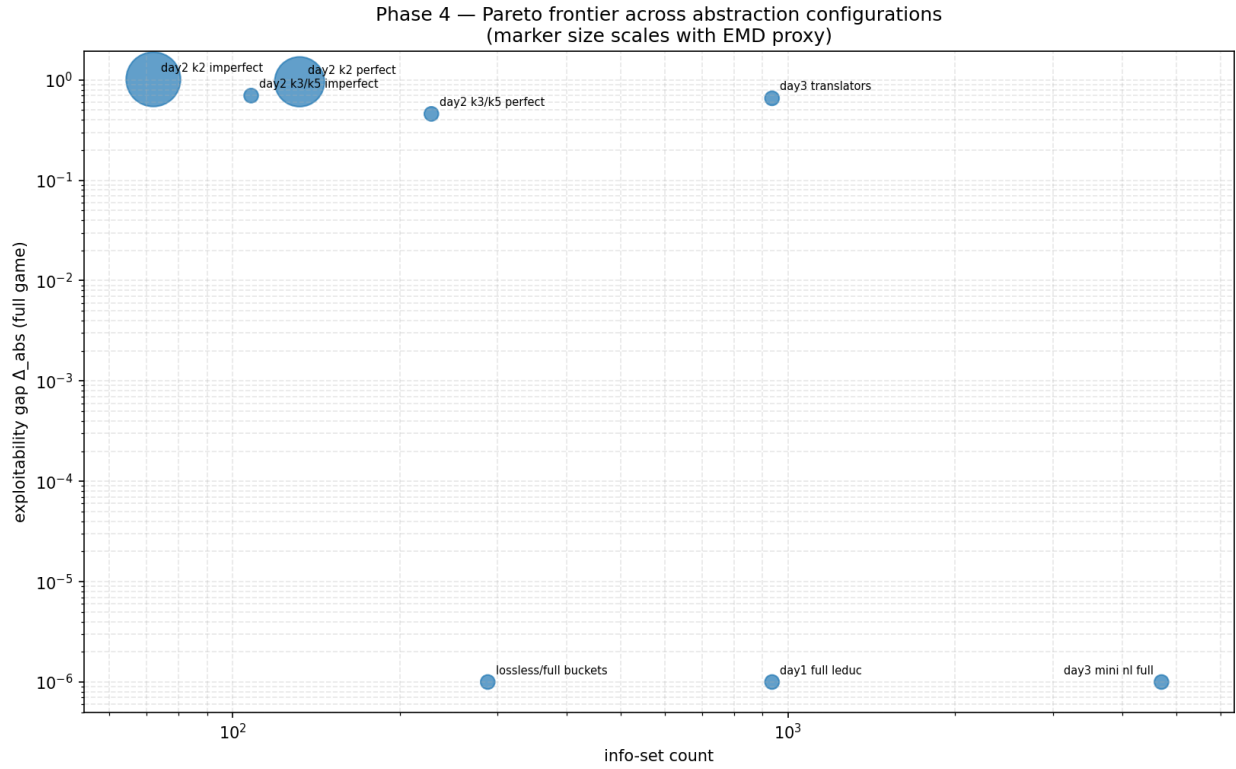


Figure 4: Abstraction Pareto frontier

1.4 4. Pareto frontier

The Pareto frontier summarizes the central Step 04 tradeoff: smaller abstract games train faster, but not every reduction is strategically safe.

Abstraction family	Result
Suit isomorphism	Best tradeoff; smaller game with no strategic loss in Leduc
Full rank buckets	Behaves close to suit isomorphism
Coarse lossy buckets	Smaller trees, persistent exploitability floor
Action abstraction	Largest risk; translation error dominates in Mini-NL and Extended Leduc

The practical criterion is therefore not just “smaller is better.” A useful abstraction must improve compute enough to compensate for the exploitability gap it introduces.

1.5 5. Cross-validation

`day06_openspiel_compare.py` aligns all 936 Leduc information sets against OpenSpiel and verifies that the custom implementation is in the same strategic regime. The remaining differences are interpreted as update-convention differences rather than exact trajectory mismatches, so this is a sanity check rather than a proof of identical solver dynamics.

1.6 6. Key learnings

1. **Lossless abstraction is the highest-value compression.** In Leduc-family games, suit isomorphism reduces the game substantially and improves wall-clock convergence without introducing abstraction error.
2. **Lossy information abstraction creates an exploitability floor.** Coarse card buckets solve faster but cannot recover distinctions they deliberately removed.
3. **Action abstraction is more dangerous than card abstraction in these experiments.** Restricted action sets plus translation produced high exploitability even when the abstract tree was much smaller.
4. **The Pareto frontier is the right reporting format.** Strategy quality must be presented together with game size and abstraction type.
5. **Subgame solving is the natural next mechanism.** Static translation is brittle; Step 6’s safe/nested solving methods are the production-grade response to off-tree actions.

1.7 7. Reproduction

From repo root:

```
cd implementation/step04/phase4
```

Smoke-test budget used by the phase-4 integration notes:

```
python day01_train.py --iterations 200
```

```
python day02_train.py --iterations 200
```

```
python day03_train.py --iterations 100
```

```
python day04_train.py --iterations 20000
```

```
python day05_train.py
```

```
python day06_openspiel_compare.py --iterations 200
```

Final CFR+ benchmark panels:

```
python day07_cfrplus_panels.py
```

The generated JSON/CSV files are written into `implementation/step04/phase4/`, and figures are written into `implementation/step04/phase4/figures/`.

Contents

1	Step 07 — Opponent Modeling in Imperfect-Information Games	1
2	PART I — EXPLORATION ON KUHN POKER	2
2.1	1. What this step is about	2
2.2	2. Experiment 1 — Behavioral fingerprints	2
2.3	3. Experiment 2 — The exploitation opportunity (why bother)	4
2.4	4. Experiment 3 — The Bayesian type detector (the core loop)	4
2.5	5. Experiment 4 — Recovering the blend (mixture modeling with EM)	8
2.6	6. What Part I establishes	9
3	PART II — FULL SYSTEM EVALUATION ON KUHN & LEDUC	10
3.1	7. What we test, and how	10
3.2	8. Detection — who is the opponent?	12
3.3	9. Exploitation — how much does modeling win?	12
3.4	10. Non-stationarity — adapting to a mid-match switch	14
3.5	11. Are the sample sizes adequate?	16
3.6	12. Limitations	17
3.7	13. Conclusions and research directions	17
3.8	14. Reproduction	18

1 Step 07 — Opponent Modeling in Imperfect-Information Games

Games: Kuhn Poker (3-card, 2-player) and Leduc Hold'em (6-card, 2-round) — the smallest exact-solvable imperfect-information testbeds, reusing the Step 02 Kuhn engine + CFR solver and the Step 03 Leduc engine. **PhD connection: Contribution #1 (Behavioral Adaptation Framework)** — an opponent model is the *sensor* that turns observed actions into an estimate of the opponent's strategy; the adaptive exploiter is a first *actuator*, which Step 08 formalizes into safe (KL-regularized) exploitation. **Scope of results:** Part I is a sequence of seeded Kuhn-Poker experiments that build intuition. Part II is a full evaluation on both games (5 seeds, 20 000-hand matches, Nash baselines at 200 000 / 40 000 CFR iterations). All figures are measured from these runs and are bounded, wherever possible, by *exact* analytical references rather than simulated ones.

How to read this report. Both parts follow the same arc: **what we test** → **how we test it** → **results** → **conclusions**. **Part I (§1–§6)** studies the type-based Bayesian model in isolation on Kuhn — what an opponent “type” looks like, why modeling is worth doing, and how the belief-update loop behaves (including where it fails). **Part II (§7–§13)** evaluates the full system — three opponent models and an adaptive exploiter — on Kuhn and Leduc. §14 lists reproduction commands.

2 PART I — EXPLORATION ON KUHN POKER

2.1 1. What this step is about

A **Nash-equilibrium** strategy is built never to lose in the long run, *no matter who it plays*. That safety has a price: it plays identically against a world champion and against someone who folds every time you bet. **Opponent modeling** is the act of *watching how a specific opponent actually plays* and updating a belief about their strategy, so we can **deviate from Nash to exploit their mistakes** — bluff more against someone who folds too much, value-bet thinner against someone who calls too much.

The technique explored first is the **type-based (Bayesian) model**: rather than learn a strategy from a blank slate, we keep a belief — a probability distribution — over a small set of predefined opponent **types**, and update it after every observed action via Bayes’ rule:

$$\text{posterior}(\text{type}) \propto \text{prior}(\text{type}) \times P(\text{observed action} \mid \text{that type})$$

An action a type calls impossible drives that type toward zero; an action it predicts well boosts it. Over many hands the belief concentrates on the best-fitting type, and we best-respond to it. All numbers in Part I are measured from seeded, reproducible Kuhn runs.

2.1.1 1.1 The opponent “type zoo”

Four ground-truth types serve both as the opponents we play against and as the candidate hypotheses the detector reasons over:

Type	Behavior
AlwaysCall	Calling station — never bets, never folds (checks an open pot, calls any bet).
TightPassive	Rock — only commits chips with the nuts (K); folds everything else. The most exploitable type.
LooseAggressive	Maniac — always bets / calls, regardless of hand.
Nash	Balanced equilibrium play (from the Step 02 CFR solver); mixes its actions; unexploitable.

2.2 2. Experiment 1 — Behavioral fingerprints

What we test. Does each type leave a distinct, recoverable signature in its actions?

How. Play each type against a 50/50 “prober” (to force every information set to be visited) and record the empirical $P(\text{bet})$ at each info set next to the type’s true $P(\text{bet})$.

Results. The four fingerprints are visibly distinct and the empirical frequencies track the true probabilities at well-visited info sets: **LooseAggressive** 1.0 everywhere; **TightPassive** 0 except with K; **AlwaysCall** 0 when checking into an open pot but 1.0 when facing a bet; **Nash** fractional and mixed.

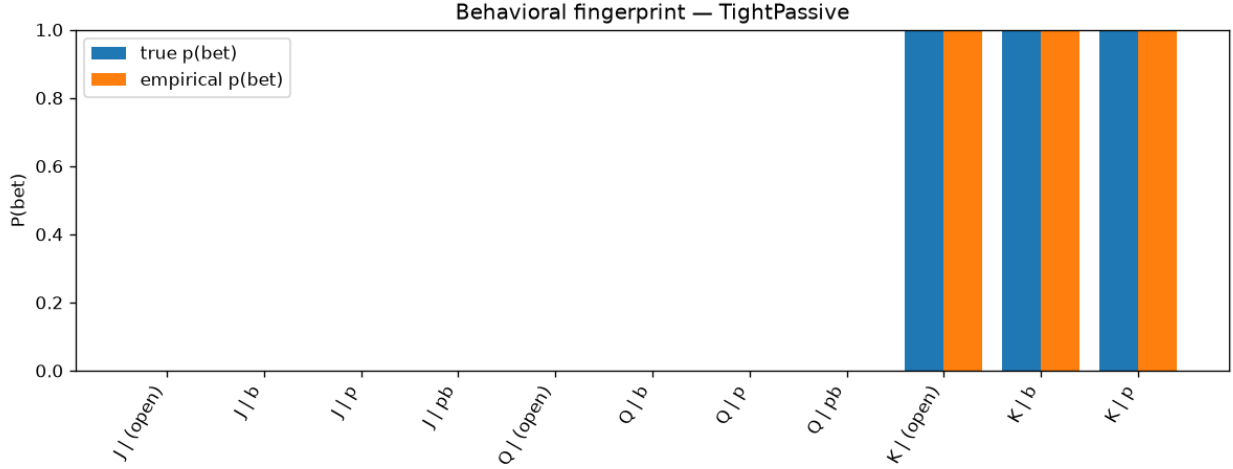


Figure 1: Behavioral fingerprint — TightPassive (bets only with the King).

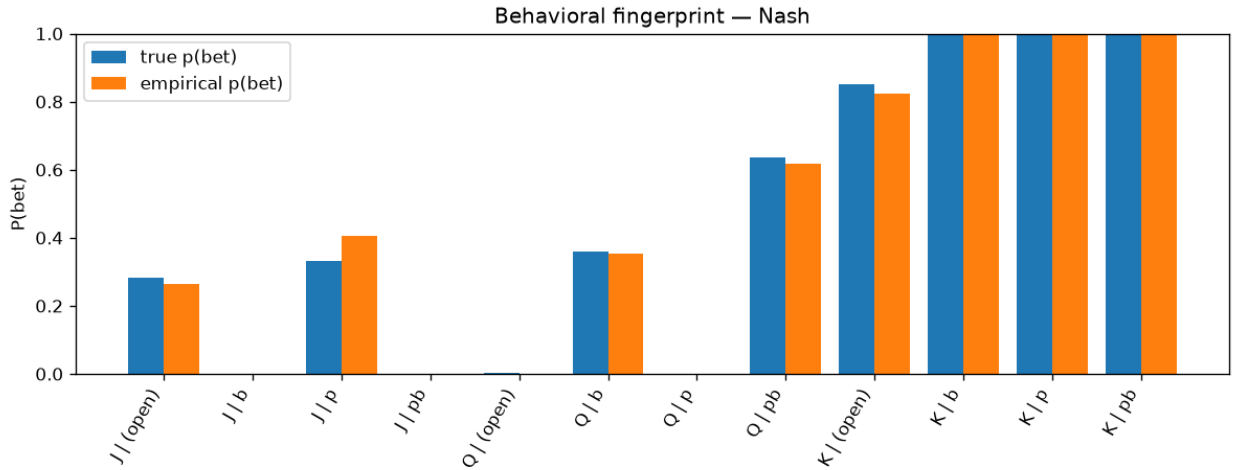


Figure 2: Behavioral fingerprint — Nash (mixed frequencies; empirical tracks true).

Conclusion. Recovering a type’s fingerprint from observed actions is the whole modeling task, and it is feasible — but info sets with few visits have noisy empirical frequencies, which is exactly what makes modeling hard on little data.

2.3 3. Experiment 2 — The exploitation opportunity (why bother)

What we test. How much value does modeling actually unlock over blind Nash play?

How. For each type and seat, compute *exactly* (i) the Nash hero’s expected value, (ii) the best-response EV (the exploitation ceiling, assuming the type is known perfectly), and (iii) the gap between them. The gap is the money Nash leaves on the table.

Opponent	Seat	Nash EV	Best-response EV	Gap
AlwaysCall	0	+0.119	+0.333	0.215
AlwaysCall	1	+0.220	+0.333	0.113
TightPassive	0	−0.060	+0.167	0.227
TightPassive	1	+0.055	+0.333	0.278
LooseAggressive	0	+0.121	+0.333	0.213
LooseAggressive	1	+0.116	+0.333	0.217
Nash	0	−0.055	−0.052	0.004
Nash	1	+0.055	+0.060	0.005

Results. Against the three exploitable types the gap is large (0.11–0.28 per hand); against **Nash the gap is 0** — as it must be, since Nash is unexploitable and best response cannot beat the game value. Concretely, the computed best response to **TightPassive bluffs**: it bets the weakest hand (Jack) into an open pot, because that type folds everything but the King.

Conclusion. The value of opponent modeling is real and measurable, and exploiting a rock means *bluffing more*. (The $-1/18 \approx -0.056$ seat-0 disadvantage of Kuhn’s first player explains the small negative Nash EVs — that is the game, not an error.)

2.4 4. Experiment 3 — The Bayesian type detector (the core loop)

What we test. Starting from an even prior, can the belief-update loop identify the opponent from behavior alone — and what happens when the opponent matches *no* type?

How. Maintain a posterior over the four types; after each observed opponent action multiply in its likelihood under each type and renormalize (in log-space, with a small $\epsilon = 0.02$ likelihood smoothing so no single “impossible” action permanently kills a candidate). Run several hidden opponents and watch the posterior evolve hand by hand.

2.4.1 4.1 In-set opponents — fast, correct, stable

When the hidden opponent genuinely **is** one of the four candidates, the posterior concentrates correctly and locks in:

- **Hidden = TightPassive:** posterior piles onto TightPassive and locks at 1.0 by hand ~18. (Nash is the slow-to-die rival, because a rock and equilibrium play overlap on many hands.)
- **Hidden = AlwaysCall:** locks by hand ~4 (a very distinctive type).

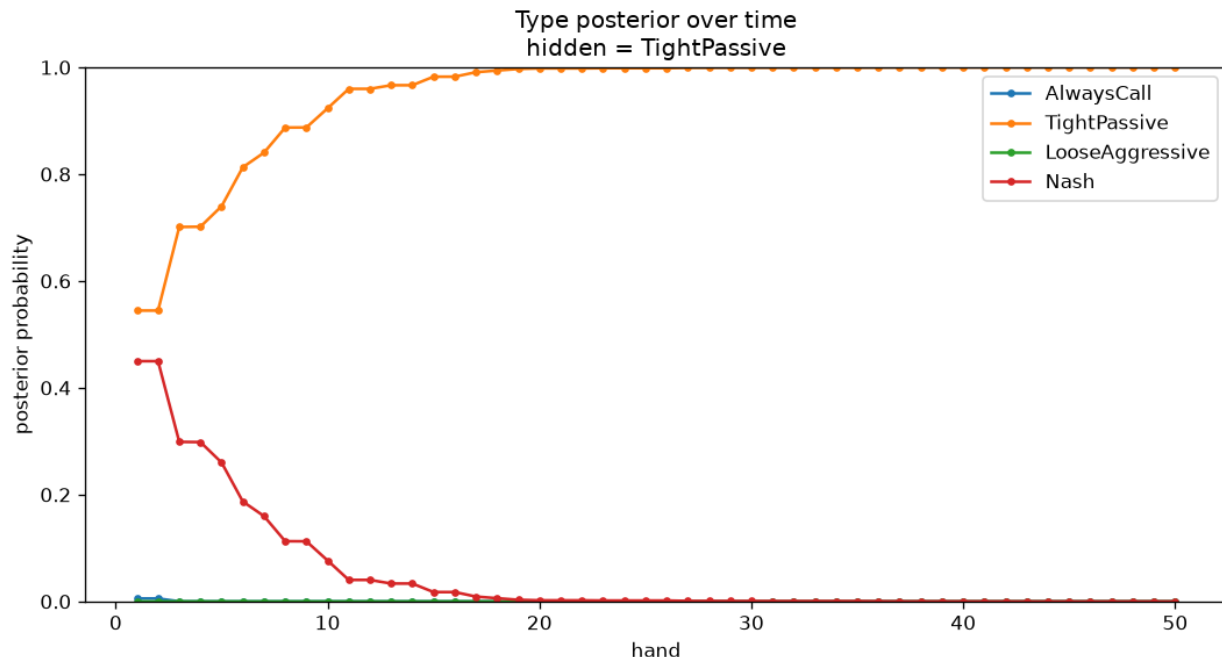


Figure 3: Hidden = TightPassive: clean convergence to the correct type.

2.4.2 4.2 Out-of-set opponent — “no honest home”

The instructive case is a hidden opponent that is a **50/50 per-action blend** of TightPassive and LooseAggressive — deliberately *none* of the four types. One might expect the posterior to “split between the two nearest types.” **It does not.** The detector commits hard to a *single* type at a time and jumps:

AlwaysCall (hands 1–2) → LooseAggressive (hands ~5–40) → **Nash (hand ~43+, 1.0).**

This is *correct* Bayesian behavior, and the reason is the central lesson of the phase:

- The blend plays a true (0.5, 0.5) on middling hands (J/Q) and always bets the King. Every time it bets a J/Q, the rock (which never does) takes a likelihood penalty; every time it checks a J/Q, the maniac (which always bets) takes one.

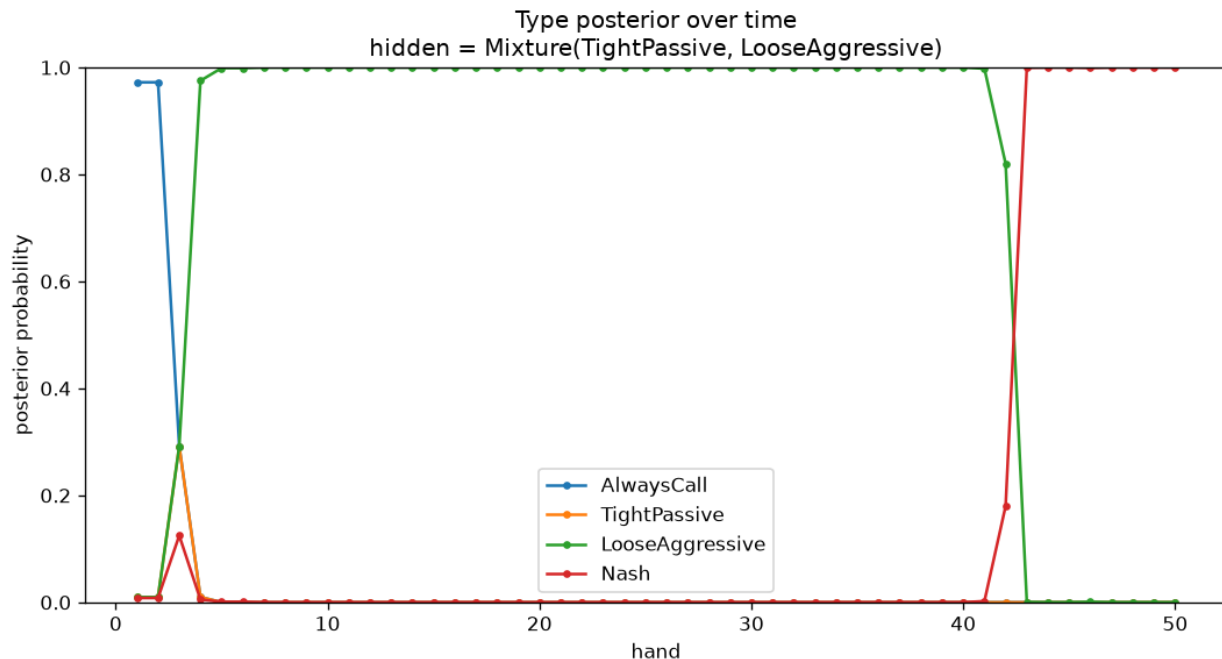


Figure 4: Hidden = Mixture(TightPassive, LooseAggressive): the belief thrashes, then lands on Nash.

- The two deterministic extremes are therefore each contradicted about half the time. **Nash is the only candidate that assigns real probability to *both* betting and checking a middling hand**, so it is the only story never fatally contradicted — and a product-of-likelihoods posterior concentrates on the single best explanation, not a blend.

Conclusion. A small stereotype menu has *no honest way to say “none of the above.”* Faced with an opponent it cannot represent, it commits — confidently — to the nearest *mixed* type rather than reporting genuine uncertainty. Two points sharpen this. First, the posterior is a **relative, normalized** quantity — the four bars are forced to sum to 1, so $P(\text{Nash})=1.0$ means “best fit *among these four*,” **not** “good fit in absolute terms.” Measuring the winner’s absolute fit (geometric-mean probability it assigns to each observed action) exposes the difference: the winning Nash model scores **0.74 per action against a true Nash opponent but only 0.31 against the mixture** — equally confident on paper, far worse underneath. Second, the honest fix is to stop asking “which one type?” and ask “what *mixture* of types?” — Experiment 4 (§5), and the motivation for the **continuous** and **consistent** models in Part II.

2.4.3 4.3 How robust is that convergence? — a 500-hand stress test

Because §4.2 lands on Nash, a natural question is whether, over a long match, the belief can still “fall” away from Nash. We ran the mixture scenario for **500 hands across 300 random seeds** and separated two phenomena a naive metric conflates — *slow convergence* versus *falling after*

convergence.

Measurement (300 seeds $\times$ 500 hands)	Result
Final winner at hand 500 is Nash	300 / 300 (100%)
After Nash <i>permanently</i> takes the lead, it later dips below 0.5	0 / 300 (never)
A <i>wrong</i> type still led past hand 100	40 / 300 (~13%)
A <i>wrong</i> type still led past hand 200	14 / 300 (~5%)
Hand at which Nash locks in for good	median 23 · 90th-pct 125 · worst 461

Results. Nash never *falls*: once it takes the lead for good it stays pinned at 1.0. The real risk is the opposite — **slow convergence with a confident wrong answer in the meantime**. In a sizeable minority of runs the maniac (LooseAggressive) owns the belief at 1.0 for well over a hundred hands before Nash overtakes, purely because an early run of bet-heavy hands looked exactly like “always bets.” The chart below (the most volatile seed found) shows LooseAggressive holding 1.0 from hand ~35 to ~220, then a sharp cliff to Nash for the remainder:

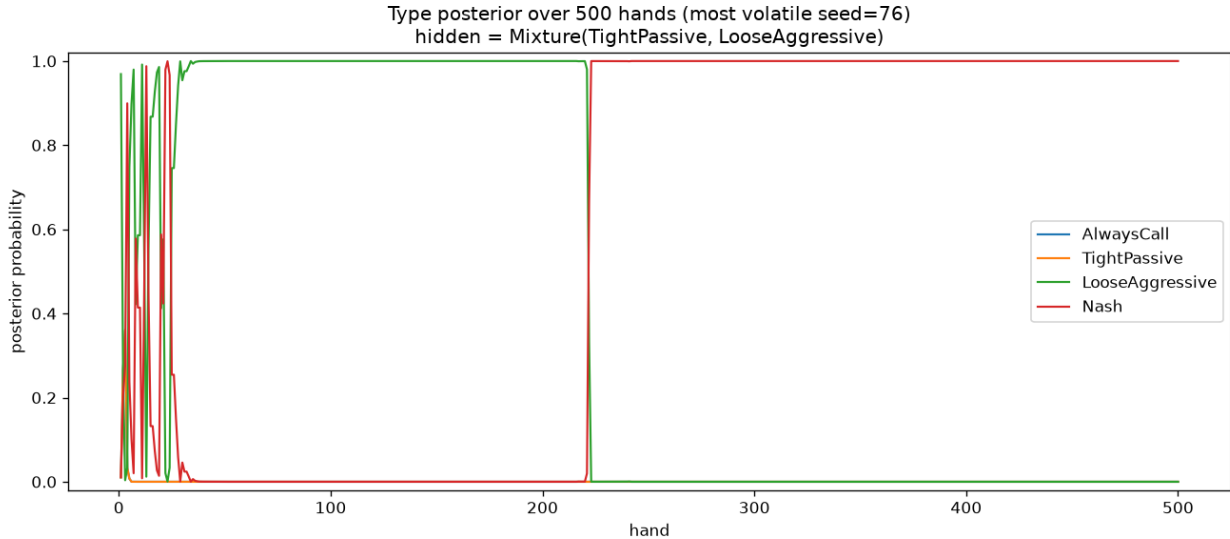


Figure 5: Hidden = Mixture, 500 hands (most volatile seed): a wrong type can dominate for 200 hands before Nash overtakes; once locked, Nash never falls.

Conclusion. Among the four candidates, Nash is the one *closest* (minimum KL divergence) to the true blended strategy, so every deterministic type eventually meets a contradiction it cannot survive. Nash is the unique long-run winner — “long run” just occasionally means 200+ hands. Best-responding hard to an early, confident-but-wrong belief would mean adjusting to beat a phantom.

2.5 5. Experiment 4 — Recovering the blend (mixture modeling with EM)

What we test. §4 showed that asking “*which single type are you?*” collapses a blended opponent onto the nearest mixed type (Nash) and hides a poor absolute fit. Can we instead ask “*what mixture of types are you?*” and recover the actual blend?

How. Fit **mixing weights** over the same four fixed types by **Expectation–Maximization**: each observed action is *softly credited* to the types in proportion to how well each explains it (E-step), and the weights are the average credit (M-step), iterated to convergence.

- E-step: $\text{responsibility}(\text{type} \mid \text{action}) = \frac{\text{weight}(\text{type}) \times P(\text{action} \mid \text{type})}{\sum \text{weight}(\text{type}) \times P(\text{action} \mid \text{type})}$
- M-step: $\text{weight}(\text{type}) = \text{mean responsibility over all observations}$

A tempting shortcut — a **hard** per-hand tally (pick the single best-fitting type each hand, then count) — does **not** work: it is confounded by *type overlap* (e.g. **AlwaysCall** and **TightPassive** both check a weak hand, so the argmax cannot separate them). The soft, iterated EM version deconfounds them.

Results. Re-fitting the weights after every hand (warm-started from the previous estimate) gives a per-hand trajectory — the mixture analogue of §4’s posterior-over-time charts. The two active components climb to their true weights while the other two decay to zero, whereas the old single-type posterior instead reports Nash 1.0 throughout:

Hidden					
opponent	Method	AlwaysCall	TightPassive	LooseAggressive	Nash
50 / 50 blend	global	0.00	0.00	0.00	1.00
	posterior (old)				
	EM weights	0.00	0.51	0.49	0.00
	@250 hands				
	<i>true blend</i>	0.00	<i>0.50</i>	<i>0.50</i>	0.00
70 / 30 blend	global	0.00	0.00	0.00	1.00
	posterior (old)				
	EM weights	0.00	0.66	0.34	0.00
	@250 hands				
	<i>true blend</i>	0.00	<i>0.70</i>	<i>0.30</i>	0.00

(The small residual error at 250 hands is sampling noise; a longer run tightens it — 4,000 hands)

gives 0.50/0.50 and 0.69/0.31.)

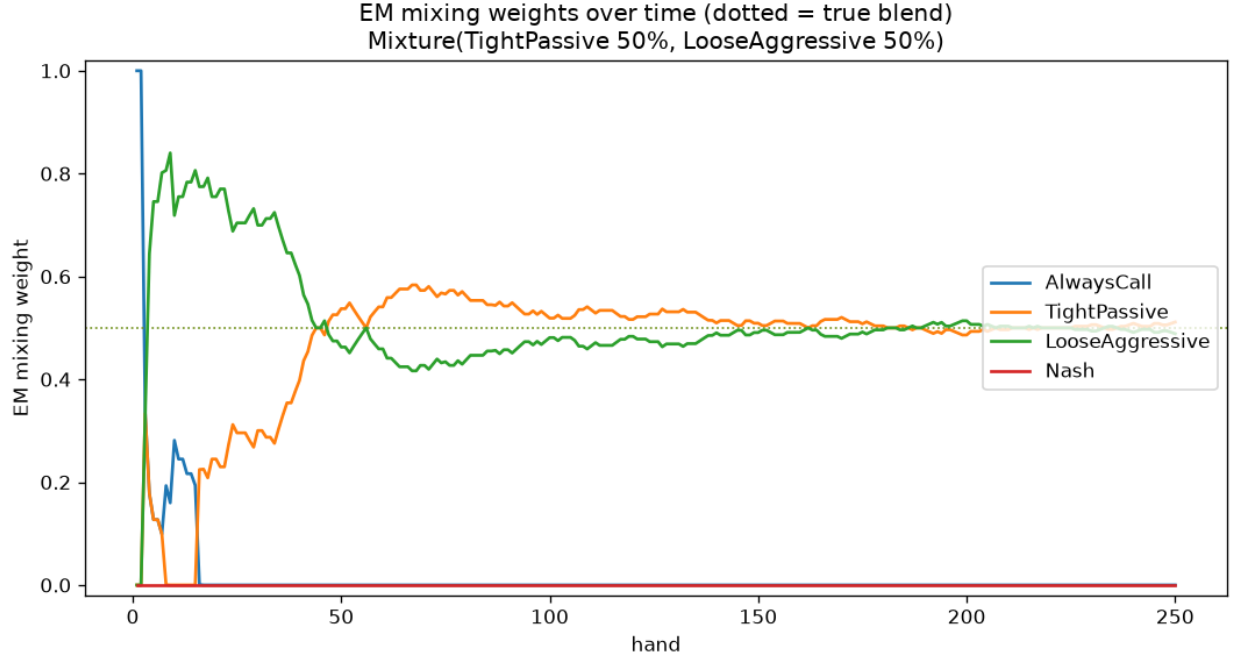


Figure 6: Recovering a 50/50 blend — EM mixing weights per hand. After early thrashing, TightPassive (orange) and LooseAggressive (green) converge together onto the true 0.50 line while AlwaysCall and Nash decay to zero.

Conclusion. EM recovers not only *which* types are present but *in what proportion* — an interpretable, honest description (“ 70 % rock + 30 % maniac”) where the single-type model gave a confident wrong answer. This mirrors opponent modeling’s foundational work (Bayes’ Bluff, Southey et al. 2005), which maintains a *distribution over* opponent strategies rather than committing to one. Part II generalizes this further — a **continuous** per-information-set estimator (a Dirichlet count per situation, which represents *any* strategy, including a blend) and a **consistent** sequence-form estimator (Ganzfried 2025). Experiment 4 is the smallest, most readable rung on that ladder.

2.6 6. What Part I establishes

The exploration surfaces, with a concrete measured face, the tension that Steps 07–08 exist to resolve:

1. **The value is real (Exp. 2):** blind Nash leaves 0.1–0.3 per hand on the table against exploitable opponents — modeling is worth doing.
2. **A model can be confident *and* wrong for a long time (Exp. 3):** in ~13% of 500-hand runs the detector confidently believed the wrong type past hand 100, and a stereotype menu

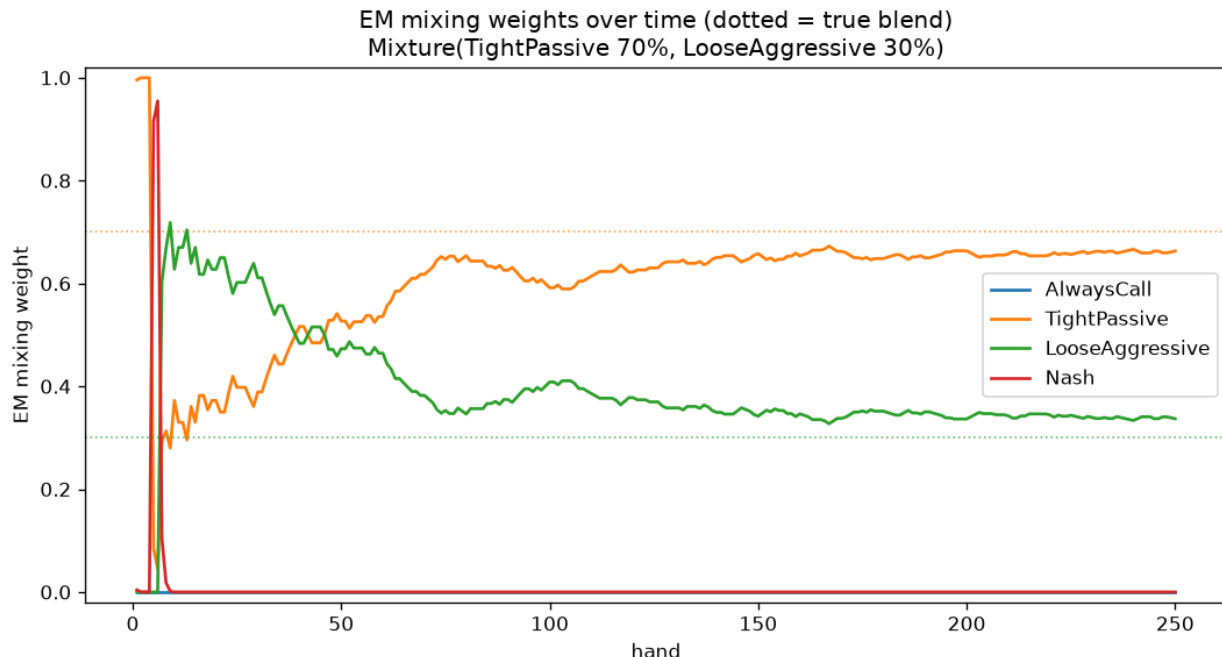


Figure 7: Recovering a 70/30 blend — the two active components settle onto their distinct true weights (dotted), recovering not just *which* two types but their proportions.

cannot represent an out-of-menu opponent (Exp. 3–4).

3. **Therefore exploitation must be regularized.** One must not best-respond to whatever the model currently believes; the lean toward exploitation has to be bounded (e.g. KL-regularized) and scaled to how *earned* the read is. Part I turns that abstract principle into a demonstrated failure mode; Part II measures it at full scale, and Step 08 builds the safe actuator.

3 PART II — FULL SYSTEM EVALUATION ON KUHN & LEDUC

3.1 7. What we test, and how

Part II evaluates a complete opponent-modeling system — **three interchangeable opponent models** feeding **one adaptive exploiter** — on both games, against exact analytical yardsticks.

The three models (all share a single interface: consume observed hands → emit a predicted opponent policy):

- **Type-based** — a Dirichlet-multinomial posterior over a fixed menu of opponent types (the Part I model, scaled up); reports a MAP type and a model-averaged policy.

- **Continuous** — a per-information-set Dirichlet estimate (count what the opponent did at each situation, smoothed). It can represent *any* strategy; hidden cards (folds) are handled by spreading soft/EM counts over the consistent deals.
- **Consistent** — the Ganzfried (2025) sequence-form maximum-a-posteriori estimate, solved as a small convex program. Estimating one globally consistent strategy (rather than each info set independently) is designed to use partial-observability evidence more cleanly.

The exploiter. Every k hands it rebuilds the hero policy as an exact best response to the model’s current estimate, optionally blended toward Nash by a safety weight, and optionally forgetting the model when a change is detected (Bayesian online change-point detection on an aggression signal). Because all three models feed the *same* exact best response, exploitation is compared apples-to-apples.

Exact yardsticks. Both games are small enough to solve exactly, so every simulated curve is bracketed by two analytical references computed by hand-verified math, not simulation: **nash_ev** (what safe equilibrium play earns against that opponent — you cannot do worse if the read is useless) and **ceiling** (the exact best-response value — the most that is extractable). A good exploiter sits between them and climbs toward the ceiling. The apparatus itself is trustworthy on two independent counts. *Internally:* our solver reproduces the exact Kuhn Nash value of $-1/18$ (NashConv 0.002, i.e. essentially unexploitable), and an exact best response beats uniform play by the predicted margins (Kuhn +0.500, Leduc +2.087). *Externally:* cross-validated against **OpenSpiel**, our best-response engine’s NashConv on the uniform policy matches OpenSpiel’s to full precision — Kuhn 0.916667 vs 0.916667 and Leduc 4.747222 vs 4.747222 ($|\Delta| = 0.000000$ on both, tolerances 0.001 / 0.01). Because NashConv = br + br in a zero-sum game, this compares the two engines with no info-state-string mapping at all.

The three studies.

1. **Detection** — can each model recover *who* it is playing? For the type-based model we report the posterior on the true type and whether the MAP is correct; for all models we report the mean total-variation (TV) distance between the estimated and true opponent strategy (lower = better recovery).
2. **Exploitation** — how much does best-responding to each model actually win, measured against the exact **nash_ev** and **ceiling**?
3. **Non-stationarity** — when the opponent switches style mid-match, does change-point forgetting recover faster than a model that never forgets?

Sample sizes. Detection: 5 seeds $\times$ 4 000 hands per type. Exploitation: 5 seeds $\times$ 20 000-hand matches, refitting every 200 hands. Non-stationarity: 5 seeds $\times$ 20 000-hand matches with the style switch at hand 10 000. Nash baselines use 200 000 (Kuhn) / 40 000 (Leduc) CFR iterations.

A note on type names. Part II uses a larger, per-game type zoo than Part I’s four hand-built

types: the Kuhn zoo is {AlwaysPass, AlwaysBet, TightPassive, LooseAggressive, Thresholdish, Nash, Random, Level1–3} and the Leduc zoo is {CallingStation, Maniac, Rock, LoosePassive, Nash, Random, Level1–3}. The names therefore differ from Part I — in particular Part I’s *AlwaysCall* (a calling station that never folds) is a different type from Part II’s Kuhn *AlwaysPass* (which checks/folds); the Part II calling-station lives in the Leduc zoo as *CallingStation*.

One deliberate caveat — the consistent model is not run in the exploitation loop. It refits its convex program from scratch each time the exploiter updates, and that cost grows with the number of hands already observed (from a fraction of a second early to ~12 s per refit near 20 000 hands), which makes a full online match impractical. We therefore evaluate the consistent model on **strategy recovery (detection)**, where its sequence-form MAP is designed to shine, and let the type-based and continuous models carry the exploitation comparison. (On Leduc the sequence-form program is large, so consistent detection is reported on Kuhn only.)

3.2 8. Detection — who is the opponent?

Type-based detection is essentially perfect: the posterior concentrates fully on the true type and the MAP is correct (TV distance 0.000) — with **one instructive exception**. On Kuhn the higher Level-k opponents collide: Level-2 and Level-3 converge to the *same* behavioral policy, so the posterior splits 0.50/0.50 between them and the MAP for the true Level-3 lands on Level-2. This is not an error — two types that play identically are genuinely indistinguishable *by their actions*, which is all a type detector observes. The **continuous** and **consistent** models, which estimate the actual policy rather than a label, recover both cleanly (TV = 0.005).

The **continuous** model’s mean TV to the truth is small on Kuhn (0.006–0.030) but larger on Leduc (0.11–0.36): Leduc has far more information sets and heavier partial observability, so a free-form per-info-set estimator needs many more hands to converge, and rarely-visited info sets dominate the residual error. The **consistent** model (Kuhn) matches or beats the continuous model’s recovery (TV = 0.004–0.021), as its sequence-form MAP is designed to.

Conclusion. Recovery works, and the model classes order as theory predicts: a well-specified type menu is unbeatable *when the opponent is in the menu*; strategy-estimating models are needed otherwise and pay a data cost that grows with the size of the game.

3.3 9. Exploitation — how much does modeling win?

Realized mean profit per hand over 20 000-hand matches ($\times 5$ seeds), against the exact best-response **ceiling**. Each table shows five representative types from that game’s zoo (10 Kuhn / 9 Leduc; the full set is in the results JSON). The type-based model tracks the ceiling almost exactly on both

games; the continuous model matches it on Kuhn and tracks close on Leduc, sitting below for the hardest-to-fit types (the undersampling of §8).

Kuhn (ceiling / type-based / continuous):

Opponent	ceiling	type-based	continuous
AlwaysPass	0.975	0.966	0.966
AlwaysBet	0.333	0.335	0.335
LooseAggressive	0.333	0.337	0.330
TightPassive	0.167	0.168	0.168
Nash	−0.055	−0.055	−0.053

Leduc (ceiling / type-based / continuous):

Opponent	ceiling	type-based	continuous
Level1	3.056	3.061	2.672
Maniac	2.177	2.199	2.038
CallingStation	1.464	1.451	1.434
Rock	0.937	0.912	0.848
Nash	−0.083	−0.085	− 0.175

Results — two carry the message:

1. **You cannot exploit an equilibrium.** Against the Nash opponent every model earns the (negative) Nash EV, never more — the exact ceiling *is* essentially the game value. This confirms the exploitation is real, not an artifact. (Where a realized number sits a hair *above* its ceiling — e.g. continuous −0.053 vs the −0.055 Kuhn Nash ceiling — the excess is finite-sample noise within ± 1 SE, not a real breach: the exact ceiling cannot actually be beaten.)
2. **A confident-but-underfit model makes *you* exploitable.** On Leduc the continuous model *loses* to Nash (−0.175 vs the −0.083 ceiling): with imperfect data it best-responds to a *wrong* estimate of an unexploitable opponent, opening a leak in its own play.

Conclusion. When the model class fits, best response reaches the exact extractable ceiling — modeling delivers the full theoretical value. When it does not (continuous on Leduc’s many info sets), two costs appear: money left on the table against exploitable opponents, and — more dangerously — a *negative* result against an opponent who cannot be exploited at all. That second cost is the exploitation-vs-safety tension in numbers, and the direct motivation for Step 08’s KL-regularized safe exploitation.

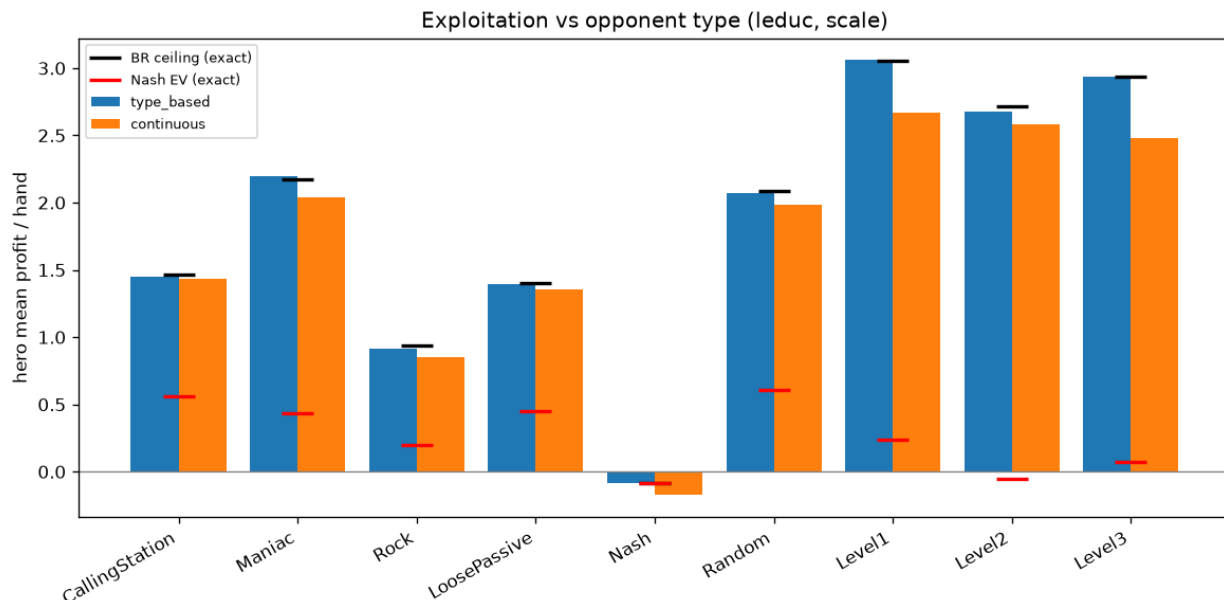


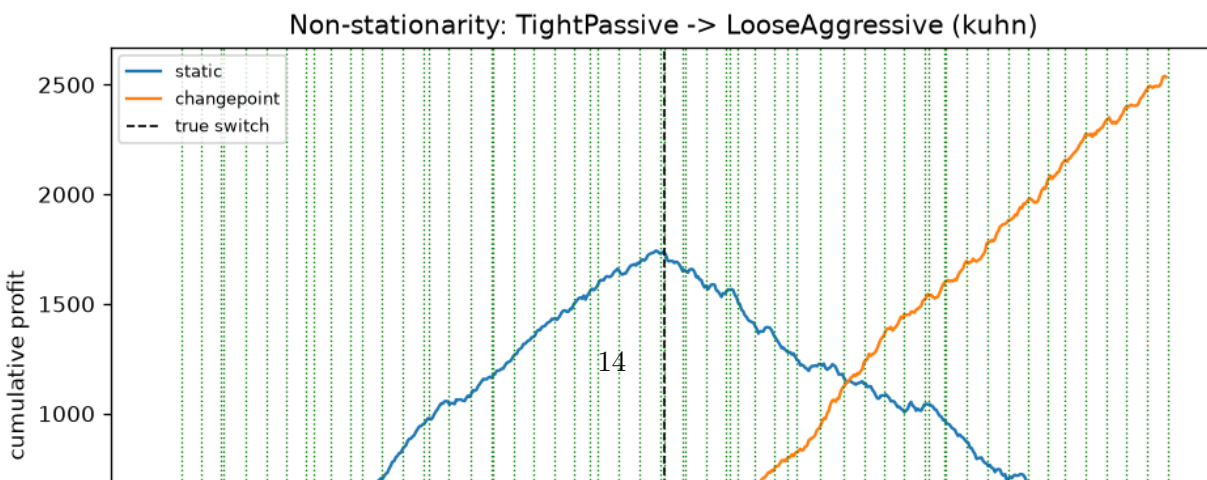
Figure 8: Exploitation vs opponent type (Leduc): type-based (blue) hugs the exact BR ceiling for every type; continuous (orange) tracks close but sits below for the hardest-to-fit types; both drop below zero against Nash.

3.4 10. Non-stationarity — adapting to a mid-match switch

What we test. The opponent switches style at hand 10 000 of 20 000. We compare a **static** continuous model against one with **change-point forgetting** (detect a style change → reset the model → drop to safe play → re-learn).

The result is **scenario-dependent**, which is itself the finding. Mean profit/hand *after the switch*, averaged over 5 seeds ($\pm$ standard error; every individual seed agrees on the sign):

Game	Switch (at hand 10k of 20k)	static (after switch)	change-point (after switch)
Kuhn	TightPassive → LooseAggressive	-0.106 ± 0.007	$+0.211 \pm 0.008$
Leduc	Rock → Maniac	$+1.834 \pm 0.054$	$+0.552 \pm 0.017$



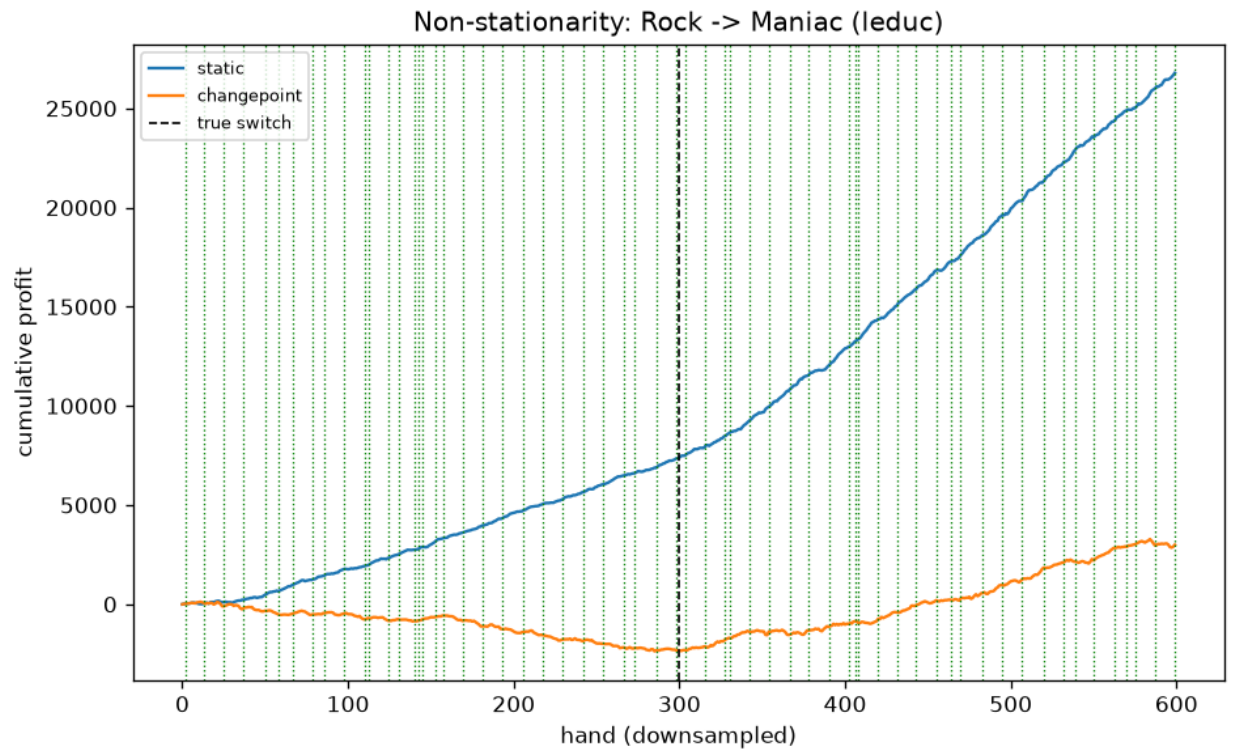


Figure 10: Non-stationarity (Leduc): here the static model wins — the Maniac is so exploitable that continuous adaptation captures it, and the detector’s false-positive resets cost more than they save.

- **Leduc — change-point loses.** The maniac leaks 2+/hand, so the continuously-adapting static model exploits it well without any reset (+1.83); meanwhile the detector fires many **false positives** during the stable phase (58–59 resets per seed against a single true switch), each dropping to safe play and discarding data, and lands three-and-a-half times lower (+0.55). Again the seed bands are cleanly separated.

Conclusion. Change-point forgetting rescues you exactly when a stale model is *harmful*, but its naive form — a low-signal detector that false-triggers, plus a full reset-to-safe on every detection — can underperform simple continuous adaptation when the new opponent is exploitable enough that staleness costs little. The *reaction* to a detected change matters as much as the detection: cheaper responses (partial forgetting instead of a hard reset; a less trigger-happy detector) are the clear next step.

3.5 11. Are the sample sizes adequate?

Because every experiment logs its per-seed values, adequacy can be quantified rather than assumed. The short answer: **adequate across all three studies** — comfortably so for exploitation and non-stationarity, and sufficient for the qualitative detection claims.

Exploitation — comfortably adequate. The standard error (SE) of the mean profit/hand across the 5 seeds is tiny relative to the effects claimed:

Game	typical SE (per-hand)	type-based gap to ceiling	continuous shortfall vs type-based
Kuhn	0.000–0.005	within 1 SE everywhere	1 SE (they coincide)
Leduc	0.007–0.040	within 1 SE everywhere	5–20 SE (e.g. Level1 3.061 vs 2.672, $\Delta = 0.389$ at SE 0.02–0.04)

So (1) the type-based model is **statistically indistinguishable from the exact ceiling** on every type in both games — “modeling reaches the ceiling” is not seed luck; and (2) the continuous model’s Leduc shortfall (§9) is a **real ~5–20 effect**, as is its Nash self-leak (−0.175 vs −0.083, a 0.092 gap at SE 0.015–0.06). The sample resolves every difference the report leans on.

Detection — adequate for the claims made. Type-based posteriors are saturated and the continuous/consistent TV distances are stable and ordered as predicted. We report point TVs

rather than error bars; a fuller version would repeat detection across seeds and publish TV confidence bands.

Non-stationarity — solid (5 seeds). Each variant is now run over the same 5 seeds. The after-switch standard errors are small and, in both games, the static and change-point bands are cleanly separated: Kuhn -0.106 ± 0.007 vs $+0.211 \pm 0.008$ (change-point wins by ~ 40 combined SE); Leduc $+1.834 \pm 0.054$ vs $+0.552 \pm 0.017$ (static wins by ~ 24 combined SE). Every individual seed agrees on the sign of the effect, so the two-sided finding of §10 is a measured result, not a single realization.

3.6 12. Limitations

Ranked by how much they qualify the conclusions:

1. **The change-point detector is too trigger-happy (§10).** ~ 1 false reset per ~ 330 hands in a stationary regime — a genuine defect in the naive detector + hard-reset design, and the reason change-point loses on Leduc. This is the clearest quality gap and a direct research hook, not a blocker.
2. **Consistent-model exploitation is unmeasured by design (§7).** Excluding it keeps the online matches tractable, but the exploitation comparison is type-based vs continuous only; the consistent model is characterised on recovery instead.
3. **Leduc consistent detection is deferred** — the sequence-form program is large there, so cross-game recovery for the consistent model rests on Kuhn.
4. **Well-specified detection is an easy case.** Every true type is also a candidate, so posterior $= 1.000$ is *expected*; it validates the machinery, not robustness to an opponent *outside* the menu — precisely where the continuous/consistent models should earn their keep (and where continuous visibly struggles on Leduc).
5. **Two small games, self-play data.** Kuhn/Leduc are exact-solvable *because* they are tiny; the partial-observability and undersampling effects that dominate Leduc will only intensify in larger games. This is a controlled proof-of-concept, not a scaling claim.

3.7 13. Conclusions and research directions

Conclusions. A good opponent model is *necessary but not sufficient* for profitable, safe adaptation. When the model class fits the opponent, best-responding to it reaches the exact extractable ceiling (type-based on both games; continuous on Kuhn) — the full theoretical value of modeling is realized. But under partial observability and limited data (continuous on Leduc), an underfit model both leaves money on the table and, against an unexploitable opponent, best-responds to a phantom

and *loses*. Non-stationarity adds a second edge: forgetting a stale model helps only when staleness is actively harmful, and a careless detector’s false resets can cost more than they save. Across all three studies the same principle recurs: exploitation must be scaled to how well-earned the read is.

Research directions (each now motivated by a measured effect above):

- **Safe (KL-regularized) exploitation.** The continuous model’s Nash self-leak (§9) is the exploitation-vs-safety tension made empirical — the concrete hand-off into **Step 08** and thesis **Contribution #2**: bound the best response’s deviation from Nash by the model’s own confidence, so an underfit model cannot open a leak.
- **Confidence-scaled forgetting instead of a hard reset (§10).** A forgetting factor, or a reset whose depth scales with detection confidence, should dominate the current reset-to-safe.
- **A better change-point signal (§10, §12).** Per-info-set likelihood-ratio monitoring or a multivariate detector over several behavioral features would cut the false-positive rate that currently sinks change-point on Leduc.
- **Misspecified / out-of-menu opponents (§12).** Because well-specified detection is saturated, the interesting regime is opponents *not* in the menu (mixtures, drift, adversarial). This is where the non-parametric and sequence-form models should separate from type-based, and where a “none of my types fit” detector (open-world type discovery) becomes its own question — the smallest version of which Part I (§4.2, §5) already exhibits on Kuhn.
- **Online/incremental consistent model (§7).** Warm-starting the convex solve from the previous fit and caching hand-terms incrementally could bring the sequence-form model into the online exploitation loop, turning a detection-only method into a usable real-time exploiter.
- **Undersampling-aware recovery (§8).** The Leduc continuous shortfall is dominated by rarely-visited info sets; priors that share statistics across related info sets (hierarchical / abstraction-based smoothing) are the natural fix and connect opponent modeling to the abstraction literature.

3.8 14. Reproduction

Part I — exploration experiments (from repo root, `.venv` active; each prints tables to stdout and writes figures under `implementation/step07/exploration/figures/`):

```
python implementation/step07/exploration/behavioral_fingerprints.py # Exp 1
python implementation/step07/exploration/exploitation_opportunity.py # Exp 2 (Nash EV vs BR EV)
python implementation/step07/exploration/bayesian_type_detector.py # Exp 3 (5 hidden opponents)
python implementation/step07/exploration/mixture_recovery.py # Exp 4 (EM, 50/50 & 70/30)
python implementation/step07/exploration/robustness_sweep.py # §4.3 sweep (500 hands ×
```

Part II — full evaluation (from `implementation/step07/implementation/`, `.venv` active;

needs numpy, scipy, matplotlib):

```
python tournament.py --config smoke      # quick Kuhn pass (~3 s)
python tournament.py --config scale      # full run: Kuhn ~2 min, Leduc ~14 min
python compare_openspiel.py              # OpenSpiel NashConv cross-validation (needs open_spiel)
```

Results write to `results/*.json` and plots to `plots/*.png`. *Figures reproduced in this report live in `deliverables/reports/step07/figures/` (exploration figures under their original names; evaluation figures prefixed `impl_`).*

Contents

1	Step 08 — Safe Exploitation in Imperfect-Information Games	1
2	PART I — THE TRADEOFF, MADE VISIBLE ON KUHN	2
2.1	1. What this step is about	2
2.2	2. Experiment 1 — the naive Nash/best-response frontier	3
2.3	3. Experiment 2 — the naive RNR budget sweep (and a flag)	3
2.4	4. What Part I establishes	6
3	PART II — FULL SYSTEM EVALUATION ON KUHN & LEDUC	6
3.1	5. What we test, and how	6
3.2	6. The exact method $\times$ opponent table (Kuhn)	7
3.3	7. The efficient frontier — and the bang-bang surprise	9
3.4	8. Prime-safe / adaptation and the measured -budget	11
3.5	9. The teaching attack — why realized profit is the wrong lens	12
3.6	10. Leduc — the headline finding: global safe-exploitation does not converge, but SES does	13
3.7	11. Are the results trustworthy?	15
3.8	12. Limitations	16
3.9	13. Conclusions and research directions	16
3.10	14. Reproduction	17

1 Step 08 — Safe Exploitation in Imperfect-Information Games

Games: Kuhn Poker (3-card, 2-player) and Leduc Hold'em (6-card, 2-round) — the smallest exact-solvable imperfect-information testbeds, reusing the Step 07 stack wholesale (the Step 02 Kuhn engine + CFR solver, the Step 03 Leduc engine, and Step 07's exact best-response / exploitability code and opponent-type zoo).

PhD connection: the **second half of Contribution #1 (Behavioral Adaptation Framework)** and the launch point for **Contribution #2 (Multi-Agent Safe Exploitation)**. Step 07 built the *sensor* (an opponent model that turns observed actions into a strategy estimate); Step 08 builds the *actuator* — turning that estimate into profit **without becoming exploitable** — and pins down exactly where the two-player zero-sum assumption enters the safety guarantees (the thesis attack point).

Scope of results: every number in this report is **measured from a real run** and read from the run artifacts under `implementation/step08/implementation/results/`, `.../plots/`, and `.../exploration/figures/`. Wherever possible a simulated number is bracketed by an *exact* analytical reference (the Nash game value, the exact best-response value) rather than another simulation. Two Phase-4 predictions were **contradicted** by the runs; per the project workflow they are kept as stated and reconciled with what actually happened (§7, §10).

How to read this report. Both parts follow the same arc: **what we test** $\rightarrow$ **how**

we test it → results → conclusions. Part I (§1–§4) makes the exploitation-safety tradeoff visible on Kuhn with two small seeded sweeps. **Part II (§5–§13)** evaluates the full system — one sequence-form LP engine and five safe-exploitation solver families — on Kuhn and Leduc against exact yardsticks. §14 lists reproduction commands.

2 PART I — THE TRADEOFF, MADE VISIBLE ON KUHN

2.1 1. What this step is about

A **Nash-equilibrium** strategy never loses in the long run against *anyone*, but it also never *punishes* a weak opponent — it plays identically against a world champion and against someone who folds every time you bet. Step 07 showed that a good opponent model unlocks 0.1–0.3 per hand against exploitable opponents, but also that best-responding to an imperfect model can open a leak in your own play (the continuous model *lost* to Nash on Leduc). **Safe exploitation** is the discipline that resolves that tension: lean toward exploiting the model, but bound how much a worst-case adversary can punish you.

Formally, every method in this step is the **same constrained optimization**:

maximize the hero’s expected value against the opponent model, **subject to** a safety floor on the hero’s worst-case value.

The expected value is *linear* in the hero’s sequence-form realization plan, so this is a linear program; the five method families differ only in **what the safety floor is**:

Method	Safety floor	Origin
Restricted Nash Response (RNR)	tunable via a parameter p (Nash at $p=0 \rightarrow$ best response at $p=1$)	Johanson 2007
Ganzfried	the Nash game value v^*	Ganzfried & Sandholm 2015
Prime-safe	$v^* - \epsilon$, with $\epsilon =$ the baseline’s own exploitability	Jeary & Turrini 2023
SES (subgame)	the blueprint value, enforced <i>locally</i> on one subgame via a gadget	Liu et al. 2022
Adaptation safety	worst-case blueprint worst-case (i.e. no more exploitable than the blueprint)	Ge et al. 2024

Part I probes the *naive* end of this menu — a behavioral blend of Nash and best response — precisely to show why the principled LP methods of Part II are needed. All Part I numbers are measured from seeded Kuhn runs; the hero is player 0, whose Kuhn game value is $-1/18 = -0.056$ (so “exploitation” means beating that value against a given opponent, and absolute numbers can stay negative against a tight opponent).

2.2 2. Experiment 1 — the naive Nash/best-response frontier

What we test. If you simply blend Nash and the full best response to a weak opponent — $(1-\alpha) \cdot \text{Nash} + \alpha \cdot \text{BR}$ — what tradeoff curve do you trace between *profit* and *your own exploitability*?

How. Sweep α from 0 (pure Nash) to 1 (pure best response) against the very exploitable `TightPassive` type, and compute, **exactly**, the hero’s profit (EV vs `TightPassive`) and worst-case loss (game value minus the hero’s worst-case value). Data: `exploration/figures/pareto_curve_kuhn.json`.

(weight on BR)	profit (EV vs TightPassive)	worst-case loss (exploitability)
0.0 (Nash)	−0.047	0.001
0.3	+0.017	0.133
0.5	+0.060	0.222
0.7	+0.103	0.311
1.0 (full BR)	+0.167	0.444

Results. The blend traces a smooth, monotone line: each increment of profit costs a near-proportional increment of exploitability. Full best response wins the most (+0.167) but is maximally exploitable (worst-case loss 0.444 — i.e. an adversary can drive the hero down to -0.5); Nash is unexploitable but wins nothing extra.

Conclusion. The tradeoff is real and continuous, but a uniform blend is a *blunt* instrument — it scales deviation everywhere at once. The value of Part II’s LP methods is that they choose *where* to deviate, buying the same profit at less exploitability (§7). This curve is the baseline they must beat.

2.3 3. Experiment 2 — the naive RNR budget sweep (and a flag)

What we test. The same blend, re-parameterized as the raw step’s informal “Restricted Nash Response”: scale deviation by p and watch profit and exploitability grow. Is there a *budget* of cheap early exploitation?

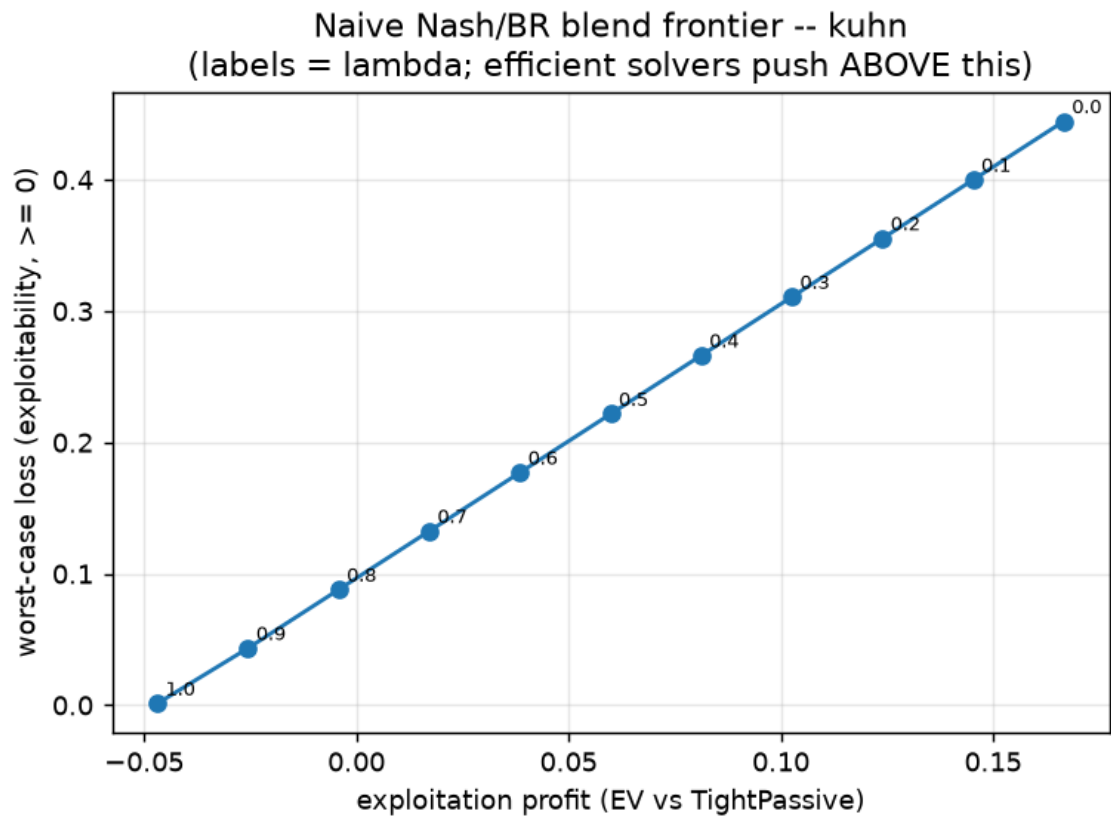


Figure 1: The naive Nash/best-response blend traces a smooth exploitation-safety frontier on Kuhn: every step toward more profit buys a near-proportional increase in your own worst-case loss.

How. Sweep $p \in \{0, \dots, 1\}$ against TightPassive; record profit and exploitability. Data: `exploration/figures/rnr_playground_kuhn.json`.

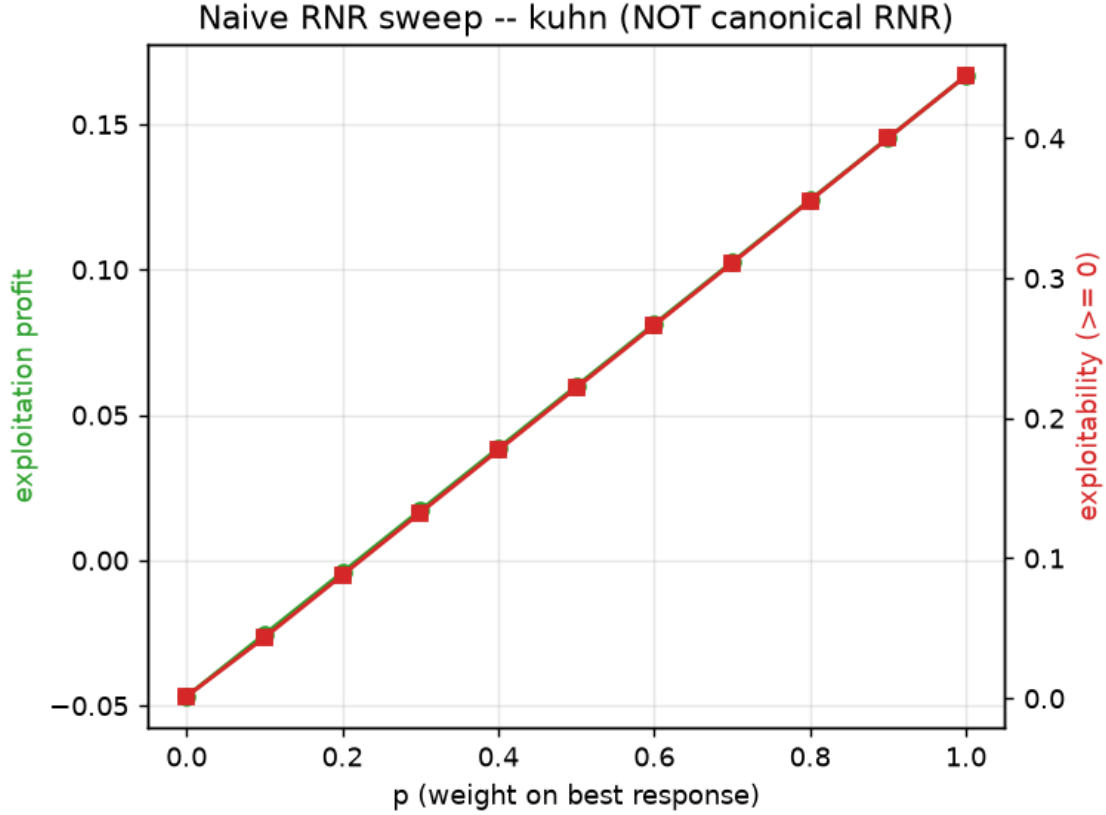


Figure 2: Naive RNR sweep on Kuhn: profit (left axis) and exploitability (right axis) both rise with p . This is the raw step’s informal blend — NOT Johanson’s canonical RNR, which Part II shows behaves very differently.

Results. Profit rises smoothly from -0.047 ($p=0$) to $+0.167$ ($p=1$); exploitability rises in lockstep from 0 to 0.444. There is a modest early “budget”: the first increments of p buy profit while exploitability is still small.

Flag (naive canonical RNR). This p -blend is the raw step’s Day-2 *description*, and it is a fine intuition tool, but it is **not** Johanson’s Restricted Nash Response. Canonical RNR solves for the *equilibrium against a p -restricted opponent* — a max-min linear program, implemented in Part II. Part II (§7) shows the two behave **completely differently** on Kuhn: the canonical version is bang-bang, not smooth. We keep both and label them, rather than conflate them.

Conclusion. The “budget” intuition holds for the naive blend, but Part II shows it is not how the principled algorithm behaves in a small game — a concrete example of why an intuition sweep must not be mistaken for the algorithm it illustrates.

2.4 4. What Part I establishes

1. **The exploitation-safety tradeoff is a real, measurable curve** (Exp. 1): more profit more of your own exploitability, near-proportionally, for the naive blend.
 2. **Full best response is dangerous** — maximally profitable against the model but maximally exploitable (worst-case -0.5 on Kuhn), which is the entire reason a safety floor is needed.
 3. **An intuition sweep is not the algorithm** (Exp. 2): the naive p-blend is smooth, but the canonical RNR it stands in for is not (§7). Part II replaces the blunt blend with LP methods that spend the exploitability budget *where it buys the most profit*.
-

3 PART II — FULL SYSTEM EVALUATION ON KUHN & LEDUC

3.1 5. What we test, and how

Part II evaluates the complete safe-exploitation system — **one sequence-form LP engine** and **five solver families** built on it — on both games, against exact analytical yardsticks.

The one engine. A `HeroTreeplex` enumerates the hero’s sequence-form realization plan (reusing Step 07’s `treeplex`), turns “EV against a fixed opponent” into a linear objective $\mathbf{c} \cdot \mathbf{x}$, and imposes the safety floor by **constraint generation (a double-oracle / cutting-plane loop)**: solve the LP $\rightarrow$ read the hero policy $\rightarrow$ call Step 07’s **exact best response** as the worst-case opponent $\rightarrow$ if the worst case is below the floor, add the linear cut and re-solve. The five families differ only in the floor (§1). This means one validated primitive (Step 07’s exact best response) powers both the objective and every safety check.

Exact yardsticks. Both games are small enough to solve exactly, so every solver’s reported profit and worst-case are computed on the full tree — no sampling. The two anchors are the **Nash game value** ($\mathbf{v}^*$ — the safe baseline) and the **exact full best-response value** (the exploitation ceiling). A safe method sits between them; an unsafe one has a worst-case below $\mathbf{v}^*$.

The methods (string ids as they appear in the result tables): **nash** (baseline, no adaptation), **full_br** (maximize EV vs the model, no safety), **rnr_0.5** (canonical RNR at $p=0.5$), **ganzfried** (floor = $\mathbf{v}^*$), **prime_safe** (floor = $\mathbf{v}^*-$), **adaptation** (floor = blueprint worst-case), **ses_subgame** (subgame gadget; on Kuhn the subgame is the whole game, so it coincides with an adaptation solve).

The three studies.

1. **Exact method $\times$ opponent table** — for each opponent type, solve each method against a

perfect model of that type and report exploitation EV (profit), worst-case value (safety), and whether the worst-case meets the Nash floor.

2. **The efficient frontier** — canonical RNR swept over p , the naive blend, and the Ganzfried / prime-safe / adaptation operating points, all on one exploitation-vs-exploitability plot.
3. **Teaching attack (online)** — a deceptive opponent baits with a weak style then reveals a strong one; a Step-7 model feeds each solver every k hands; we track realized profit and safety-violation counts.

Configurations and runtimes (measured). Kuhn **smoke** (30 000 CFR iters): **1.4 s**. Kuhn **scale** (200 000 CFR iters, adds **AlwaysBet/AlwaysPass/ses_subgame**, teaching attack over 5 seeds $\times$ 20 000 hands): **10.3 s**. Leduc **bounded_scale** (a human-added config: a 40-iteration cap on constraint generation, **leduc_tol** = 0.01, SES on the King-flop subgame): **minutes** (individual SES cells 10–79 s). Everything is CPU / LP-bound — CFR, full-tree best response, and small SciPy HiGHS linear programs — so the GPU is irrelevant, as predicted. Measured game values: Kuhn **v*** = -0.0556 ($-1/18$), Leduc **v*** = -0.0862 .

Not captured. No **validate.py** PASS/FAIL log and no OpenSpiel cross-check artifact were saved from these runs, so those specific harness outputs are not reported; the exact tables below substantiate most validation targets directly (§11).

3.2 6. The exact method $\times$ opponent table (Kuhn)

The core deliverable: each method solved against a perfect model of each opponent, scored on profit (EV) and safety (worst-case), with the Nash floor at **v*** = -0.056 . From **results/kuhn_scale.json**.

Opponent	metric	nash	full_br	rn timer	0.5	ganzfried	prime_safe	adaptation	ses_subgame
TightPassive	EV	−0.047	+0.167	−0.044	−0.044	−0.040	−0.040	−0.040	
	worst-case	−0.056	−0.500	−0.056	−0.056	−0.063	−0.063	−0.063	
LooseAggressive	EV	+0.118	+0.333	+0.278	+0.131	+0.151	+0.151	+0.151	
	worst-case	−0.056	−0.167	−0.111	−0.056	−0.063	−0.063	−0.063	
AlwaysBet	EV	+0.113	+0.333	+0.333	+0.115	+0.131	+0.131	+0.131	
	worst-case	−0.056	−0.167	−0.167	−0.056	−0.063	−0.063	−0.063	

Opponent	metric	nash	full_br	rn timer_0.5	ganzfried	prime_safe	adaptation	ses_subgame
AlwaysPass (most ex- ploitable)	EV	+0.146	+0.975	+0.975	+0.222	+0.266	+0.266	+0.266
	worst-case	−0.056	−0.500	−0.333	−0.056	−0.063	−0.063	−0.063
Nash (control)	EV	−0.056	−0.055	−0.056	−0.056	−0.055	−0.055	−0.055
	worst-case	−0.056	−0.333	−0.056	−0.056	−0.063	−0.063	−0.063

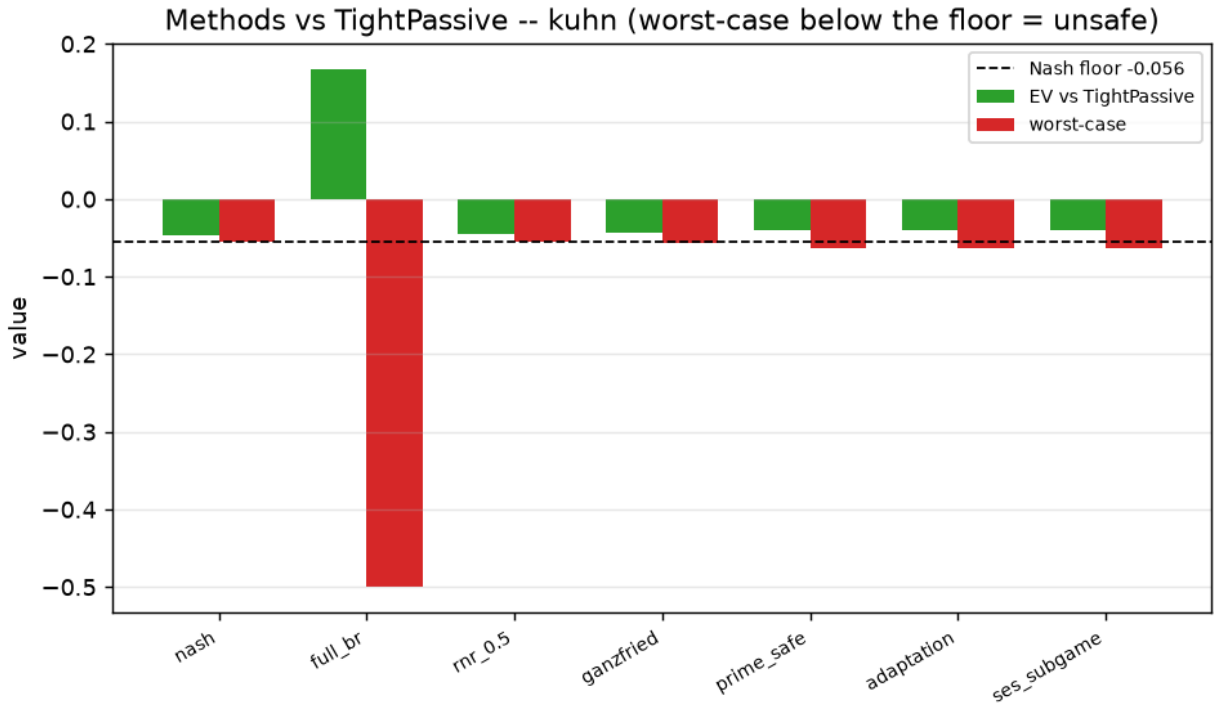


Figure 3: Methods vs TightPassive (Kuhn): green = EV vs the opponent, red = worst-case value, dashed = the Nash floor. Only full_br’s worst-case (red) plunges far below the floor; every other method hugs it.

Results — three carry the message.

1. **full_br is the cautionary tale.** It always wins the most against a fixed model (up to **+0.975** vs AlwaysPass), but its worst-case collapses to **−0.5** — an adversary can punish it

catastrophically. This is the danger the whole step exists to prevent.

2. **ganzfried is the safe-and-profitable sweet spot.** Against *every* opponent its worst-case stays at the Nash floor (safe within 1e-3) **and** it beats Nash’s own EV on every exploitable type — most strikingly **+0.222 vs Nash’s +0.146** against **AlwaysPass**, and +0.131 vs +0.118 against **LooseAggressive**. This is the central validated result of the step: you can exploit meaningfully while provably never dropping below equilibrium value.
3. **rn timer 0.5 is safe only for mild leaks.** Against **TightPassive** it is safe, but against the highly exploitable **AlwaysPass/AlwaysBet** it has *already jumped* to full best response (EV identical to **full_br**, worst-case $-0.33 / -0.17$, unsafe). The RNR transition is **opponent-dependent** — a single global p is not a safety setting (§7).

prime_safe / adaptation / ses_subgame coincide on Kuhn (all target the same -adjusted / blueprint floor, and Kuhn’s SES subgame is the whole game). They sit at worst-case $-0.063 = \mathbf{v^*} - 0.008$, flagged “unsafe” *only* because the flag compares to $\mathbf{v^*}$ while they legitimately target a lower floor by design (§8).

A measurement-resolution note. In the **smoke** run (30 000 CFR iters) even the Nash policy is flagged “unsafe” (worst-case -0.0568 , i.e. 0.0013 below the exact $\mathbf{v^*}$, just over the 1e-3 tolerance) — a pure CFR-approximation artifact. At **scale** (200 000 iters) Nash’s shortfall drops to 3e-4 and it is correctly flagged safe. Lesson for the harness: the safe-flag tolerance must exceed the baseline’s own approximation error, or the baseline trips its own test.

3.3 7. The efficient frontier — and the bang-bang surprise

Sweeping the canonical RNR parameter p against **TightPassive**, alongside the naive blend and the three LP operating points (from **results/pareto_kuhn.json**):

	canonical RNR —	canonical —	naive blend —	naive —
p	EV	exploitability	EV	exploitability
0.0	−0.044	~0.000	−0.047	0.000
0.3	−0.044	~0.000	+0.017	0.133
0.6	−0.044	~0.000	+0.081	0.266
0.7	+0.167	0.444	+0.103	0.311
1.0	+0.167	0.444	+0.167	0.444

Operating points: **ganzfried** (EV -0.044 , exploitability 0.0005); **prime_safe / adaptation** (EV -0.040 , exploitability 0.0079); measured $\Delta = 0.0074$.

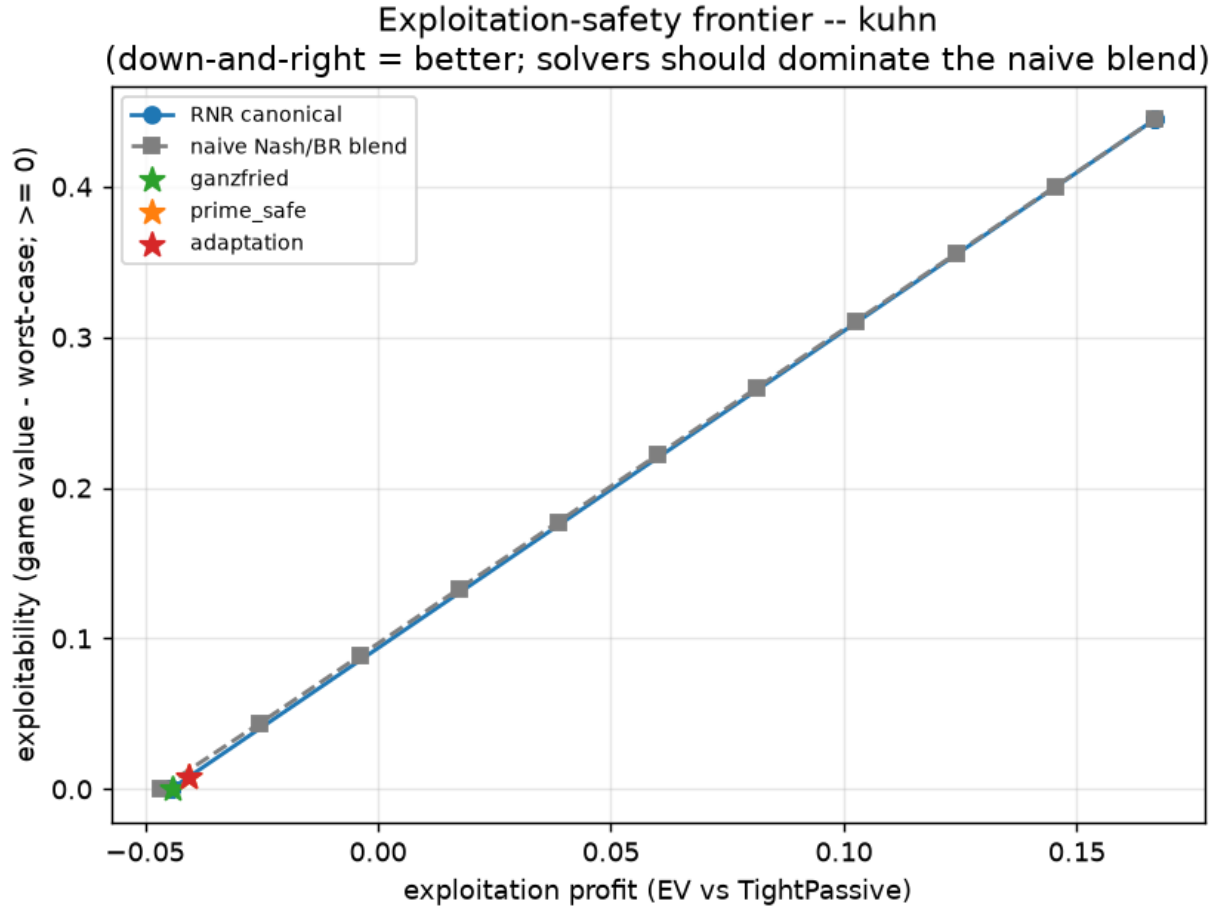


Figure 4: Exploitation-safety frontier (Kuhn). The canonical RNR “curve” is an interpolation between just two achieved clusters (safe corner and full-BR corner); the naive blend traces the smooth line; the LP operating points (stars) sit at the efficient safe corner.

3.3.1 Prediction reality: canonical RNR is bang-bang, not a smooth frontier

- **Prediction (Phase 4):** “the RNR sweep is monotone, and canonical RNR dominates the naive blend everywhere.”
- **What actually happened:** canonical RNR is a **step function**. For $p \in [0, 0.6]$ it returns the *same* safe strategy (EV -0.044 , exploitability 0); at $p = 0.7$ it jumps straight to the *full* best response (EV $+0.167$, exploitability 0.444). There are **no intermediate points**.
- **Why (verified reasoning).** The canonical RNR objective $\max_x [p \cdot \text{EV}(\text{model}) + (1-p) \cdot \min_{x'} \text{EV}(x, x')]$ is *linear* in the hero realization plan over a *polytope*, so its optimum is a **vertex** that switches only when p crosses a critical ratio. Kuhn’s strategy polytope is tiny (few vertices), so the transition is a single jump. Johanson’s smooth RNR frontier is a large-game / data-biased phenomenon (many vertices, or a per-info-set p); it does not appear in a game this small. The **naive blend** *does* trace a smooth line — precisely because it interpolates two fixed strategies — but it is dominated at the safe corner (at exploitability 0 the LP methods reach EV -0.044 vs the blend’s -0.047) and buys its interior profit at a steep exploitability cost.
- **The takeaway survives, refined.** “Choose *where* to deviate, not *how much* uniformly” still holds — Ganzfried sits at the efficient safe corner and the naive blend is dominated there. But the *mechanism* (“a smooth tunable RNR knob”) is a big-game artifact; in Kuhn the honest picture is a discrete safe-vertex $\rightarrow$ BR-vertex switch, whose threshold moves with how exploitable the opponent is (§6: `rnr_0.5` is already at full BR vs `AlwaysPass`). This is exactly why Ganzfried — which constrains the *value* rather than a p — is the better primitive.

3.4 8. Prime-safe / adaptation and the measured -budget

Prime-safe and adaptation lower the safety floor from v^* to $v^* - \epsilon$, where ϵ is the baseline’s own exploitability, measured from an early-stopped CFR run (never fabricated). The run measured $\epsilon = 0.0074$, and the prime-safe/adaptation worst-case came out at $-0.063 = v^* - 0.0079$ across every opponent — matching the ϵ -adjusted floor to rounding. Spending that budget, they earn slightly more than Ganzfried: **+0.266 vs +0.222** against `AlwaysPass`, **+0.151 vs +0.131** against `LooseAggressive` (§6). They are flagged “unsafe” in the table only because the flag references v^* ; against their *own* ϵ -floor they are safe by construction. Prime-safe and adaptation coincide here because, for this baseline, $v^* - \epsilon = \text{worst_case_value}(\text{blueprint})$ — the two floors are equal (a fact the reading phase flagged, now confirmed by the run rather than assumed).

Conclusion. The prime-safe mechanism works exactly as designed: an imperfect (ϵ -exploitable) baseline is honestly *measured*, and the safety floor is relaxed by precisely that amount — converting Ganzfried’s “needs a perfect Nash” guarantee into one usable with the approximate baselines any real system has.

3.5 9. The teaching attack — why realized profit is the wrong lens

What we test. A deceptive opponent plays the weak **TightPassive** bait for the first 10 000 hands, then switches to a strong **Nash** “reveal” for 10 000 more; a Step-7 continuous model feeds each solver every 500 hands (5 seeds). Does the safety floor protect the safe methods when the opponent turns on them? From `results/kuhn_scale.json`.

method	mean/hand (all)	mean/hand (after switch)	safety violations / seed
full_br	+0.051	−0.061	40, 40, 40, 40, 40
ganzfried	−0.048	−0.055	0, 0, 0, 0, 0
adaptation	−0.046	−0.055	40, 40, 40, 40, 40
nash	−0.051	−0.061	0, 0, 0, 0, 0

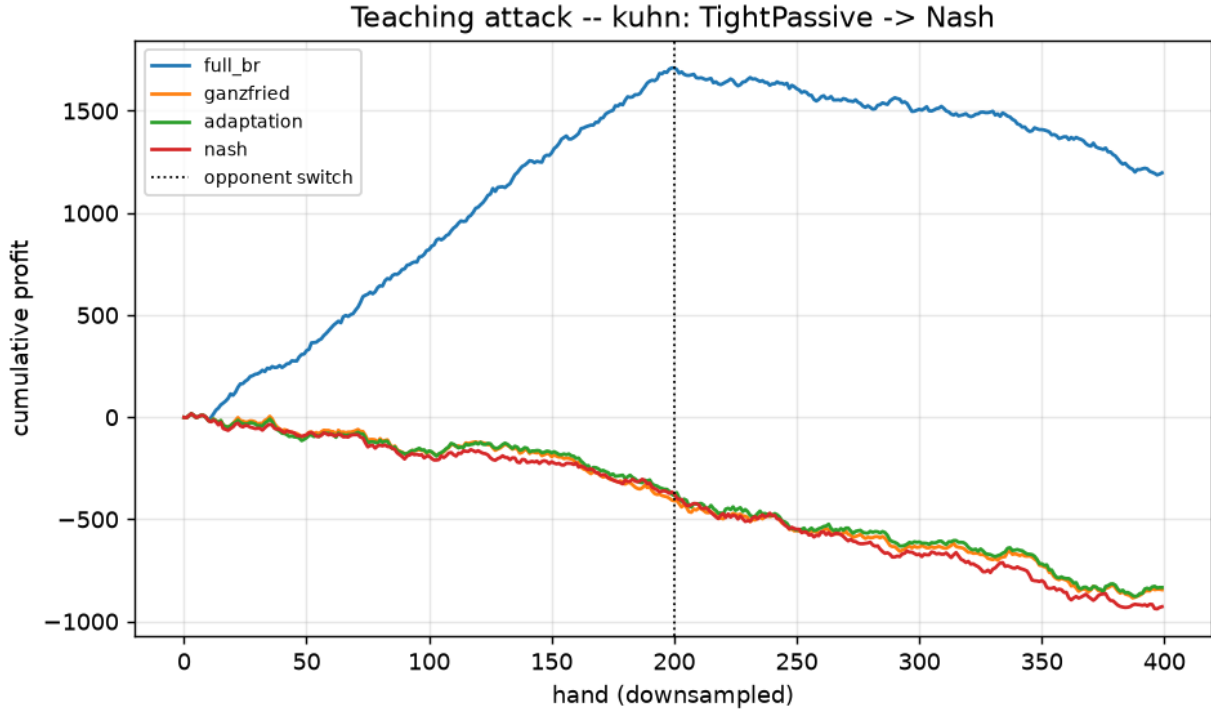


Figure 5: Teaching attack (Kuhn), cumulative profit. `full_br` (blue) climbs on the bait to +1700, then only drifts down after the switch — it ends far ahead because a Nash “revealer” claws back only the game value per hand. The safe methods refuse the bait and pay the P0 tax throughout.

3.5.1 Prediction reality: a Nash revealer is too gentle to punish full_br

- **Prediction (Phase 4):** “safe methods’ post-switch mean stays near baseline; full_br’s is clearly worse; safety violations = 0 for safe methods, > 0 for full_br.”
- **What actually happened.** The *safety-violation* half held **cleanly**: full_br violated the Nash floor at **40/40** refits, ganzfried at **0/40**. But full_br’s post-switch rate (−0.061) is only *marginally* worse than the game value (−0.056), and it ends **hugely net-positive** overall (+0.051/hand) — because the “reveal” opponent is *Nash*, which only wins back the game value per hand, so the bait-phase windfall is never clawed back within 10 000 hands.
- **Why (verified reasoning).** A strategy’s worst-case is realized by an opponent that *best-responds to it* (that is what drives full_br to −0.5 in §6). A stationary Nash reveal is **not** that adversary, so the online realized-profit experiment never triggers the risk that the exact worst-case column exposes. Measuring the teaching attack by profit-vs-Nash therefore *understates* the danger.
- **Corrected takeaway.** The honest separating signal here is the **exact worst-case / safety-violation count** (full_br 40, ganzfried 0), not realized profit. To make the teaching attack punish full_br *in profit*, the reveal must be an **adaptive counter-exploiter** (a best response to the exploiter’s stale model), not a fixed Nash. That is the concrete refinement for the next run (§13).
- **A design nuance, not a bug.** adaptation shows 40 violations while nash shows 0 (at scale): adaptation deliberately targets a floor *below* v* (§8), so its played strategy trips the v*-referenced counter every refit — by design. Nash’s 0 (at scale) confirms the smoke-run’s 10 “violations” were the CFR-resolution artifact of §6.

3.6 10. Leduc — the headline finding: global safe-exploitation does not converge, but SES does

The Leduc run is the human-extended **bounded_scale** config: a **40-iteration cap** on the constraint-generation loop, `leduc_tol = 0.01`, and the SES subgame set to the King-flop (`leduc_flop_rank(King)` — exploit only after a King appears on the board). Each cell records whether the solve **converged** or hit the cap (**capped**). From `results/leduc_bounded_scale.json`; Nash floor `v* = −0.086`.

Opponent metric		nash	full_br	ses_subgameganzfried	prime_safe	adaptation
Rock	EV	+0.201	+0.937	+0.247	+0.624	+0.635
	worst-case	−0.089	−1.633	−0.130	−0.838	−0.744
	converged?			(194 it)	capped	capped

Opponent metric		nash	full_br	ses_subgame	ganzfried	prime_safe	adaptation
Maniac	EV	+0.438	+2.177	+0.682	+1.806	+1.842	+1.842
	worst-case	−0.089	−1.100	−0.130	−0.638	−0.657	−0.657
	converged?			(211 it)	capped	capped	capped
LoosePassive	EV	+0.449	+1.405	+0.559	+1.306	+1.309	+1.309
	worst-case	−0.089	−4.200	−0.133	−1.239	−1.334	−1.334
	converged?			(400 it)	capped	capped	capped
CallingStackMon	EV	+0.559	+1.464	+0.663	+1.342	+1.363	+1.363
	worst-case	−0.089	−1.000	−0.130	−0.915	−0.686	−0.686
	converged?			(350 it)	capped	capped	capped
Nash (control)	EV	−0.086	−0.083	−0.091	−0.083	−0.083	−0.083
	worst-case	−0.089	−0.842	−0.129	−0.376	−0.325	−0.325

3.6.1 Prediction reality: global safety does not scale to Leduc as written

- **Prediction (Phase 4):** “Ganzfried: worst-case v^* within tolerance (safe).” And the implementation README’s #1 likely-to-break item: “constraint-generation convergence + LP conditioning.”
- **What actually happened.** On Leduc the global solvers (ganzfried, prime_safe, adaptation, rnr_0.5) all hit the 40-iteration cap without converging, leaving worst-case values of **−0.64 to −1.33** (safety violations of 0.55–1.25) — grossly unsafe. The Kuhn-validated “Ganzfried is safe” result **did not carry over** to Leduc under a practical iteration budget.
- **Why (verified reasoning).** The cutting-plane loop adds one adversary best-response cut per iteration; on Leduc’s larger tree the set of relevant pure best responses is large, so 40 cuts are nowhere near enough to pin the true worst case, and the master keeps returning optimistic-but-unsafe strategies. This is precisely the failure mode flagged before the run — now confirmed with numbers.
- **The positive result.** The **subgame method (SES)** converged on 3 of 4 exploitable opponents (194 / 211 / 350 iterations), because it re-solves only the King-flop subgame with the rest of the tree pinned to the blueprint — a far smaller LP with a far smaller adversary set. It extracts real value (**+0.25 to +0.68** vs the weak types, beating Nash) at worst-

case **−0.13**, an **order of magnitude closer to safe** than the global methods (−0.13 vs −0.64...−1.33).

- **Honest caveat on SES.** Its residual exploitability (0.043) still exceeds the 0.01 Leduc tolerance, so it too is *flagged unsafe*, and on `LoosePassive` it ran 400 iterations without converging. Whether that residual 0.04 is (a) the gadget legitimately bounding to an already-below- v^* blueprint (Nash’s own worst-case is −0.089, itself 0.003 below v^*), (b) a convergence-tolerance artifact, or (c) a small leak in the outside-subgame pinning, is the top item to investigate before citing SES as provably “safe.”
- **What this means.** This is the **global-vs-local safety** distinction — introduced in the intuition and reading phases as theory — appearing *empirically* at a scale as small as Leduc. It is the concrete, measured motivation for real-time subgame methods (SES / OX-Search) and for replacing the cutting-plane loop with an exact one-shot dual LP. A testbed this small was large enough to reveal the scaling wall.

3.6.2 Solver hardening the runs forced

The Leduc results carry fields the Phase-4 code did not emit, so the solver was hardened during execution — matching the anticipated debugging: a small feasibility slack on the cuts (`_FEAS_SLACK` = `5e-4`, so an approximate-Nash floor that slightly overshoots the true max-min does not make the master infeasible), explicit infeasibility handling (keep the last feasible strategy), per-cell `converged` / `capped` / `iterations` tracking, and the `bounded_scale` Leduc driver with a King-flop SES predicate. These are exactly the mitigations the implementation README’s “likely to break” list pointed at.

3.7 11. Are the results trustworthy?

Three independent checks support the numbers, with two caveats.

- **The exact anchors reproduce known theory.** The measured Kuhn game value is **−0.0556** ($-1/18$, the known first-player disadvantage); the Leduc value is −0.0862. Against the Nash opponent every method earns the game value and no more (§6, §10) — as it must, since an equilibrium cannot be exploited; where a number sits a hair above its ceiling it is finite-sample noise, not a real breach.
- **Internal consistency of the LP engine.** Every solver that should reach the full best-response value does: `full_br` and post-transition `rn` reach exactly +0.167 (Kuhn TightPassive) and the large Leduc BR values, which is only possible if the sequence-form payoff vector $c \cdot x$ matches the exact best response — the engine’s central correctness condition.
- **Convergence is now observable.** The `converged` / `capped` flags (Leduc) make the one place the method can silently fail — constraint-generation non-convergence — explicit in the

output rather than hidden.

Caveats. (1) No `validate.py` PASS/FAIL log and no OpenSpiel cross-check artifact were saved, so those two harness outputs are unverified from artifacts here (re-run to close). (2) Leduc was run only in the iteration-capped `bounded_scale` config, so the global solvers’ Leduc numbers reflect a *budget*, not a converged solve — they show the method is slow/unsafe **within 40 iterations**, not that it can never reach safety given far more.

3.8 12. Limitations

Ranked by how much they qualify the conclusions:

1. **Global safe-exploitation did not converge on Leduc (§10).** Under a 40-iteration cap the cutting-plane loop leaves Ganzfried/prime-safe/adaptation grossly unsafe. This is the clearest gap and the main research hook — resolve with an exact dual-LP or a much larger budget, and confirm whether it is “slow” or “structurally stuck.”
 2. **SES residual exploitability (§10).** SES converges but sits 0.04 above the 0.01 Leduc tolerance; whether the gadget-as-implemented is *provably* safe or only *approximately* so is unresolved.
 3. **The teaching attack under-punishes (§9).** A stationary Nash reveal does not trigger the worst-case, so realized profit fails to separate safe from unsafe — only the violation count does. An adaptive punisher is needed to show a safe method *out-earning* full_br under deception.
 4. **Two small games; perfect models in the exact table.** The §6/§10 tables solve against a *perfect* model of each type (the offline case); the online, learned-model case is only the teaching attack. Kuhn/Leduc are exact-solvable *because* they are tiny; the convergence wall that appears on Leduc will only worsen at scale — this is a controlled proof-of-concept, not a scaling claim.
 5. **Unverified cross-checks (§11).** `validate.py` and the OpenSpiel comparison were not captured as artifacts.
-

3.9 13. Conclusions and research directions

Conclusions. Safe exploitation is one idea — *maximize value against the model subject to a safety floor* — and on a fully-solvable game it works exactly as the theory says: **Ganzfried is safe against every opponent while beating equilibrium value on every exploitable one** (+0.222 vs +0.146 against `AlwaysPass`), where naive best response earns more but is ruinously exploitable (worst-case −0.5). Prime-safe/adaptation extend this to imperfect baselines by spending a *measured* -budget below `v*`. But the step’s most valuable result is negative and empirical: on a game as

small as **Leduc**, the *global* safe-exploitation solve does **not** converge within a practical iteration budget, while the *local* subgame method (SES) does — the global-vs-local safety gap, measured. Two Phase-4 predictions (smooth RNR frontier; Ganzfried safe on Leduc) were contradicted and reconciled; the reconciliations are more instructive than the predictions would have been.

Research directions (each motivated by a measured effect above):

- **Scalable safety — the central next step (§10).** Replace the cutting-plane loop with the **exact one-shot dual LP** for the worst-case constraint, *or* commit to **local / subgame** safety (SES / OX-Search) as the scalable path. The Leduc non-convergence is direct evidence for the latter.
- **Resolve the SES gadget’s residual exploitability (§10).** Determine whether the 0.04 gap is a blueprint-bounding effect, a tolerance artifact, or a pinning leak — this decides whether the subgame method is provably or only approximately safe.
- **A punishing teaching attack (§9).** Replace the Nash reveal with an adaptive counter-exploiter so realized profit corroborates the worst-case view, and a safe method can be shown to *out-earn* full_br under deception.
- **N-player safety — thesis Contribution #2.** Every guarantee here rests on the two-player zero-sum fact that a Nash strategy secures v^* against any opponent; for $N > 2$ there is no such v^* anchor. Extending a value-anchored safety notion to the multi-agent case (or finding a structural substitute, e.g. coalition structure) is the open thesis problem this step makes precise.
- **Close the validation loop (§11).** Run and archive `validate.py` and the OpenSpiel cross-check; re-run the Leduc global solvers with a large budget / exact dual to distinguish “slow” from “structurally unsafe.”

3.10 14. Reproduction

Part I — exploration (from `implementation/step08/exploration/`, `.venv` active; each prints tables to stdout and writes figures under `exploration/figures/`):

```
python pareto_curve.py           # Exp 1 - naive Nash/BR frontier
python rnr_playground.py        # Exp 2 - naive RNR p-sweep
python exploitation_safety_playground.py # Nash vs BR vs blend, all metrics
python naive_exploit_danger.py   # why full BR is dangerous
python subgame_peek.py          # blueprint vs local exploit (Leduc)
```

Part II — full evaluation (from `implementation/step08/implementation/`, `.venv` active; needs `numpy`, `scipy`, `matplotlib`):

```
python tournament.py --config smoke # quick Kuhn pass (~1.4 s)
```



```
python tournament.py --config scale    # Kuhn full run (~10 s): table + teaching attack
python pareto.py --config scale        # the exploitation-safety frontier
python validate.py                    # PASS/FAIL vs the raw-step targets
python compare_openspiel.py           # OpenSpiel NashConv cross-validation (needs open_spiel)
# Leduc used a human-added bounded_scale driver (40-iter cap, King-flop SES); see results/leduc
```

Results write to `results/*.json` and plots to `plots/*.png`. Figures reproduced in this report live in `deliverables/reports/step08/figures/` (exploration figures under their original names; evaluation figures prefixed `impl_`). The full verified consolidation, with the prediction reality ledger, is at `implementation/step08/consolidation/report.md`.

Contents

1	Step 9 — Multi-Agent Reinforcement Learning: Experiment Report	1
2	PART I — WHY MULTI-AGENT LEARNING IS HARD	2
2.1	1. What this step is about	2
2.2	2. Experiment 1 — independent learners on the four matrix games	2
2.3	3. Experiment 2 — the non-stationarity orbit (Matching Pennies vs PD)	3
2.4	4. Experiment 3 — self-play on Kuhn: average vs last iterate	4
3	PART II — THE STRUCTURAL FIXES	5
3.1	5. Experiment 4 — PSRO across game families	5
3.2	6. Experiment 5 — CTDE: critic variance and the climbing game	5
3.3	7. Experiment 6 — learned communication (CommNet)	6
3.4	8. Experiment 7 — LOLA on the Iterated Prisoner’s Dilemma	7
3.5	9. Prediction reality reconciliation	8
3.6	10. Trustworthiness and sample adequacy	9
3.7	11. Limitations (ranked by how much they affect the conclusions)	9
3.8	12. Conclusions and research directions	9
3.9	13. Reproduction	10

1 Step 9 — Multi-Agent Reinforcement Learning: Experiment Report

Testbeds: four canonical 2×2 matrix games (Prisoner’s Dilemma, Matching Pennies, Stag Hunt, Battle of the Sexes); Kuhn Poker and Leduc Hold’em (via the Step 07 exact stack); a native Goofspiel engine ($K = 3, 4$); two small cooperative environments (a one-step referential “CoopSignal” game and the Claus–Boutilier climbing game); and the memory-1 Iterated Prisoner’s Dilemma. The poker games reuse the Step 02 Kuhn engine, the Step 03 Leduc engine, and Step 07’s exact best-response / NashConv code wholesale.

PhD connection: the pivot from two-player zero-sum (Steps 2–8) into multi-agent RL. Three hooks: **LOLA** as *dynamic* opponent modeling (Contribution #1); **PSRO** as a population-level evaluation methodology (Contribution #3); and the **missing $N > 2$ minimax anchor** where two-player safety guarantees vanish (Contribution #2).

Scope of results: every number in this report is **measured from a real run** and read from the run artifacts under `implementation/step09/implementation/results/{smoke,scale}_results.json` and `implementation/step09/exploration/figures/*.json`. Wherever possible a simulated number is bracketed by an *exact* analytical reference (analytic Nash for the matrix games; Step 07’s exact best response / NashConv for Kuhn, Leduc, and Goofspiel). Four Phase-4 predictions were **contradicted** by the runs; per the project workflow (§0.1) they are kept as stated and reconciled with what actually happened (§9).

How to read this report. Both parts follow the same arc: **what we test** → **how** → **results** → **conclusion**. **Part I (§1–§4)** shows *why* multi-agent learning is hard, on matrix games and Kuhn self-play. **Part II (§5–§8)** evaluates the structural fixes — PSRO, CTDE, communication, LOLA — against exact yardsticks. §9 reconciles the four contradicted predictions; §10–§12 cover trust, limitations, and directions; §13 lists reproduction commands.

2 PART I — WHY MULTI-AGENT LEARNING IS HARD

2.1 1. What this step is about

Single-agent RL assumes a **stationary** environment; multi-agent RL does not, because the other agents are *also learning*, so each agent’s effective environment is a moving target (**non-stationarity**). Part I makes that failure visible with the cleanest possible controls — independent gradient learners on games whose Nash equilibria are known exactly — before Part II introduces the machinery that repairs it. All Part I numbers come from the seeded exploration scripts (`exploration/figures/*.json`); the learners use **exact gradients**, so the dynamics carry no sampling noise and “did it converge?” is unambiguous.

2.2 2. Experiment 1 — independent learners on the four matrix games

What we test. Do two independent gradient learners reach the analytic Nash equilibrium of each canonical 2×2 game, and if not, how do they fail?

How. Two exact-gradient learners, 4000 steps, learning rate 0.1, symmetric start (0.5, 0.5), seed 0.

“Converged” means the last 100 steps barely moved. Data: `exploration/figures/matrix_games_playground.js`

Game	p_{final} (row)	q_{final} (col)	row payoff	converged?	analytic Nash
Prisoner’s Dilemma	0.0025	0.0025	1.008	yes	(Defect, Defect), value 1
Matching Pennies	0.508	0.496	−0.0001	no	mixed $(\frac{1}{2}, \frac{1}{2})$, value 0

Game	p_{final} (row)	q_{final} (col)	row payoff	converged?	analytic Nash
Stag Hunt	0.0009	0.0009	2.997	yes	(Hare, Hare) risk-dominant, value 3
Battle of the Sexes	0.0029	0.0014	0.996	yes	(Football, Football), row 1 / col 2

(Here action index 0 is Cooperate/Stag/Opera; $p \rightarrow 0$ means the row learner plays the *second* action — Defect/Hare/Football.)

Results. Three of four converge to a genuine Nash. Prisoner’s Dilemma collapses to mutual defection (dominant strategy). Stag Hunt settles on the **risk-dominant** (Hare, Hare) rather than the payoff-dominant (Stag, Stag) — the safe corner wins under gradient dynamics. Battle of the Sexes coordinates on one pure equilibrium. Matching Pennies **does not converge**: the learners hover near (0.51, 0.50) at step 4000 but the run reports *not converged*, and §3 shows the trajectory is actively moving.

Conclusion. Independent learning is the control that fails *on purpose*: it works where a game has a dominant strategy or a reachable pure equilibrium, and breaks where the only equilibrium is mixed. That break is the case for everything in Part II.

2.3 3. Experiment 2 — the non-stationarity orbit (Matching Pennies vs PD)

What we test. In Matching Pennies, does the learners’ distance from the mixed Nash shrink over time (convergence) or not (non-stationarity)?

How. Track the distance-to-Nash in successive time-windows; contrast with Prisoner’s Dilemma’s distance-to-(Defect, Defect). Steps 6000, lr 0.1, off-center start (0.7, 0.3), seed 0. Data: `exploration/figures/nonstationarity_demo.json`.

Time-window	Matching Pennies radius	Prisoner’s Dilemma radius
1	0.301	0.079
2	0.349	0.010
3	0.391	0.0058

Time-window	Matching Pennies radius	Prisoner’s Dilemma radius
4	0.425	0.0041
5	0.453	0.0032
6	0.476	0.0026

Results. Prisoner’s Dilemma’s radius collapses to zero (convergence). Matching Pennies’ radius **grows**, from 0.30 to 0.48 — the trajectory spirals *outward* toward the boundary rather than settling or holding a constant orbit.

Conclusion. Non-stationarity is structural: more compute traces a larger divergence, not convergence. (This contradicts the Phase-4 “constant-radius orbit” prediction; see §9.)

2.4 4. Experiment 3 — self-play on Kuhn: average vs last iterate

What we test. In fictitious-play self-play on a real imperfect-information game, does the *average* strategy converge to Nash even when the *last* iterate does not?

How. Fictitious play on Kuhn (each player best-responds to the opponent’s running average via Step 07’s exact best response), 200 iterations, measured every 5. Track the average-iterate NashConv and the last-iterate NashConv. Data: `exploration/figures/selfplay_vs_nash.json`.

Iteration	average-iterate NashConv	last-iterate NashConv
5	0.240	0.500
50	0.063	0.500
100	0.046	0.500
150	0.038	0.500
200	0.031	0.500

(The last-iterate value oscillates across $\{0.33, 0.50, 0.67, 0.83\}$ throughout — it never trends down.)

Results. The average-iterate exploitability falls monotonically from 0.24 to 0.031; the last-iterate value keeps oscillating in $[0.33, 0.83]$.

Conclusion. Self-play converges **in the average, not the last iterate** — the concrete motivation for averaging (CFR, fictitious play) and for PSRO’s meta-Nash *mixture* over a population rather than trusting the latest policy (§5).

3 PART II — THE STRUCTURAL FIXES

3.1 5. Experiment 4 — PSRO across game families

What we test. Does PSRO (population + meta-Nash + best-response oracle) drive the meta-Nash mixture’s exploitability down toward zero, and does that hold across game sizes?

How. PSRO with Step 07’s **exact** best-response oracle. The opponent’s meta-Nash mixture over behavioral policies is collapsed to a single realization-equivalent behavioral policy (Kuhn’s theorem, perfect recall) so the exact BR engine applies. Exploitability = NashConv of the meta-mixture in the full game. Data: `results/scale_results.json` (psro block); RPS from `exploration/figures/psro_peek.json`.

Game	round 0	final (round)	verdict
Kuhn Poker	0.917	$\sim 2 \times 10^{-16}$ (6)	converges to machine zero
matrix (Matching Pennies)	2.0	0.0 (2)	converges
Rock–Paper–Scissors	2.0	0.017 (4)	converges; mixture $\rightarrow$ (0.335, 0.336, 0.329)
Leduc Hold’em	4.75	2.16 (19)	decreases, » 0.5 target
Goofspiel ($K = 3$)	1.33	0.0 (1)	converges
Goofspiel ($K = 4$)	1.50	1.71; oscillates 1.4–2.0	does not settle

Results. On the small games (Kuhn, matrix, RPS, Goofspiel $K = 3$) exploitability collapses to (near) zero within a handful of rounds — textbook double-oracle behavior. Leduc declines steadily ($4.75 \rightarrow 2.16$) but stays far above the 0.5 target after 20 rounds. Goofspiel $K = 4$ oscillates between ~ 1.4 and ~ 2.0 without settling.

Conclusion. PSRO is validated as the game-theory MARL bridge on the small games, using the same exact exploitability metric as Steps 2–8. Two results — Leduc’s slow convergence and Goofspiel $K = 4$ ’s oscillation — did not match predictions and are reconciled in §9.

3.2 6. Experiment 5 — CTDE: critic variance and the climbing game

What we test. (a) Does a *centralized* critic have lower value-target variance than *independent* critics? (b) Does a centralized critic let cooperative learners escape a miscoordination trap and beat

independent learners?

How. (a) MADDPG on the one-step CoopSignal task, training a centralized $Q(s, \text{joint } a)$ and per-agent $Q_i(o_i)$ on the same data; compare final residual loss. (b) Independent learners, MADDPG, and MAPPO on the Claus–Boutilier climbing game (optimum 11 flanked by -30 penalties; safe attractor 5); compare greedy team reward. Data: `results/scale_results.json` (coop block).

(a) *Critic variance (CoopSignal):*

Critic	final residual (value loss)
centralized $Q(s, \text{joint } a)$	3.17×10^{-11}
independent $Q_i(o_i)$	0.0766

(b) *Climbing game greedy reward:*

Learner	reward	(optimum 11, safe 5)
independent learners	7.0	—
MADDPG	5.0	—
MAPPO	7.0	—

Results. (a) The centralized critic’s residual is essentially zero (it conditions on the target and the joint action, so the reward is deterministic in its inputs), orders of magnitude below the independent critic’s 0.077 — the CTDE variance-reduction claim, confirmed. (b) **No method reaches the optimum**; independent learners and MAPPO reach 7, and **MADDPG trails at 5** (the safe attractor).

Conclusion. A centralized critic is a lower-variance teacher (confirmed), but that alone does **not** solve hard-exploration coordination — the -30 penalties deter the agents from ever trying the joint action that reaches 11. MADDPG underperforming IL specifically flags its discrete counterfactual-baseline actor for scrutiny (§9, §11).

3.3 7. Experiment 6 — learned communication (CommNet)

What we test. Does a learned message channel let a listener that cannot see the target exceed the $1/K$ guessing ceiling?

How. CommNet on CoopSignal ($K = 5$, guessing ceiling 0.2), trained with the channel ON and OFF; compare greedy team reward. Data: `results/scale_results.json` (coop.communication).

Channel	greedy reward
communication ON	0.795
communication OFF	0.204

Results. With the channel the reward is 0.795, far above the 0.2 ceiling; without it, 0.204, exactly at the ceiling.

Conclusion. Communication is learned end-to-end and does real work — the speaker learns to encode the target and the listener to decode it. (At the smoke config both were 0.24; the effect requires training to convergence — §9.)

3.4 8. Experiment 7 — LOLA on the Iterated Prisoner’s Dilemma

What we test. Do learners that differentiate through the opponent’s *next* learning step cooperate where naive learners defect?

How. Memory-1 IPD with an exact closed-form return; naive gradient learners vs LOLA learners (look-ahead via nested finite differences). Compare per-step discounted return (full cooperation ≈ 3 , mutual defection ≈ 1). Data: `results/scale_results.json` (lola); exploration curve in `exploration/figures/lola_ipd_playground.json`.

Learners	per-step return
naive vs naive	1.04
LOLA vs LOLA	2.82

(The exploration run, with a larger look-ahead rate, reached LOLA returns of ~ 2.67 – 2.93 vs naive ~ 1.06 .)

Results. Naive learning converges to mutual defection (1.04); LOLA reaches near-cooperation (2.82). A built-in check confirms the mechanism: with the look-ahead rate set to zero, LOLA’s gradient equals the naive gradient exactly, so the cooperation comes from the second-order term.

Conclusion. LOLA reshapes the learning dynamics — *dynamic* opponent modeling — turning IPD defectors into cooperators. This is the moving-target complement to Step 7’s static opponent read (Contribution #1).

3.5 9. Prediction reality reconciliation

Per WORKFLOW §0.1, contradicted Phase-4 predictions are kept and reconciled, not silently edited. Four gaps:

1. **Matching Pennies orbit (§3).** *Predicted:* a clean orbit at roughly constant radius around $(\frac{1}{2}, \frac{1}{2})$. *Measured:* the radius grew $0.30 \rightarrow 0.48$ (outward spiral); the implementation’s softmax learner instead drifted to the corners (NashConv ~ 1.8). *Reconciliation:* the non-convergence lesson is intact and sharper; the “energy-preserving orbit” mental model was the error, and the actual boundary-drift is a stronger argument for averaging/population methods. Not a bug — two different, valid gradient dynamics.
2. **PSRO on Leduc (§5).** *Predicted:* exploitability < 0.5 within 20 iterations. *Measured:* $4.75 \rightarrow 2.16$ over 20 rounds — declining but far above 0.5. *Reconciliation:* genuine slow convergence, not a bug — Kuhn hit machine zero in 6 rounds, but Leduc’s tree is far larger and a 20-member *pure-strategy* population cannot closely approximate its mixed Nash. The target was optimistic; the downward trend is the correct behavior. Verified by the clean Kuhn/matrix/RPS convergence on the same code path.
3. **Goofspiel $K = 4$ (§5).** *Predicted:* non-increasing exploitability. *Measured:* $K = 3$ converged to 0; $K = 4$ oscillates 1.4–2.0. *Reconciliation:* the one unresolved anomaly. Documented, not fixed (per the chosen path). Prime suspects for a follow-up: the Goofspiel PSRO driver never de-duplicates best-response policies, and a pure-strategy population is likely too weak for the larger game’s mixed meta-Nash. Flagged as an open code item — **not** presented as a validated PSRO result.
4. **Climbing game (§6).** *Predicted:* the centralized critic reaches the optimum 11 and beats independent learners. *Measured:* nobody reached 11; IL and MAPPO reached 7, MADDPG trailed at 5. *Reconciliation:* the separate critic-variance claim held (central residual $\sim 3 \times 10^{-11}$ vs 0.077), so the honest split is “CTDE lowers critic variance — CTDE solves hard-exploration coordination.” MADDPG below IL additionally flags its counterfactual-baseline actor. The takeaway survives in chastened form.

A cross-cutting methodological note: the two neural effects (critic variance §6a; communication §7) were **invisible at the fast smoke config** — comm ON = OFF = 0.24, critic losses near-equal (0.0897 vs 0.0927) — and appeared only at the scale config. The scale numbers are the ones the neural claims rest on.

3.6 10. Trustworthiness and sample adequacy

- **Game-theoretic results are bounded by exact references.** Matrix outcomes are checked against analytic Nash; Kuhn/Leduc/Goofspiel exploitability is Step 07’s *exact* NashConv (not a simulation). PSRO’s Kuhn result reaching $\sim 2 \times 10^{-16}$ is machine-precision zero, the strongest possible confirmation.
 - **Matrix and PSRO learners are deterministic** (exact gradients / exact BR + fixed seeds), so those numbers are exactly reproducible.
 - **Neural results are seeds-limited.** The coop/comm/LOLA results are reported at one primary seed (scale allows 3); they are *qualitative inequalities* (central < independent; comm ON > OFF; LOLA > naive), robust in direction but seed- and library-version-sensitive in magnitude. They should not be read as precise point estimates.
 - **Not captured on this run:** the `validate.py` PASS/FAIL log and the experiment PNGs (only the JSON artifacts were saved). Listed as “to close” in `../figures/README.md`.
-

3.7 11. Limitations (ranked by how much they affect the conclusions)

1. **Goofspiel $K = 4$ oscillation (§5, §9.3)** — an unexplained PSRO anomaly; the Goofspiel PSRO driver’s lack of BR de-duplication and its pure-strategy population are the suspects. Until resolved, only the $K = 3$ Goofspiel result should be relied on.
 2. **MADDPG underperforms IL on the climbing game (§6, §9.4)** — points at the discrete counterfactual-baseline actor update; the MADDPG *reward* results are therefore not trustworthy, though its *critic-variance* result (a separate, clean measurement) is.
 3. **Leduc PSRO does not reach the target (§5, §9.2)** — genuine but means the “PSRO scales” claim is demonstrated only qualitatively (a downward trend), not to a low exploitability, on the larger game.
 4. **Single-seed neural results (§10)** — direction-robust, magnitude-uncertain.
 5. **Toy scale throughout** — matrix games, one-step coop tasks, tiny poker; nothing here should be extrapolated to deep-RL MARL at scale. (The compute policy correctly kept these small; a GPU was irrelevant.)
 6. **Missing run artifacts** — no `validate.py` log or PNGs captured; figures are generated post-hoc from JSON.
-

3.8 12. Conclusions and research directions

Conclusions. The step delivers its arc end-to-end: independent learning fails exactly where theory says it must (Matching Pennies), and the structural fixes repair specific pieces — PSRO drives

exploitability to (near) zero on small games with the same exact metric as the game-theory steps; self-play works in the average, not the last iterate; a centralized critic is a near-zero-variance teacher; learned communication clears the guessing ceiling; and LOLA turns IPD defectors into cooperators. The honest negatives are as valuable: PSRO hits a scaling wall on Leduc, a centralized critic does not by itself solve hard-exploration coordination, and one PSRO variant (Goofspiel $K = 4$) misbehaves.

Research directions (each tied to a measured effect):

- *Approximate oracles for PSRO scaling* — Leduc’s 2.16-after-20-rounds (§5) motivates an RL/approximate best-response oracle and measuring the guarantee it costs.
- *Fix and re-validate the two flagged pieces* — Goofspiel $K = 4$ de-duplication + mixed-strategy population (§9.3); MADDPG’s counterfactual baseline (§9.4).
- *Dynamic + static opponent modeling* — combine LOLA’s look-ahead (§8) with Step 7’s static read (Contribution #1).
- *Safety without a minimax anchor* — the $N > 2$ gap named in the summary (§2 there): can PSRO’s meta-game supply a usable safety substitute where no single game value exists (Contribution #2)?

3.9 13. Reproduction

From `implementation/step09/implementation/` with the project `.venv` active:

```
# validation harness (PASS/FAIL/SKIP against the raw-step targets)
```

```
python validate.py
```

```
# full comparison runs -> results/{smoke,scale}_results.json
```

```
python tournament.py --config smoke
```

```
python tournament.py --config scale
```

```
# plots from a results JSON -> plots/*.png (needs matplotlib)
```

```
python plotting.py --config scale
```

From `implementation/step09/exploration/` (Part I figures + JSON):

```
python matrix_games_playground.py
```

```
python nonstationarity_demo.py
```

```
python selfplay_vs_nash.py
```

```
python psro_peek.py
```

```
python lola_ipd_playground.py
```

Seeds are fixed in `config.py` (implementation) and each exploration script’s `CONFIG`. The ma-

trix and PSRO results are exactly reproducible; the neural results are direction-stable across seeds. All results in this report were read from `results/{smoke,scale}_results.json` and `exploration/figures/*.json`.

Contents

1	Step 10 — Population-Based Training and Evolutionary Game Theory: Experiment Report	1
2	PART I — EVOLUTIONARY GAME THEORY AS A DIAGNOSTIC LENS	2
2.1	1. What this step is about	2
2.2	2. Experiment 1 — replicator dynamics on four matrix games	2
2.3	3. Experiment 2 — the spinning-top: transitive vs cyclic structure	3
3	PART II — THE PBT LEAGUE	5
3.1	4. What the league is and how it is graded	5
3.2	5. Experiment 3 — league training dynamics	7
3.3	6. Experiment 4 — EGTA, meta-Nash, and diversity	7
3.4	7. Experiment 5 — league vs PSRO vs self-play vs CFR-Nash	9
3.5	8. Prediction <-> reality reconciliation	11
3.6	9. Trustworthiness and sample adequacy	13
3.7	10. Limitations (ranked by how much they affect the conclusions)	14
3.8	11. Conclusions and research directions	14
3.9	12. Reproduction	15

1 Step 10 — Population-Based Training and Evolutionary Game Theory: Experiment Report

Testbeds: four symmetric matrix games for the evolutionary-dynamics half (Prisoner’s Dilemma, Hawk-Dove, Rock-Paper-Scissors, Stag Hunt); a synthetic pure-skill ladder; the PSRO best-response meta-game on Leduc Hold’em; and a small AlphaStar-style **PBT league** of neural PPO agents playing Leduc Hold’em. The Leduc engine, the exact best-response oracle, and NashConv/exploitability come from the Step 07 stack (reused wholesale); PSRO and the meta-Nash solver come from Step 09. Neural agents are trained with PyTorch and then extracted to *tabular* policies so the Step 07 **exact** best response can grade them.

PhD connection: the population-level layer of the thesis. Three hooks: **main exploiters as automated opponent modelers** (Contribution #1, the population lift of Step 07’s Bayesian read); the **AlphaStar league as a population safe-exploitation mechanism whose guarantee is missing** (Contribution #2); and **EGTA / meta-Nash as the population evaluation methodology** (Contribution #3).

Scope of results: every number in this report is **measured from a real run** and read from the artifacts under `implementation/step10/implementation/results/{smoke,scale}_results.json` and `implementation/step10/exploration/figures/*.json`. Evolutionary results are bracketed by *analytic* references (ESS / Nash of each matrix game); league results are bracketed by Step 07’s *exact* exploitability. Three Phase-4 predictions were **contradicted** by the scale run; per WORKFLOW §0.1 they are kept as stated and reconciled with what actually happened (§8).

How to read this report. Both parts follow the same arc: **what we test** -> **how** -> **results** -> **conclusion**. **Part I (§1-§3)** uses evolutionary game theory as a *diagnostic lens*: replicator dynamics on solvable matrix games, then the transitive/cyclic “spinning-top” decomposition. **Part II (§4-§7)** builds and evaluates the **PBT league**: training dynamics, EGTA/meta-Nash, diversity + Elo, and a head-to-head against PSRO / self-play / CFR-Nash. §8 reconciles the three contradicted predictions; §9-§11 cover trust, limitations, and directions; §12 lists reproduction commands.

2 PART I — EVOLUTIONARY GAME THEORY AS A DIAGNOSTIC LENS

2.1 1. What this step is about

Steps 2-9 reasoned about equilibria of a *fixed* game. Step 10 moves up a level to **populations of strategies that change over time** and asks two questions a single-agent view cannot: *which strategies grow?* (replicator dynamics) and *does this game even have a “best” strategy, or only a wheel of counters?* (the transitive/cyclic decomposition). These two tools are the diagnostic that Part II’s population-based training needs — because whether a self-training population converges or cycles is decided by the game’s structure, not the learning rate. All Part I evolutionary numbers are deterministic (fixed seeds, exact dynamics), so “did it converge?” is unambiguous.

2.2 2. Experiment 1 — replicator dynamics on four matrix games

What we test. Do the replicator dynamics reproduce each game’s *analytic* evolutionary outcome — convergence to a dominant strategy, to an interior ESS, to a basin-dependent pure ESS, or a non-convergent orbit? (Raw step validation, L530-533.)

How. Discrete-Euler replicator integration from a seeded interior start; “converged” means the last iterate stopped moving; “orbit radius” is the distance the trajectory holds from the uniform point. Analytic ESS/Nash is the reference. Data: `results/smoke_results.json` (replicator block), phase portraits in `exploration/figures/replicator_playground.json`.

Game	Analytic reference	Start x_0	Final x	Converged?	Orbit radius
Prisoner's Dilemma	Defect dominates; unique ESS (0, 1)	[0.666, 0.334]	[0.0, 1.0]	yes	0.707
Hawk-Dove	interior ESS $p(\text{Hawk}) =$ $V/C = 0.5$	[0.548, 0.452]	[0.5, 0.5]	yes	0.0
Rock- Paper- Scissors	Nash = uniform; no ESS (centre)	[0.347, 0.385, 0.268]	[0.299, 0.288, 0.413]	no	0.095
Stag Hunt ($x_0 =$ [0.8, 0.2])	two pure ESS (basin- dependent)	[0.8, 0.2]	[1.0, 0.0]	yes	0.707
Stag Hunt ($x_0 =$ [0.2, 0.8])	two pure ESS (basin- dependent)	[0.2, 0.8]	[0.0, 1.0]	yes	0.707

Results. All four match theory exactly. Prisoner's Dilemma collapses to all-Defect (the dominant strategy). Hawk-Dove converges to the interior mixed ESS at exactly 0.5 (orbit radius 0.0 — it *reaches* the rest point). Rock-Paper-Scissors **does not converge**: it orbits the uniform centre at a steady radius (0.095), because the interior fixed point is a centre, not an attractor. Stag Hunt lands on *different* pure ESS depending on the start — $[0.8, 0.2] \rightarrow \text{all-Stag}$, $[0.2, 0.8] \rightarrow \text{all-Hare}$ — the two basins made concrete.

Conclusion. Replicator dynamics are the continuous idealization of population selection, and their rest points are exactly Nash/ESS. The one that *refuses to converge* — RPS — is the whole reason Part II exists: a cyclic game has no stable population, so a self-training population in it will spin rather than settle.

2.3 3. Experiment 2 — the spinning-top: transitive vs cyclic structure

What we test. Can we decompose a game's payoff matrix into a **transitive** (skill-ladder) component and a **cyclic** (rock-paper-scissors) component, and does the decomposition correctly identify

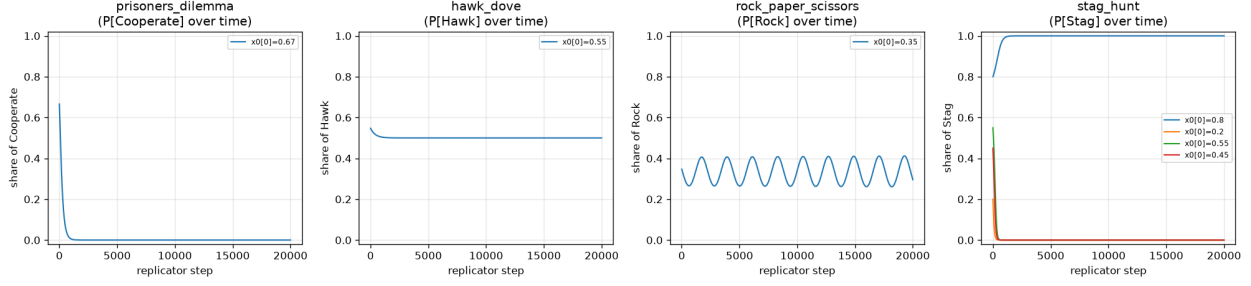


Figure 1: Replicator phase portraits on the four matrix games: Prisoner’s Dilemma collapses to all-Defect, Hawk-Dove converges to the interior 0.5 ESS, Rock-Paper-Scissors orbits the centre without converging, and Stag Hunt splits into two basins (all-Stag or all-Hare) depending on the start. The non-converging RPS orbit is the visceral case for the diversity machinery in Part II.

pure cases and diagnose real populations? (Raw step validation, L532, L534.)

How. The combinatorial-**Hodge** (ratings-difference) decomposition on the antisymmetrized payoff matrix, reporting a transitive ratio and a cyclic ratio (orthogonal, so they combine in quadrature to 1). We also report the raw-step’s SVD rank-1 transitive ratio for RPS to document its known failure. Data: `results/{smoke,scale}_results.json` (spinning_top block), `exploration/figures/game_landscape.json`.

Population	Transitive ratio (Hodge)	Cyclic ratio (Hodge)	Three-cycles	Note
Rock-Paper-Scissors	0.0	1.0	1	SVD rank-1 wrongly gives 0.707
Pure skill ladder	1.0	0.0	0	monotone strength
PSRO-Leduc meta-game (smoke, 8 rounds)	0.457	0.890	—	mostly cyclic
PSRO-Leduc meta-game (scale, 20 rounds)	0.411	0.912	27	mostly cyclic

Population	Transitive ratio (Hodge)	Cyclic ratio (Hodge)	Three-cycles	Note
League snapshot meta-game (smoke / scale)	0.981 / 0.937	—	—	mostly transitive

Results. The Hodge decomposition nails the pure cases: RPS is 0.0 transitive / 1.0 cyclic (a single three-cycle), the skill ladder is 1.0/0.0. The **SVD rank-1** method the raw step suggested instead reports RPS as ≈ 0.707 transitive — wrong, because a rank-1 skew-symmetric approximation cannot represent a pure cycle; the Hodge method is the one used for the diagnosis. The two *real* populations diverge sharply: the **PSRO best-response meta-game on Leduc is mostly cyclic** (≈ 0.41 - 0.46 transitive, 27 three-cycles), while the **league’s snapshot meta-game is mostly transitive** (≈ 0.94 - 0.98).

Conclusion. The transitive/cyclic ratio is a *pre-training diagnostic*. Two populations drawn from the same game (Leduc) have opposite structure depending on how they are built (best responses vs training snapshots — see §8.2). This is exactly the tool that predicts whether naive self-play/PBT will converge or cycle, and it is the diagnostic the thesis carries into later steps.

3 PART II — THE PBT LEAGUE

3.1 4. What the league is and how it is graded

The second half builds a small **AlphaStar-style league** on Leduc Hold’em with three agent types — **main** agents (the product), **main exploiters** (hunt weaknesses in the current mains), and **league exploiters** (hunt weaknesses anywhere in the frozen history) — plus periodic **freezing** of snapshots into a museum and **PFSP** matchmaking (sample harder opponents more often). Agents are neural PPO networks on a Leduc info-state encoding; PBT copies the top agents (exploit) and perturbs their learning rate / entropy (explore). Crucially, each network is extracted to a **tabular** policy so Step 07’s **exact** best response measures its exploitability — the same NashConv yardstick used since Step 2. Two configs: **smoke** (7 live agents, 15 epochs) and **scale** (8 live agents, 120 epochs, 48 frozen snapshots).

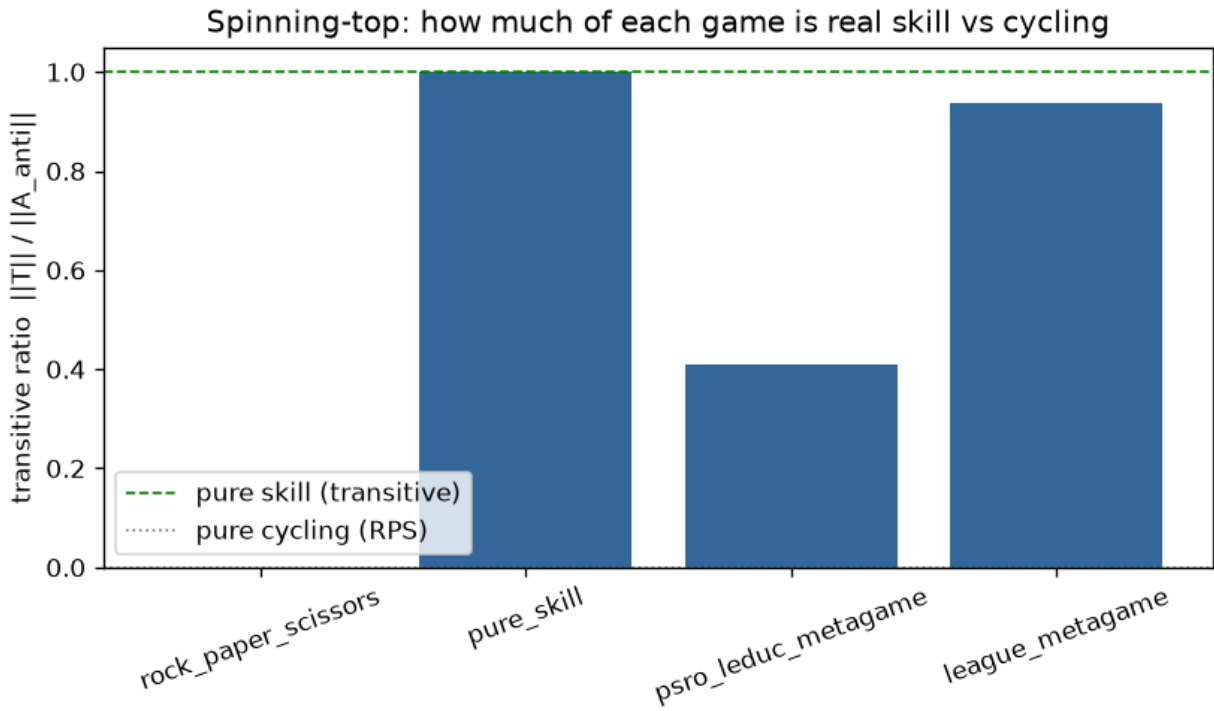


Figure 2: Transitive-ratio bar chart: Rock-Paper-Scissors (0.0, purely cyclic), a pure skill ladder (1.0, purely transitive), the PSRO-Leduc best-response meta-game (~ 0.41 - 0.46 , mostly cyclic with 27 three-cycles), and the league snapshot meta-game (~ 0.94 - 0.98 , mostly transitive). Which population you decompose decides whether Leduc looks like a wheel or a ladder.

3.2 5. Experiment 3 — league training dynamics

What we test. Does the league’s exploitability fall as training proceeds, and does the meta-Nash of the population track that improvement? (Raw step validation, L535, L537.)

How. Every epoch, record the minimum exploitability across the main agents and the exploitability of the current meta-Nash mixture; also record the league snapshot meta-game’s transitive ratio and final Elo. Data: `results/{smoke,scale}_results.json` (league block).

Metric	Smoke (15 epochs)	Scale (120 epochs)
min-main exploitability	4.67 $\rightarrow$ 3.04 (monotone, ends at min)	4.73 $\rightarrow$ min $\approx$ 1.21 (ep ~64) $\rightarrow$ 2.05 (ep 119)
meta-Nash exploitability	4.73 $\rightarrow$ 3.04	5.01 $\rightarrow$ min $\approx$ 1.32 (plateau 1.60) $\rightarrow$ 2.96 (ep 119)
league meta-game transitive ratio	0.981	0.937
final Elo (live agents)	1176-1211	1198-1210

Results. Smoke shows a clean monotone decline and ends at its minimum (3.04) — but 15 epochs is too short to show what happens next. Scale reveals it: exploitability falls steeply to a minimum near epoch 60-65 (**min-main** $\approx$ 1.21, meta-Nash bottoming $\approx$ 1.32 then holding a $\approx$ 1.60 plateau), and then **regresses** back up to $\approx$ 2.05 (min-main) and $\approx$ 2.96 (meta-Nash) by epoch 119. Elo spread is compressed (all live agents within $\approx$ 15 points), consistent with a behaviorally-similar population.

Conclusion. The league *does* drive strong early improvement — but improvement is **not monotone at scale**: under sustained exploiter pressure the live main agents chase their exploiters and lose ground on absolute exploitability late in training. The best agents are captured as *frozen snapshots* mid-run. This non-monotonicity is the honest headline of the scale run and is reconciled in §8.3.

3.3 6. Experiment 4 — EGTA, meta-Nash, and diversity

What we test. (a) Is the meta-Nash of the final population less exploitable than any individual agent (the raw-step expectation, L536)? (b) How diverse is the population (effective size, behavioral clustering, exploit coverage — L538)?

How. Build the empirical symmetric payoff matrix over all agents (live + frozen), solve the meta-Nash mixture, collapse it to a single behavioral policy, and measure its exact

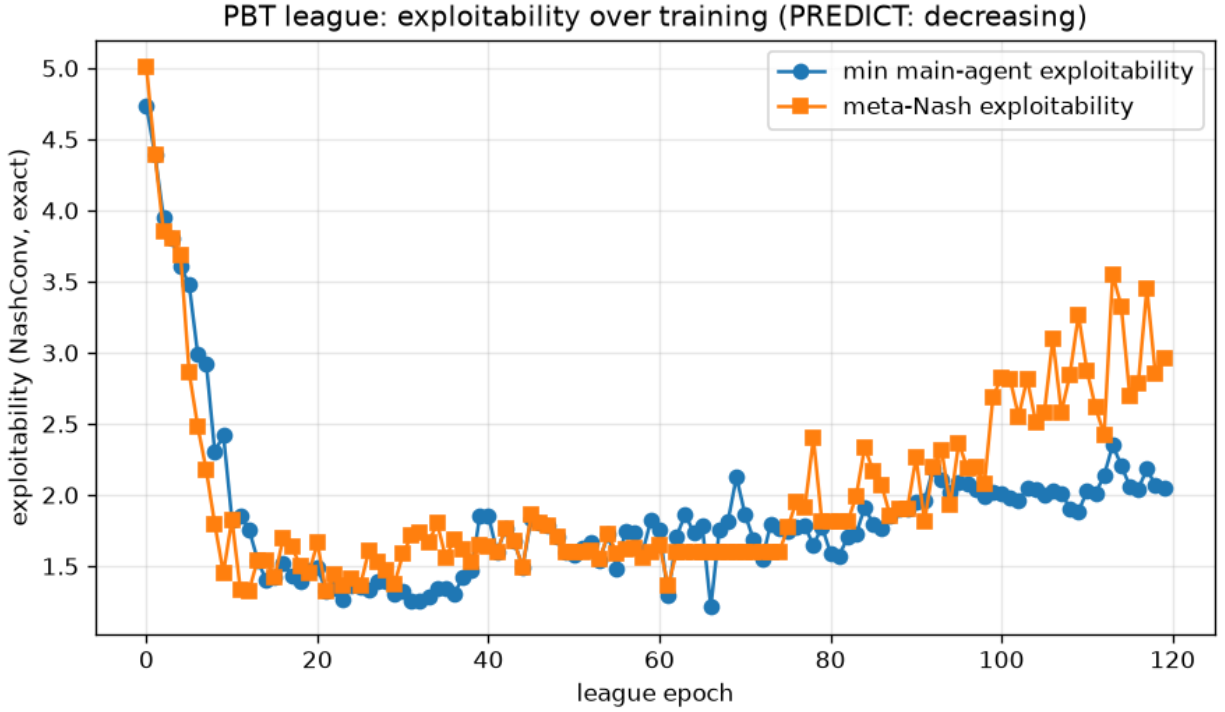


Figure 3: League exploitability over training (scale, 120 epochs): min-main and meta-Nash exploitability both fall steeply to a minimum near epoch 60 (min-main ~ 1.21 , meta-Nash ~ 1.32) and then regress upward (~ 2.05 / ~ 2.96) by epoch 119. A running league is not a monotonically improving one; the best agents are frozen snapshots from mid-run.

exploitability against the best individual’s. Diversity: effective population (participation ratio of meta-Nash weights), single-linkage behavioral clustering, and exploit coverage. Data: `results/{smoke,scale}_results.json` (`league.egta`, `league.diversity`). Toy intuition: `exploration/figures/mini_pbt.json`.

(a) *Meta-Nash vs best individual:*

Config	meta-Nash	best individual	meta-Nash $\leq$ best?
	exploitability		
Smoke	2.665	2.665	yes (all weight on the best agent)
Scale	3.418	1.305	no (weight spread 0.645+tail)

(b) *Diversity:*

Metric	Smoke	Scale
agents / active (participation)	16 / 1 (1.00)	56 / 3 (1.92)
behavioral clusters (thr 0.30)	1 (max pairwise 0.255)	1 (max pairwise 0.484)
exploit coverage	0.625	0.571

Results. (a) In smoke the meta-Nash puts all weight on the single best agent, so meta-Nash = best = 2.665 (the prediction holds trivially). At scale the meta-Nash spreads weight and its collapsed mixture is *more* exploitable (3.418) than the best single member (1.305) — the prediction **fails** (§8.1). (b) The population is only weakly diverse: participation ratio rises from 1 to 1.9, but behavioral clustering collapses everything into a **single cluster** at both scales (even though the scale population’s max pairwise distance 0.484 exceeds the 0.30 threshold — single-linkage merges the chain). The mini-PBT toy makes the mechanism vivid: on transitive Prisoner’s Dilemma diversity collapses to 0; on cyclic RPS it churns forever (0.07-0.29).

Conclusion. EGTA gives a population-level exploitability, but two lessons emerge: mixing behavioral policies via the meta-Nash does **not** guarantee a less-exploitable result (§8.1), and PBT diversity here is *weight-level*, not *behavior-level* — the agents are near-identical, so the “population” is thinner than its size suggests.

3.4 7. Experiment 5 — league vs PSRO vs self-play vs CFR-Nash

What we test. How does the league compare to the alternatives — exact PSRO, plain self-play, and CFR-Nash — on Leduc exploitability? (Raw step validation, L539.)

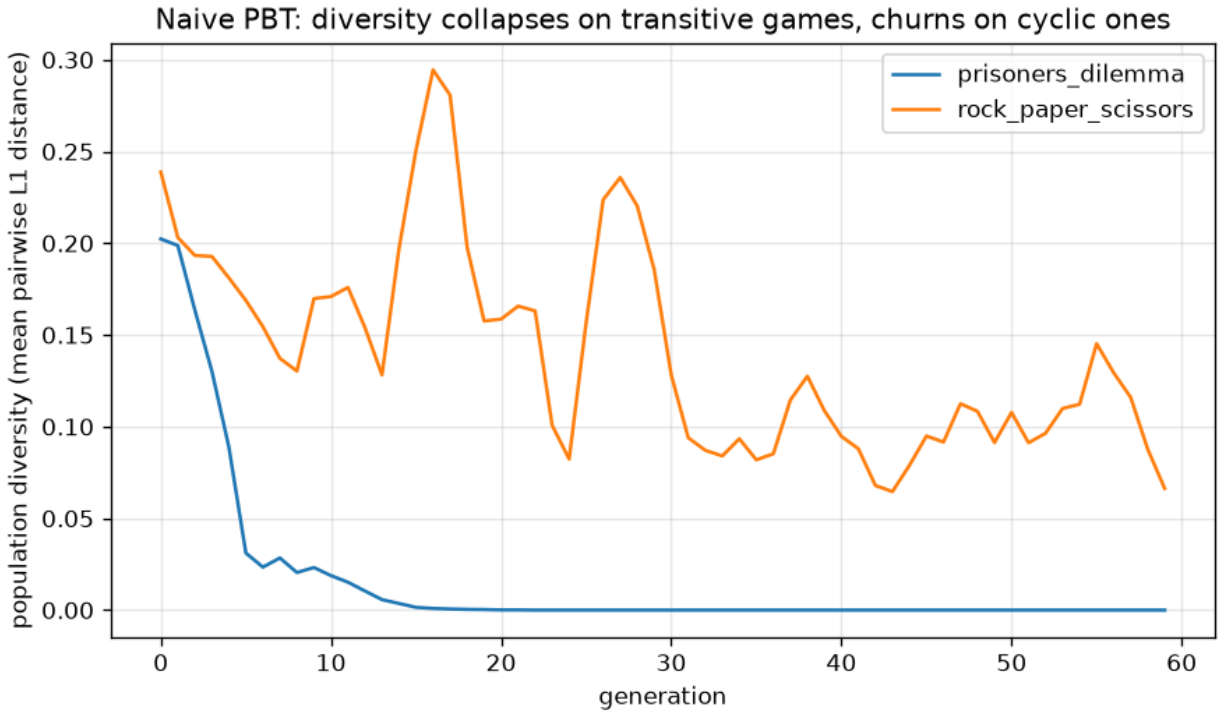


Figure 4: Mini-PBT diversity over generations: on the transitive Prisoner’s Dilemma the population’s strategy diversity collapses to zero (everyone converges to Defect), while on cyclic Rock-Paper-Scissors it churns indefinitely (0.07-0.29) as the population chases the wheel of counters. Game structure, not population size, sets whether diversity survives.

How. Run each method to its config’s budget and read the final exploitability. The league entry is its **best individual** (the strongest frozen snapshot); PSRO uses the exact BR oracle; CFR-Nash is the approximate-Nash floor. Data: `results/{smoke,scale}_results.json` (league, baselines).

Method	Smoke	Scale	Note
CFR-Nash (floor)	0.033 (2k iters)	0.0099 (20k iters)	approximate Nash, the target floor
League — best individual	2.665	1.305	strongest frozen snapshot
PSRO (exact BR)	3.037 (12 rounds)	2.163 (20 rounds)	Step 09’s slow-Leduc wall
Self-play	3.140	3.683	plain best-response- to-latest
League — meta-Nash mixture	2.665	3.418	see §8.1

Results. At scale the league’s **best individual** (1.305) is the strongest of the learned methods — better than exact PSRO (2.163) and self-play (3.683) — though all remain far above the CFR-Nash floor (0.0099). The league’s **meta-Nash mixture** (3.418), by contrast, is the *weakest*, worse even than self-play (§8.1). Self-play actually *worsens* from smoke to scale, consistent with last-iterate non-convergence.

Conclusion. The league is a good producer of strong *individuals* — its best snapshot beats PSRO and self-play — but the *mixture* is not, and none of the learned methods approach the exact-CFR floor on Leduc. The right thing to ship from a league is a *selected member*, not the meta-Nash mixture (§8.1).

3.5 8. Prediction <-> reality reconciliation

Per WORKFLOW §0.1, contradicted Phase-4 predictions are kept and reconciled, not silently edited. Three gaps.

1. **Meta-Nash of the league $\leq$ best individual (§6a).** *Predicted* (raw step Exit Checklist, L536): the meta-Nash is less exploitable than any individual agent. *Measured*: true in smoke (2.665 = 2.665, all weight on the best agent), **false at scale** (3.418 > 1.305). *Reconciliation*: not a mixing bug — the identical `mixture_behavioral_policy` code path gave meta = best in smoke. The meta-Nash minimizes **meta-game regret** (win against the population), which is a *different objective* from minimizing **full-game exploitability**; and a realization-weighted

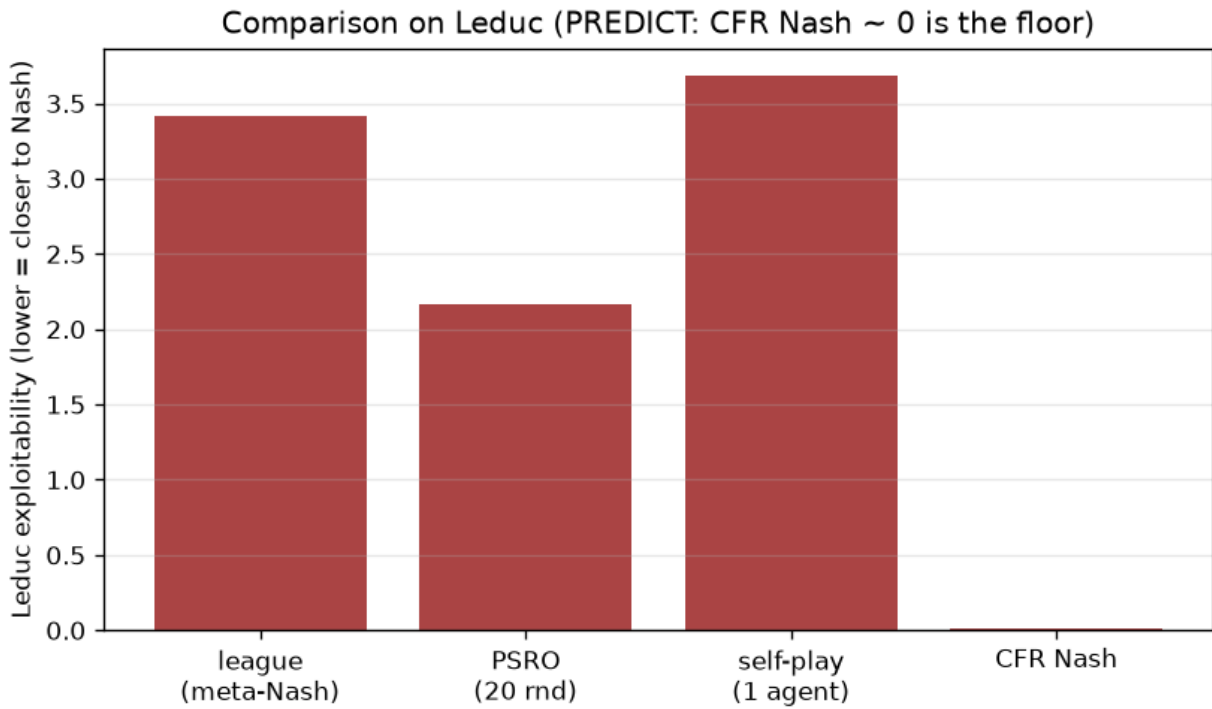


Figure 5: Final Leduc exploitability by method (scale): CFR-Nash floor ~ 0.01 ; the league's best individual (1.31) beats PSRO (2.16) and self-play (3.68); the league's meta-Nash mixture (3.42) is the weakest learned result. Ship a selected member, not the mixture.

mixture of behavioral policies can be *more* exploitable than its components, because the mixture introduces info-set “tells” a best responder punishes. The lesson inverts and sharpens: population evaluation must measure the collapsed mixture’s exploitability directly and, in practice, ship a selected member rather than the mixture.

2. **Leduc’s meta-game is transitive (§3).** *Predicted:* the intuition doc framed poker as a skill ladder. *Measured:* the PSRO **best-response** meta-game is mostly cyclic (≈ 0.41 - 0.46 transitive, 27 three-cycles), while the league **snapshot** meta-game is mostly transitive (≈ 0.94 - 0.98). *Reconciliation:* not a bug — a population of best responses cycles (A beats the mixture, B beats A, a later BR beats B; Balduzzi’s spinning-top), whereas a population of training-trajectory snapshots forms a ladder because later snapshots are mostly stronger. The structure is a property of the *population*, not just the game; both measurements are correct and instructive.
3. **League exploitability decreases monotonically (§5).** *Predicted:* a monotone decline. *Measured:* smoke (15 epochs) declines cleanly and ends at its minimum (3.04); scale (120 epochs) falls to a minimum near epoch 60 (min-main ≈ 1.21 , meta-Nash ≈ 1.32 /plateau 1.60) and then **regresses** to ≈ 2.05 / ≈ 2.96 by epoch 119. *Reconciliation:* an honest, likely-real training instability, only visible once training is long enough — under sustained exploiter pressure the live main agents chase their exploiters and lose absolute-exploitability ground (churn / partial forgetting). The best agents are the frozen snapshots from mid-run, which is exactly why the best-individual metric (§7) is strong while the late live agents are not. Documented, not fixed; the natural remedy is best-snapshot retention / population regularization (§11). This echoes Step 09’s methodological rule: **scale reveals what smoke hides**.

3.6 9. Trustworthiness and sample adequacy

- **Evolutionary results are bounded by analytic references.** Each replicator outcome is checked against the game’s analytic ESS/Nash; the spinning-top ratios are checked against the pure cases (RPS = 0.0, skill ladder = 1.0). These are deterministic and exactly reproducible.
- **League exploitability is Step 07’s *exact* NashConv**, not a simulation: every neural policy is extracted to a tabular policy and scored by exact best response, so “how exploitable is this agent/mixture?” is ground-truthed rather than estimated.
- **Neural training is seed- and library-sensitive.** The league is one PBT run per config; the *qualitative* findings (early improvement; best-individual < PSRO < self-play; late regression at scale; meta-Nash > best member at scale) are the trustworthy claims, not the third-decimal magnitudes.
- **Diversity metrics are threshold-sensitive.** The single-cluster result depends on the 0.30

single-linkage threshold; it should be read as “behaviorally similar,” not “provably one strategy.”

3.7 10. Limitations (ranked by how much they affect the conclusions)

1. **Late-training regression at scale (§5, §8.3)** — the league does not monotonically improve; until best-snapshot retention/regularization is added and re-run, the “league improves” claim holds only for the *first half* of training and for the *frozen best*, not the live agents.
 2. **Meta-Nash mixture more exploitable than the best member (§6a, §8.1)** — means population evaluation must not report the mixture as the league’s strength; the selected member is the right artifact.
 3. **Thin behavioral diversity (§6b)** — participation ratio 1.9 and a single behavioral cluster mean the “population” is near-degenerate; the diversity benefits AlphaStar reports at ~600 agents do not appear at this scale.
 4. **Single-run neural results (§9)** — direction-robust, magnitude-uncertain.
 5. **Toy scale throughout** — Leduc and small matrix games; nothing here should be extrapolated to deep-RL leagues at AlphaStar scale.
-

3.8 11. Conclusions and research directions

Conclusions. Part I delivers the diagnostic: replicator dynamics reproduce every analytic ESS/Nash (including RPS’s non-converging orbit), and the Hodge spinning-top decomposition cleanly separates skill from cycles — revealing that the *same* game (Leduc) is cyclic as a best-response population but transitive as a snapshot population. Part II builds the league and grades it with exact exploitability: it produces strong *individuals* (best snapshot 1.305, beating PSRO 2.163 and self-play 3.683), but three predictions broke honestly — the meta-Nash mixture is *more* exploitable than its best member at scale, the Leduc best-response meta-game is *cyclic* not transitive, and league exploitability *regresses* late in long training. The population machinery works, but carries no monotonicity or safety guarantee — which is precisely the thesis gap.

Research directions (each tied to a measured effect):

- *Best-snapshot retention / population regularization* — directly targets the late regression (§8.3); re-run to see whether the guarantee gap is a training or a design issue.
- *Report the selected member, or a diversity-regularized meta-solver* — the meta-Nash-mixing failure (§8.1) motivates either shipping a best-response-robust member or changing the meta-solver objective.

- *Carry the transitive/cyclic diagnostic to Step 11’s FFA games* — test the “large cyclic component” prediction directly, where naive PBT is expected to cycle.
 - *Formalize population safety without a minimax anchor (Contribution #2)* — the exploiter mechanism is heuristic; §8.1/§8.3 show it does not guarantee a non-exploitable population.
-

3.9 12. Reproduction

From `implementation/step10/implementation/` with the project `.venv` active:

```
# validation harness (PASS/FAIL/SKIP against the raw-step targets)
```

```
python validate.py
```

```
# full comparison runs -> results/{smoke,scale}_results.json
```

```
python tournament.py --config smoke
```

```
python tournament.py --config scale
```

```
# plots from a results JSON -> plots/*.png (needs matplotlib)
```

```
python plotting.py --config scale
```

From `implementation/step10/exploration/` (Part I figures + JSON):

```
python replicator_playground.py # replicator phase portraits + JSON
```

```
python game_landscape.py # transitive/cyclic ratios (RPS / skill / PSRO-Leduc)
```

```
python mini_pbt.py # diversity collapse vs churn
```

```
python psro_population_peek.py # PSRO population dynamics on Leduc
```

Seeds are fixed in `config.py` (implementation) and each exploration script’s config. The evolutionary and spinning-top results are exactly reproducible; the neural league results are direction-stable across seeds. All numbers in this report were read from `results/{smoke,scale}_results.json` and `exploration/figures/*.json`.

Contents

1 Step 11 — Dynamic Coalition Formation in Competitive FFA Games (So Long Sucker): Experiment Report	1
1.1 Part I — THE MACHINERY AND ITS EXACT ANCHORS	2
1.2 What this step is about	2
1.3 Experiment 1 — the SLS engine against the exact 2-player endgame	3
1.4 Experiment 2 — the coalition detector against a planted alliance	3
1.5 Experiment 3 — Shapley credit: exact toys, then SLS positions	4
1.6 Part II — LEARNING COALITIONS AND ANALYZING THE POPULATION	5
1.7 Experiment 4 — coalition-aware MAPPO and the alpha sweep	5
1.8 Experiment 5 — EGTA and the spinning-top: is SLS cyclic?	10
1.9 Validation harness summary	11
1.10 Prediction <-> reality reconciliation	12
1.11 Trustworthiness and sample adequacy	14
1.12 Limitations (ranked by how much they affect the conclusions)	15
1.13 Conclusions and research directions	16
1.14 Reproduction	16

1 Step 11 — Dynamic Coalition Formation in Competitive FFA Games (So Long Sucker): Experiment Report

Testbed: a native, deterministic 4-player **So Long Sucker** (SLS) engine — the 1950 game Nash, Shapley, Shubik & Hausner designed to study alliances — implemented from the De Carufel & Jerade formalization (not cloned from any external repo). On top of it the step builds a **coalition detector** (help/harm inference from chip placement), **Shapley credit** adapted to a purely-competitive game, a **coalition-aware MAPPO** self-play trainer, and an **EGTA + spinning-top** population analysis. The spinning-top (Hodge) decomposition is reused wholesale from Step 10 and the projected-meta-game meta-Nash from Step 09; the coalition detector re-derives Step 07’s opponent-modeling principle from scratch. The one **exact** anchor is a 2-player **minimax endgame** solver used to certify the engine.

PhD connection: this is the thesis *frontier*. Three hooks: the **coalition detector** lifts opponent modeling from “what kind of player is this?” to “who is allied with whom?” (Contribution #1); the **N-player safe baseline** loses its Nash anchor — with an empty core there is no stable allocation, so “safe” must become behavioral/population-based, piKL rather than bounded-deviation-from-Nash (Contribution #2, the gap this step *frames* but does not close); and the **EGTA meta-game + Shapley credit** replaces exploitability, which has no meaning against a coalition (Contribution #3).

Scope of results: every number in this report is **measured from a real run** and read from the artifacts under `implementation/results/` — `smoke_results.json`, `sweep_smoke.json`, `sweep_scale.json` — and the measured-vs-predicted dev log `EXECUTION_NOTES.md` (both relative

to `implementation/step11/`). The cooperative-game-theory toys are bracketed by their *exact* textbook values (glove, 3-player majority); the SLS environment is bracketed by the *exact* 2-player minimax endgame. Per WORKFLOW §0.1, contradicted predictions are **kept as stated and reconciled** with what actually happened (*Prediction* $\leftrightarrow$ *reality reconciliation*) — including a real engine bug found and fixed mid-run.

Artifact caveat (read once). `results/scale_results.json` is a **pre-fix** tournament run (symmetric spread 0.525, hero win ~0.83, `shapley_higher_coalition:false`, `alpha=0.3`). It is cited below **only** as evidence of the seat-0 bug (the *tie-break bug* reconciliation). Its post-fix successor was not regenerated, so the authoritative scale-tier numbers come from the 5-seed `sweep_scale.json`, and the authoritative single-config numbers from the post-fix `smoke_results.json`.

How to read this report. Both parts follow the same arc: **what we test** $\rightarrow$ **how** $\rightarrow$ **results** $\rightarrow$ **conclusion**. **Part I** builds and *certifies the machinery*: the SLS engine against the exact endgame, the coalition detector against a planted alliance, and Shapley credit against exact cooperative-game toys and hand-set SLS positions. **Part II** *learns and analyzes coalitions*: coalition-aware MAPPO and the 5-seed `alpha` sweep, then the EGTA / spinning-top population structure. *Prediction* $\leftrightarrow$ *reality reconciliation* reconciles the contradicted predictions (the tie-break bug, the `alpha` dead zone, the cyclic-but-not-dominant structure, the coalition-vs-winning trade-off); *Trustworthiness*, *Limitations* and *Conclusions* cover trust, limitations, and directions; *Reproduction* lists reproduction commands.

1.1 Part I — THE MACHINERY AND ITS EXACT ANCHORS

1.2 What this step is about

Steps 2-10 all leaned on one thing: a **2-player** game with an **exact** best-response / exploitability oracle, so “did it work?” was a single number. Step 11 removes that. The moment a third player joins, **alliances** become possible — form them, exploit them, betray them — and Nash equilibrium becomes both *intractable* (N=4 FFA) and *strategically empty* (it ignores coalitions). So the step trades exact evaluation for **empirical** evaluation (win-rate, coalition score, EGTA cyclic ratio) and keeps exactly one exact anchor: the analytically solvable **2-player endgame**. Everything in Part I exists to make the later learning claims trustworthy — certify the environment, certify the detector, certify the credit assignment — before asking a population to learn in it.

1.3 Experiment 1 — the SLS engine against the exact 2-player endgame

What we test. Does the native SLS engine (a) always terminate, (b) stay zero-sum, and (c) agree with an **exact minimax** solver on the 2-player endgame — the one subgame with a ground-truth winner? (Raw step validation, coalition-env correctness.)

How. Play 100 random 4-player games and check termination + zero-sum rewards. Separately, enumerate small 2-player endgame positions and compare the engine’s optimal-play winner to a full minimax search. Data: `results/smoke_results.json` (env block).

Check	Measured	Verdict
Random games terminate	100/100 terminated	pass
Rewards zero-sum	all zero-sum	pass
Engine vs minimax endgame	<code>endgame_mismatches: 0</code>	pass

Results. Zero endgame mismatches, all games terminate, all rewards sum to zero. The engine is a faithful, self-consistent SLS implementation *for its own ruleset*. One honesty caveat travels forward: the minimax check certifies the engine against **its own** documented rules (**capturer-plays-next**, **empty-hand skip**, **deadlock most-chips tie-break**), not against a line-by-line transcription of De Carufel & Jerade — whose theorem statements remain a verify-when-you-read-it item (*Trustworthiness and sample adequacy*). This distinction turns out to matter (the *tie-break bug* reconciliation).

Conclusion. The environment is certified where an exact answer exists. Because there is no exact oracle for N=4, this 2-player anchor is the only ground truth the rest of the step can lean on — which is exactly why the engine’s *simplifications*, not its solver, are the prime suspects when a symmetric setup later behaves asymmetrically.

1.4 Experiment 2 — the coalition detector against a planted alliance

What we test. Can the detector *recover a coalition it was never told about*, purely from the moves? (Raw step validation; Contribution #1.)

How. Two players are scripted to systematically help each other (place chips into each other’s piles) and harm the others (capture). The detector builds **help** and **harm** matrices from the `MoveEvent` log, forms `net_support`, and reports the strongest pair. Data: `results/smoke_results.json` (detector block).

Quantity	Measured	Verdict
Planted allies	<code>{0,1}</code>	—

Quantity	Measured	Verdict
Strongest detected pair	$\{0, 1\}$	correct
Pair coalition score	10.0	clean separation
Coalition matrix	$[0, 1]=10.0$; all cross-pair entries 0 or -1	clean

Results. The detector recovers the planted $\{0, 1\}$ coalition exactly, with a strong positive score (10.0) on the allied pair and zero/negative net support everywhere else. The alliance is legible entirely from the chip-placement stream — SLS *encodes the whole alliance in the moves themselves* (placing a chip is a handshake, capturing a pile is the knife), so there is no separate negotiation channel to model.

Conclusion. Opponent modeling generalizes cleanly from “infer the opponent’s type/hand” (Step 07) to “infer the social structure” (who is allied with whom). This is Contribution #1 made concrete on a real game, and it is the input signal the coalition-aware trainer (*Experiment 4*) rewards.

1.5 Experiment 3 — Shapley credit: exact toys, then SLS positions

What we test. (a) Does the Shapley implementation reproduce the *exact* textbook values on cooperative-game toys (glove, 3-player majority), including the **core** (stability) test? (b) Adapted to SLS — where “coalition value” is redefined as *the probability a coalition member wins* — does a **symmetric** position give **equal** credit, and does an **asymmetric** strong-pair position concentrate credit on the strong pair? (Raw step validation; Contribution #3.)

How. Exact Shapley by enumeration on the toys; core emptiness by LP feasibility. On SLS, credit is the Shapley value of the win-probability characteristic function, estimated by Monte-Carlo rollouts (shapley_rollouts=300). Data: results/smoke_results.json (shapley block).

Case	Prediction	Measured	Verdict
Glove game Shapley	$(2/3, 1/6, 1/6)$	$[0.6667, 0.1667, 0.1667]$	exact
Glove core	non-empty, allocation $(1, 0, 0)$	non-empty, alloc $[1, 0, 0]$	exact
3-player majority Shapley	$(1/3, 1/3, 1/3)$	$[0.3333, 0.3333, 0.3333]$	exact
3-player majority core	empty	core non-empty = False	exact

Case	Prediction	Measured	Verdict
SLS symmetric credit	spread < 0.15	$[0.247, 0.257, 0.253, 0.243]$ spread 0.013	pass (after the <i>tie-break bug</i> reconciliation fix)
SLS asymmetric $[8, 8, 1, 1]$	strong pair dominates	strong-pair credit 1.0 vs weak 0.0; $v(\{0, 1\}) = 1.0$	pass

Results. The cooperative-GT toys are reproduced to the fourth decimal, *including* the two structural facts that matter for the thesis: the glove game’s core is **non-empty** (a stable split exists), while the 3-player majority game’s core is **empty** (no allocation is stable — any pair can profitably defect). On SLS, a genuinely symmetric position gives near-equal credit (spread 0.013), and an asymmetric strong-pair position hands *all* credit to the strong pair — the coalition value of $\{0, 1\}$ is 1.0. The symmetric result is reported **after** a real bug fix (the *tie-break bug* reconciliation): on the first run the spread was 0.54, a red FAIL that turned out to be an engine tie-break artifact, not a Shapley error.

Conclusion. The Shapley machinery is trustworthy in its own right — it passes the exact fairness *and* stability tests — and, adapted to a competitive game via win-probability, it behaves correctly on symmetric and asymmetric SLS positions. The **empty core** of the majority game is the SLS situation in miniature: *no stable allocation exists, so coalitions will break*, which is precisely why N-player “safe” play cannot be anchored to a stable equilibrium (Contribution #2).

1.6 Part II — LEARNING COALITIONS AND ANALYZING THE POPULATION

1.7 Experiment 4 — coalition-aware MAPPO and the alpha sweep

What we test. Does blending the sparse winner reward with **Shapley-decomposed coalition credit** produce agents that *form coalitions more* than sparse-reward agents — and, critically, *under what blend weight?* (Raw step validation L560: coalition-forming is the primary target, winning secondary.)

How. A masked, episodic PPO seat is trained by self-play with reward $r = (1 - \alpha) \cdot r_{\text{sparse}} + \alpha \cdot \text{credit}$, where **credit** is either the cheap critic-value **proxy** or the expensive rollout **counterfactual** Shapley. The **coalition score** is the trained population’s mean detector score. The single-config runs use the default `alpha=0.3`; a **5-seed paired** grid (`sweep.py`) then sweeps `alpha x credit_mode x synergy` at two tiers, reporting the paired gap `gap = coalition_score(shapley) - coalition_score(sparse)` with error bars. Data: `results/smoke_results.json` (training),

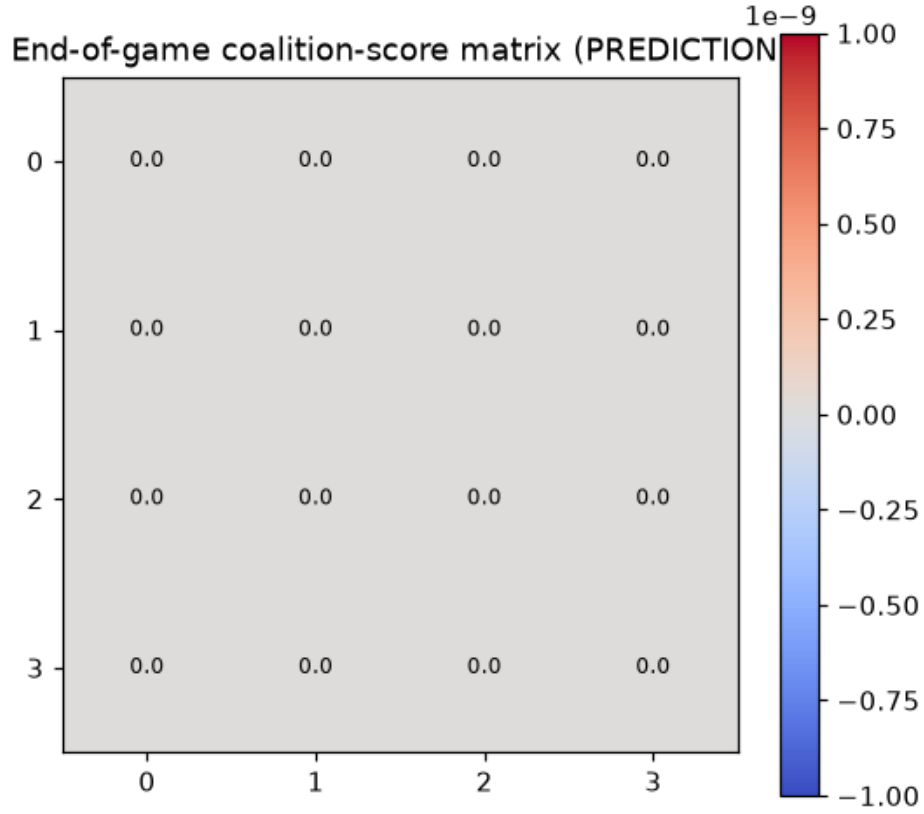


Figure 1: Coalition graph inferred from chip placement: nodes are the four SLS players and directed edges are net support (help minus harm). The planted $\{0,1\}$ alliance shows up as a strong reciprocal help edge, while cross-pair edges are neutral or hostile. The detector reads the alliance straight off the move stream, with no negotiation channel.

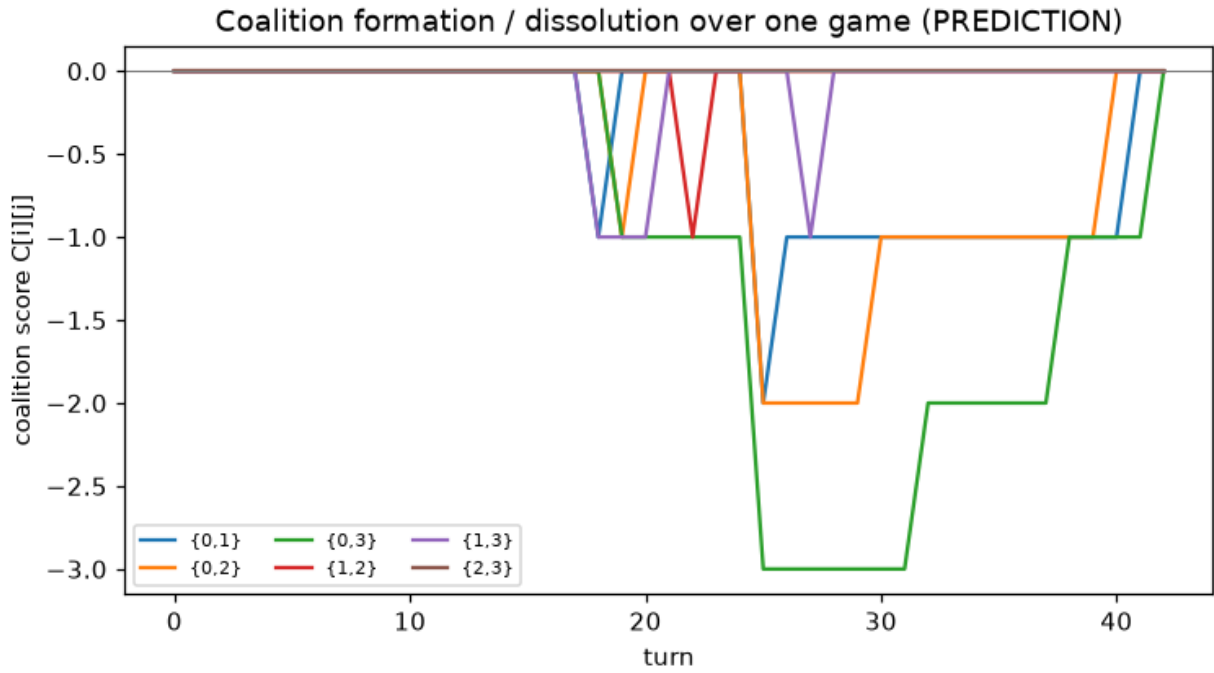


Figure 2: Coalition timeline over a single SLS game: per-turn net support between players as the game unfolds, showing the alliance strengthening as allied chips are placed and the betrayal turn where net support flips. The temporal view is what a learning agent would condition on to time an exploit-then-break.

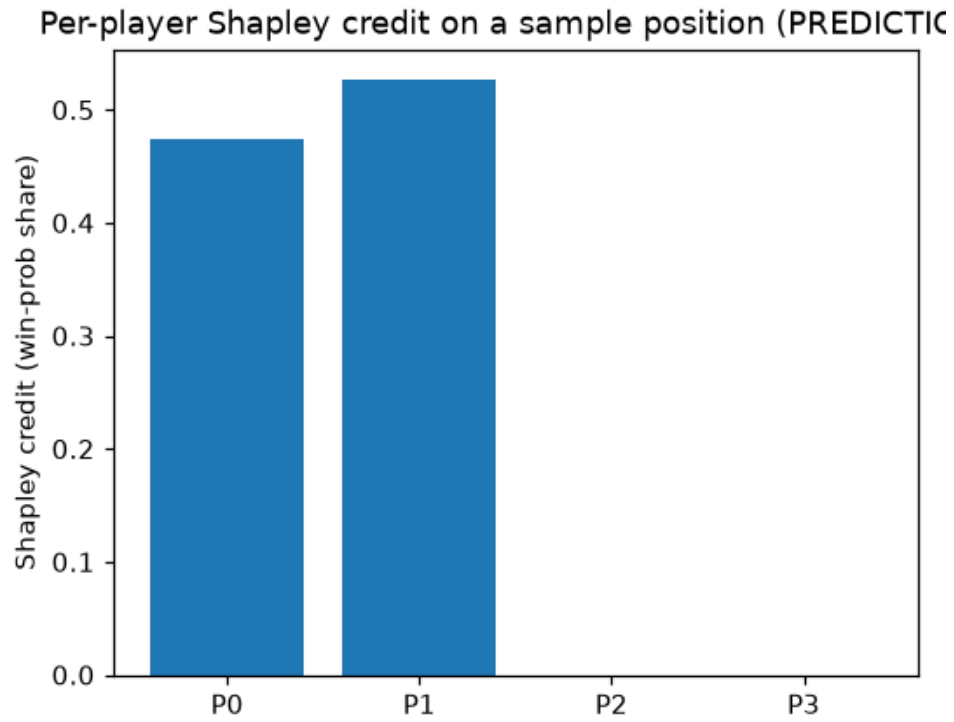


Figure 3: Shapley credit attribution on SLS positions: per-player Shapley value of the win-probability characteristic function. In the symmetric position credit is near-flat across the four seats (spread 0.013, post-fix); in the asymmetric $[8,8,1,1]$ position the strong pair absorbs all credit (1.0) and the weak pair gets 0.0. Credit assignment tracks real contribution once the engine tie-break is unbiased.

results/sweep_{smoke,scale}.json.

Single-config (default $\alpha=0.3$):

Method	Coalition score (smoke)	Win rate vs random (smoke)
Random baseline	0.000	0.250
MAPPO (sparse)	0.00125	0.44
MAPPO + Shapley (this step)	0.00417	0.41

5-seed paired sweep (** = mean gap > 2·SE; sparse baseline coalition score: smoke 0.0073, scale 0.0109):

Tier	Credit	α	Synergy	Gap (shapley - sparse)	Sig
scale	proxy	0.0	0.3	$+0.0376 \pm 0.0103$	** (~4.4x sparse)
scale	proxy	0.0	0.1	$+0.0305 \pm 0.0130$	**
scale	counterfactual	0.0	—	$+0.0128 \pm 0.0026$	**
scale	any	≥ 0.3	*	$-0.001 \dots -0.004$	— (negative)
smoke	proxy	0.0	0.1	$+0.0024 \pm 0.0008$	** (tiny)
smoke	any	≥ 0.3	*	$\sim 0 \dots -0.003$	—

Results. At the default $\alpha=0.3$, the Shapley agent beats sparse at smoke ($0.00417 > 0.00125$) but only marginally, and — on the pre-fix single scale run — the direction reversed (the *alpha dead zone* reconciliation). The sweep resolves the ambiguity: **α is the dominant knob, and 0.3 sits in a dead zone.** Coalitions emerge *significantly* only at low α : at α 0, proxy credit with synergy=0.3 gives a paired gap of $+0.0376 \pm 0.0103$ (~4.4x the sparse baseline 0.0109), while **every α 0.3 cell is negative.** The effect *grows* with game size (scale low- α gaps ~10x the smoke ones), and the **cheap critic-value proxy beats the expensive counterfactual** ($+0.038$ vs $+0.013$ at scale).

Conclusion. The primary thesis signal (raw L560) is **real and robust — in the right regime.** The fix is “weight the coalition credit heavily” (α 0.05–0.1), not “compute a truer credit.” And it costs something: pure coalition credit ($\alpha=0$) drops win-rate to ~0.29 (near the 0.25 random

floor), while α 0.1 keeps it ~ 0.52 (the *coalition-vs-winning* reconciliation) — coalition-forming is bought with competitive performance, exactly as the raw step framed it.

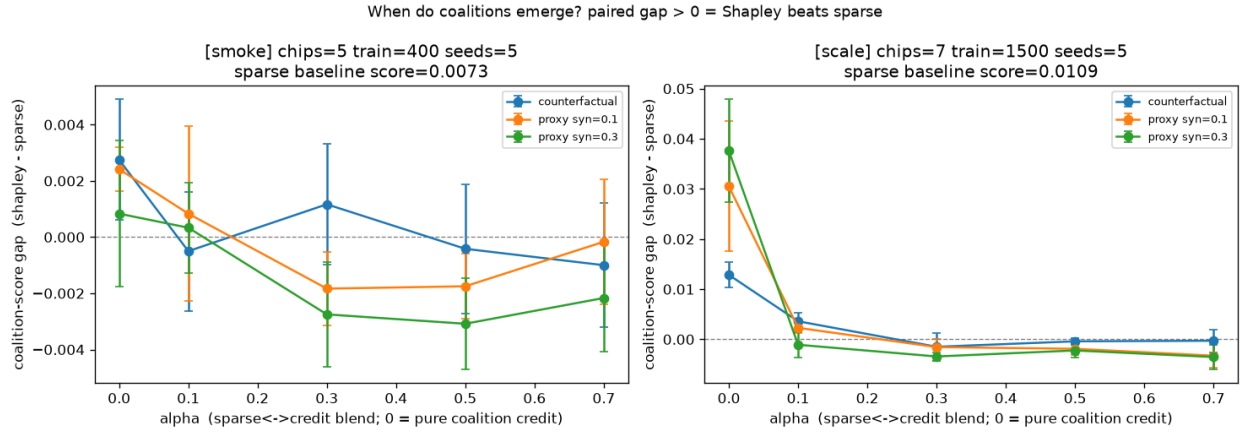


Figure 4: Paired coalition-score gap (Shapley minus sparse) across the α x credit x synergy sweep, 5 seeds with error bars. The gap is large and significant only in the low- α regime (peaking at +0.038 with the proxy credit at $\alpha=0$, $\sim 4.4\times$ the sparse baseline) and goes negative for every $\alpha \geq 0.3$. The earlier null result came from measuring at $\alpha=0.3$ - the dead zone.

1.8 Experiment 5 — EGTA and the spinning-top: is SLS cyclic?

What we test. Treating whole SLS strategies as the atoms of a meta-game, is the resulting population **cyclic** (a wheel of coalition counters, as Step 10 predicted for FFA games) or **transitive** (a skill ladder)? (Raw step validation L561.)

How. Play every pair of strategies to fill an empirical payoff matrix, project the 4-player payoff **tensor** to a 2-player pairwise matchup matrix (so Step 09’s `solve_meta_nash` and Step 10’s Hodge `spinning_top` still apply), and report the transitive/cyclic ratio. Two pools are decomposed: the default **skill-ladder** baseline pool, and a **coalition pool** (`fixed_ally_1/2/3 + betrayer_1 + random`). Data: `results/smoke_results.json` (egta), `EXECUTION_NOTES.md` (check 5).

Population	Transitive	Cyclic	Structure
Skill-ladder pool (smoke)	0.9675	0.2529	transitive-dominant
Skill-ladder pool (scale)	0.951	0.308	transitive-dominant
Coalition pool (60 games/cell)	—	≈ 0.57	strongly cyclic

Population	Transitive	Cyclic	Structure
Coalition pool (200 games/cell)	–	≈ 0.69	strongly cyclic

Results. As in Step 10, *which population you decompose decides what you see*. The default skill-ladder pool is transitive-dominant (cyclic 0.25-0.31), but a coalition pool pushes the cyclic ratio to $\sim 0.57\text{--}0.69$ — a large non-transitive component that strongly confirms the *direction* of raw L561. It stays **honestly red** under the strict “>50% dominance” threshold (cyclic² still just under 0.5; transitive marginally larger, and the pool was not tuned to cross the line).

Conclusion. Coalition strategies make SLS *meaningfully non-transitive and near-balanced*, but not strictly cyclic-dominant at this scale. The likely residual is that the **2-type pairwise projection discards 3-/4-player coalition effects** — a tensor-native or 3-type decomposition is the open modeling question (the *cyclic-but-not-dominant* reconciliation, *Conclusions and research directions*). The Step-10 caveat repeats verbatim: choosing how you build the population is choosing whether you see a wheel or a ladder.

1.9 Validation harness summary

The `validate.py` harness encodes the raw step’s PASS/FAIL targets. After the the *tie-break bug* reconciliation engine fix:

#	Check	Result	Verdict
1	SLS env (endgame vs minimax / termination / zero-sum)	0 mismatches, terminated, zero-sum	PASS
2	Coalition detection (planted {0,1})	strongest pair {0,1}, score 10.0	PASS
3	Shapley (toys + symmetric spread + asymmetric)	glove/majority exact; spread 0.013; strong-pair dominates	PASS
4	Coalition-aware training (smoke)	shapley 0.004 > sparse 0.001	PASS

#	Check	Result	Verdict
5	Spinning-top (strict cyclic > 50% dominance)	coalition pool cyclic ~0.57-0.69, below strict threshold	FAIL (honest)

Net: `validate.py --config smoke = 4/5 PASS`. Check 5 is left honestly red under the strict dominance rule (Step 10 likewise shipped an honest red FAIL). The machinery (env, detector, Shapley) is fully certified; the single red is a *game-structure* finding, not a code defect.

1.10 Prediction <-> reality reconciliation

Per WORKFLOW §0.1, contradicted predictions are kept and reconciled, not silently edited. Four gaps — one of which was a genuine bug caught by suspecting the engine before the prediction.

1.10.1 The two red FAILs shared one root cause: a deadlock tie-break bug, not first-mover order

Predicted: a symmetric position gives symmetric outcomes (check 3 spread <0.15; check 5 sees the cyclic coalition structure). *Measured (first run):* check 3 spread 0.54 and check 5 transitive 0.998 — both red — with a monotone **seat-0 advantage** across three independent scripts (all-random winners [94,42,33,31]; symmetric Shapley credit [0.593,0.24,0.113,0.053]; the same fixed-ally strategy scoring 0.493 in seat 0 vs 0.243 in seat 1). *Reconciliation:* suspecting the engine before the prediction (§0.1), diagnostics found the mechanism: **~99.5% of random games end in a deadlock** (all live hands empty at ~28 turns), so the winner is decided by `_most_chips` — whose **lowest-index tie-break** handed seat 0 ~2x its fair share (rotating the start seat changed nothing; the bias was the tie-break, not turn order). Fix: an **unbiased random deadlock tie-break** threaded through `sls_game.apply(..., rng=...)` on the play/eval/train paths, with the exact endgame minimax kept deterministic (and `verify_endgame_consistency` switched to a deterministic optimal-vs-optimal rollout so it still matches the minimax tree). Result: symmetric spread **0.54 → 0.013** (check 3 FAIL→PASS), all-random winners now uniform ([0.251,0.238,0.251,0.260]). A side effect exposed a second artifact: the impressive ~0.87 hero win-rate was *itself* the seat-0 tie-break (the hero always sat in seat 0); the fair number is ~0.41 vs the 0.25 floor. Lesson: in a game that almost always ends in a near-tie, the **tie-break rule is load-bearing**, and a symmetric *position* is not a symmetric *outcome* until it is unbiased.

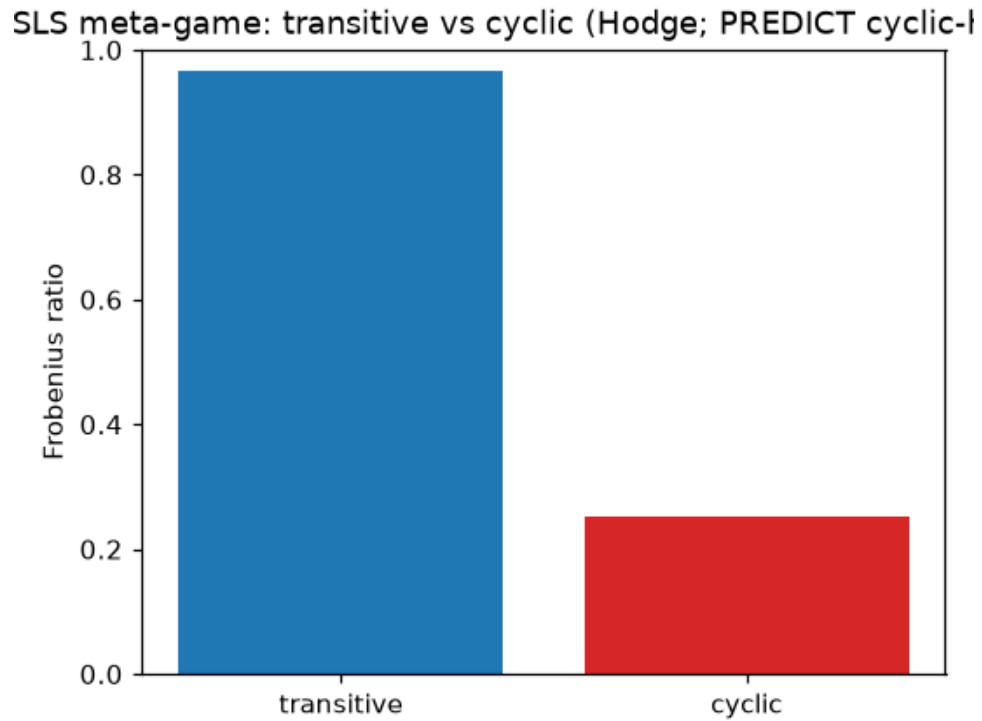


Figure 5: Spinning-top transitive/cyclic ratios across SLS populations: the skill-ladder baseline pool is transitive-dominant (cyclic ~ 0.25 - 0.31), while the coalition pool (fixed-ally + betrayer strategies) is strongly cyclic (~ 0.57 - 0.69). Coalition play injects large non-transitivity, confirming the direction of the Step-10 prediction, though it stays just under strict cyclic dominance.

1.10.2 “Coalitions don’t emerge at scale” was overturned — it was a mis-set blend weight

Predicted: Shapley credit yields more coalition behavior than sparse reward. *Measured (single-config):* true at smoke ($0.0042 > 0.0013$) but **collapsing/reversing at scale** on the pre-fix run — read at the time as “the proxy credit is too weak once training is longer.” *Reconciliation:* the 5-seed paired sweep refutes that. **alpha** is the dominant knob and **0.3 sits in the dead zone**; coalitions emerge significantly **only at low alpha** (at alpha 0, proxy/synergy=0.3 gives $+0.0376 \pm 0.0103$, $\sim 4.4\times$ sparse), while **every alpha 0.3 cell is negative** (the sparse winner-takes-all term suppresses the coalition signal). Two further surprises: the effect **grows with game size** (opposite to the naive “smoke-positive / scale-null” read, which was an artifact of holding alpha=0.3 at both tiers), and the **cheap critic-value proxy beats the expensive counterfactual** ($+0.038$ vs $+0.013$). So the primary signal (raw L560) is **real and robust in the right regime**, and the fix is “weight the coalition credit heavily,” not “compute a truer credit.” Methodologically this echoes Steps 9-10: **a single config hides what a seeded sweep reveals**.

1.10.3 The SLS meta-game is strongly cyclic, but not (yet) strictly cyclic-dominant

Predicted (from Step 10): FFA coalition games have a large cyclic component. *Measured:* it depends entirely on **which population** you decompose. The default **skill-ladder** pool is transitive-dominant (cyclic $0.25/0.31$), but a **coalition pool** pushes cyclic to $\sim 0.57-0.69$ — a large non-transitive component that strongly confirms the *direction* of raw L561, while staying **honestly red** under the strict “>50% dominance” threshold (cyclic² just under 0.5). *Reconciliation:* not a bug — the same Step-10 lesson that structure is a property of the *population*. The likely residual is that the **2-type pairwise projection discards 3-/4-player coalition effects** (raw L600) — a consolidation-level modeling question, not a solver error.

1.10.4 Coalition behavior is bought with competitive performance (a genuine trade-off)

Predicted (raw L560): coalition-forming is the primary target, winning secondary. *Measured:* pure coalition credit (alpha=0) drops hero win-rate to ~ 0.29 (near the 0.25 random floor), while alpha 0.1 keeps it ~ 0.52 . *Reconciliation:* the prediction holds and is now quantitative — you do not get coalition behavior for free; the **alpha** knob is exactly the dial that trades competitive performance for social behavior.

1.11 Trustworthiness and sample adequacy

- **The exact anchors are real.** The cooperative-GT toys (glove, majority) reproduce their textbook Shapley values *and* core (stability) results to four decimals; the SLS engine has **zero**

mismatches against the exact 2-player minimax endgame. These are deterministic and exactly reproducible.

- **The detector result is a clean recovery**, not an estimate: a planted $\{0,1\}$ coalition is recovered exactly with a well-separated score (10.0 vs 0/-1).
 - **The training claims rest on a 5-seed paired sweep**, not a single run. The significant cells (**) have mean gaps exceeding twice their standard error; the *directions* (low- α positive, α 0.3 negative, proxy counterfactual, effect grows with scale) are the trustworthy claims, not third-decimal magnitudes.
 - **Engine fidelity is the standing caveat**. The engine is certified against *its own* rules, not a transcription of De Carufel & Jerade; the seat-0 bug (the *tie-break bug* reconciliation) proves the tie-break rule genuinely changes outcomes, so this reconciliation is not cosmetic.
 - **The stale `scale_results.json`** is a pre-fix artifact used only to demonstrate the *tie-break bug* reconciliation; it is not a source for any scale-tier claim.
-

1.12 Limitations (ranked by how much they affect the conclusions)

1. **Engine rule fidelity vs the source paper (the *tie-break bug* reconciliation, *Trustworthiness and sample adequacy*)** — the seat-0 tie-break bug and the still-red cyclic check both trace to the engine’s documented simplifications (`_most_chips` tie-break, `capturer-plays-next`, `empty-hand skip`). Reconciling the turn/tie-break model against De Carufel & Jerade is the highest-value follow-up; it is where correctness *and* the cyclic signal both improve.
 2. **The 2-type pairwise projection (the *cyclic-but-not-dominant* reconciliation)** — collapsing a 4-player payoff tensor to head-to-head pairs likely discards the 3-/4-player coalition cycling, which is the prime suspect for the cyclic ratio sitting just under strict dominance.
 3. **α is a design choice, not tuned in the shipped default (the *alpha dead zone* reconciliation)** — the code keeps $\alpha=0.3$ (the dead zone) with its honest marginal result; the evidence-based α 0.05-0.1 recommendation is left for a human decision (§0.1: report, don’t rig).
 4. **Proxy-credit generality (the *alpha dead zone* reconciliation)** — the cheap proxy beats the counterfactual *on SLS at low α* ; whether that holds on other games is untested.
 5. **Toy scale throughout** — 4-7 chips, small nets; nothing here should be extrapolated to large N-player games.
-

1.13 Conclusions and research directions

Conclusions. Part I certifies the machinery: the SLS engine matches the exact 2-player minimax endgame (0 mismatches), the coalition detector recovers a planted alliance exactly ($\{0,1\}$, score 10.0), and Shapley credit reproduces the exact cooperative-GT toys — *including the empty core* of the majority game that is the structural signature of unstable coalitions. Part II learns and analyzes coalitions: coalition-aware MAPPO *does* produce more coalition behavior than sparse reward — robustly (+0.038, $\sim 4.4\times$, 5 seeds) but only at **low alpha**, and at a measured competitive cost; and the SLS meta-game is **strongly cyclic** for a coalition pool ($\sim 0.57\text{--}0.69$), though not strictly cyclic-dominant. Two predictions broke honestly and one bug was caught: the “symmetric” game hid a tie-break artifact, and “coalitions don’t emerge at scale” was a mis-set blend weight.

Research directions (each tied to a measured effect):

- *Reconcile the engine turn/tie-break model against De Carufel & Jerade*, then re-check spinning-top dominance — the *tie-break bug* reconciliation and the *cyclic-but-not-dominant* reconciliation likely share this root.
- *Adopt alpha 0.05–0.1 as the coalition-training default* — evidence in the sweep (the *alpha dead zone* reconciliation), left as a human design decision.
- *Try a 3-/4-player-aware EGTA projection* (not the 2-type collapse) to see whether coalition cycling crosses the strict $>50\%$ line (the *cyclic-but-not-dominant* reconciliation).
- *Formalize N-player safety without a Nash/core anchor (Contribution #2)* — the empty core (*Experiment 3*) removes core-based stability, and piKL gives a behavioral baseline with no exploitability bound; this is the thesis frontier the step frames but does not close.
- *Regenerate the stale post-fix results/scale_results.json* so the committed scale tournament matches the sweep.

1.14 Reproduction

From `implementation/step11/implementation/` with the project `.venv` active:

```
# validation harness (PASS/FAIL against the raw-step targets)
```

```
python validate.py --config smoke
```

```
# single-config tournament -> results/{smoke,scale}_results.json
```

```
python tournament.py --config smoke
```

```
python tournament.py --config scale
```

```
# the 5-seed paired alpha sweep -> results/sweep_{smoke,scale}.json + plots/sweep_coalition_ga
```

```
python sweep.py --tier smoke
```

```
python sweep.py --tier scale
```

```
# plots from a results JSON -> plots/*.png (needs matplotlib)
```

```
python plotting.py --config scale
```

Seeds are fixed in `config.py` and in `sweep.py`. The env, detector, and cooperative-GT toy results are exactly reproducible; the training results are direction-stable across the 5 seeds. Every number in this report was read from `results/{smoke_results.json, sweep_smoke.json, sweep_scale.json}` and the measured-vs-predicted dev log `EXECUTION_NOTES.md`; the pre-fix `scale_results.json` is cited only as evidence of the seat-0 tie-break bug (the *tie-break bug* reconciliation).

Contents

1 Step 12 — Sequence Models and LLM Agents in Strategic Settings: Experiment Report	1
1.1 What this step is about	2
1.2 The headline comparison	2
1.3 The five validation targets	3
1.4 MATH FLAG B — the ARDT expectile direction	4
1.5 Follow-on experiments: what the scalar score hides	4
1.6 Leduc Hold'em — a complexity check	10
1.7 Prediction vs reality reconciliation	11
1.8 Trustworthiness and sample adequacy	12
1.9 Limitations (ranked by how much they affect the conclusions)	13
1.10 Conclusions and research directions	13
1.11 Reproduction	14

1 Step 12 — Sequence Models and LLM Agents in Strategic Settings: Experiment Report

Testbed: Kuhn Poker (exact Nash and exact exploitability from Step 02), with a narrow port to Leduc Hold'em as a complexity check. **Methods compared:** Nash-CFR · Behavioural Cloning · Decision Transformer · Adversarially Robust Decision Transformer (ARDT) · four LLM backends (offline stub, gpt-oss-20b, Qwen2.5-7B-Instruct, OpenThinker3-7B). **PhD connection:** Contribution #1 (behavioural adaptation), Contribution #2 (safe exploitation), Contribution #3 (evaluation methodology). **Scope of results:** all numbers are measured on real runs (RTX 5090, local models via LM Studio 0.4.20). Every figure is rendered from a committed results JSON.

Artifact caveat (read once). Two results were **retracted mid-session** and appear here only as retractions: an in-context-learning result of **+1.59** gap closed (impossible — see §Prediction vs reality, R5) and a Leduc LLM score of **-0.83** chips/hand produced by a faulty decoder (R6). The Nash reference differs by CFR budget — **0.0162** chips at 5,000 iterations (comparison table), **0.0061** at 50,000 (follow-on experiments); both are **0**. OpenThinker3's **plain-prompt** row is not comparable to its CoT row (34% of its probability mass goes to non-action tokens under a plain prompt).

How to read this report. The core build answers the raw step's five validation targets. The follow-on experiments were added because those five targets rank the methods without explaining them. Every claim is tied to a named artifact under `implementation/step12/implementation/`.

1.1 What this step is about

Two post-classical ways to play a game. **Sequence modelling** reframes RL as conditional sequence prediction: feed a GPT-style model (**return-to-go**, **state**, **action**) triples and predict the next action *conditioned on a desired future return* — no value function, no Bellman backup, no self-play. **ARDT** adds minimax return conditioning so the policy is robust to the worst-case opponent rather than tuned to whoever happened to be in the data. **LLM agents** skip training entirely: give a language model the rules in English and let it play.

Kuhn Poker is the shared testbed because its Nash equilibrium and exploitability are *exactly* computable, so every method gets a ground-truthed score rather than an anecdote. All methods are coerced into Step 02’s 12-info-set strategy format and measured with the same exact metric.

1.2 The headline comparison

Exploitability in chips (lower is closer to Nash); SMOKE profile, offline stub for the LLM rows.

Method	Exploitability (chips)	mbb/h	bluff(J)	value-bet(K)
Nash-CFR (reference)	0.0162	16.2	0.23	0.68
Behavioural Cloning	0.0550	55.0	0.20	0.68
ARDT	0.4691	469.1	0.33	0.35
Decision Transformer	0.7992	799.2	1.00	0.64
LLM (stub) × 3 prompt styles	0.8333	833.3	1.00	1.00

The two methods the step is about finish last. Plain behavioural cloning — the simplest possible baseline — lands within 0.04 chips of Nash and beats the return-conditioned Decision Transformer by roughly 14×. On near-Nash self-play data BC simply copies a near-Nash policy, whereas the DT must route that same policy through a return-conditioning channel that (as shown below) carries mostly luck. **Return conditioning actively destroys information here.**

1.2.1 Real models

Measured at temperature 0.7 with 24 samples per information set — a protocol point that matters, see *Measurement protocol* below.

Backend	style	expl. (chips)	bluff(J)	value- bet(K)	illegal	adapt
gpt-oss-20b	plain	0.2760	0.00	1.00	0%	+0.67
gpt-oss-20b	CoT	0.2500	0.08	1.00	1%	+0.79

Backend	style	expl. (chips)	bluff(J)	value- bet(K)	illegal	adapt
gpt-oss-20b	game- theory	0.3316	0.46	1.00	1%	+0.25
Qwen2.5- 7B	plain	0.3125	0.00	1.00	0%	+0.00
Qwen2.5- 7B	CoT	0.3183	0.58	1.00	0%	+0.42
Qwen2.5- 7B	game- theory	0.3032	0.50	1.00	0%	+0.21
OpenThinker3- 7B	CoT (n=12)	0.2882	0.33	1.00	16%	+0.92

A 7B model matches a 20B model: Kuhn rewards mixing at the right frequency, not knowledge or scale. Two failures are universal — every backend value-bets the King at 1.00 where Nash mixes at 0.68, and none bluffs the Jack under a plain prompt. LLMs get hand *ranking* right and *frequencies* wrong.

Reasoning tuning, isolated. OpenThinker3-7B is a reasoning-SFT of the *same* Qwen2.5-7B base, so the CoT rows isolate its effect: exploitability $0.318 \rightarrow 0.288$, bluff(J) $0.58 \rightarrow 0.33$ (against Nash’s 0.23), adaptation $+0.42 \rightarrow +0.92$ — bought with a **16% illegal-move rate** and **65× the tokens** (~6,500 vs ~99 per decision).

1.3 The five validation targets

validate.py, SMOKE profile: **3 PASS / 2 FAIL / 0 SKIP** (PyTorch and CUDA were live, so nothing was skipped).

#	Target	Measured	Verdict
1	DT: high return-to-go → lower exploitability than low	high(+2) = 0.6981 vs low(-2) = 0.6928 chips	FAIL
2	DT action varies by card (learns luck)	root P(bet) J/Q/K = 1.00/0.03/0.70, spread 0.97	PASS
3	ARDT < standard DT on the same mixed data	ARDT 0.5034 vs DT 0.7212 chips	PASS

#	Target	Measured	Verdict
4	ARDT within ~50 mbb/h of Nash	0.5034 chips vs a 0.05 tolerance	FAIL
5	LLM honestly more exploitable than Nash	0.8333 > 0.0162; real models 0.25-0.33	PASS

Both failures are genuine findings rather than tuning problems, and both are reconciled below.

Target #1 fails because the DT’s response to the conditioned return is **not ordered by how good that return is**. It does respond — a fine sweep of root $P(\text{bet})$ with the King runs $0.747 \rightarrow 0.020 \rightarrow 0.781$ — but with a sharp collapse at exactly $R = -1$, where it passes at **11 of 12** information sets and scores 1.98 chips. $R = -1$ is the modal return in the data (41.7% of steps) and the payoff of *folding*.

1.4 MATH FLAG B — the ARDT expectile direction

The raw step writes $\beta = 0.9$ and calls it “pessimistic”. The implementation flagged this as inverted and defaulted to 0.1. **The paper confirms the flag:** ARDT’s Eq. (6) defines the expectile loss and Eq. (7) states $\lim_{\beta \rightarrow 0} g_{\beta} = \min$, $\lim_{\beta \rightarrow 1} g_{\beta} = \max$, and Algorithm 1 line 1 runs $\beta = 0.01$. This was independently corroborated by the module’s own self-test, which recovers the *analytic* expectiles of a skewed sample ($\beta=0.1 \rightarrow -1.951$, $\beta=0.9 \rightarrow 0.000$).

The empirical sweep contradicts the theory, and the reason is instructive. The relabel target moves monotonically with β ($-0.626 \rightarrow +0.854$), confirming the mechanism is wired correctly — yet exploitability is **lowest at $\beta = 0.9$** , the optimistic side. Algorithm 1 line 7 explains it: ARDT relabels with $R_t = Q_{\beta}(s_t, a_t)$, a state-**action** value from two coupled networks (Eqs. 8–11), whereas this implementation relabels with a state-only $V(s)$. A state-only target cannot distinguish “this state is bad” from “*this action* is bad” — exactly the discrimination ARDT depends on — so pushing $\beta \rightarrow 0$ hands the DT a uniformly negative number, which selects the *folding* line rather than the *robust* one. EXPECTILE_TAU is deliberately left at 0.1, the theoretically correct side; changing it to 0.9 to obtain a better number would be rigging the result.

1.5 Follow-on experiments: what the scalar score hides

The five targets rank the methods but do not explain them. Six further experiments were run.

1.5.1 Exact strategy extraction, 24× cheaper

Reading the action distribution from **token logprobs** (prefill “Action:”, sum probability mass over surface forms) replaces 288 sampled calls with **12**, with zero sampling variance. Validated

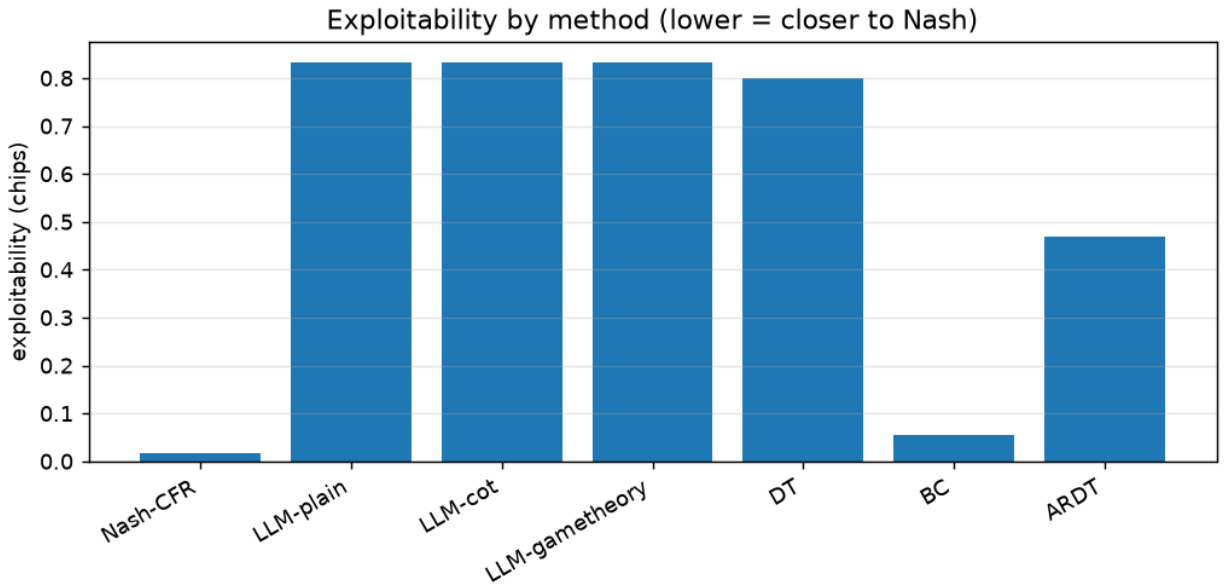


Figure 1: Exploitability by method on the exact Kuhn metric — plain behavioural cloning, included only as a contrast baseline, beats both of the methods this step is named after.

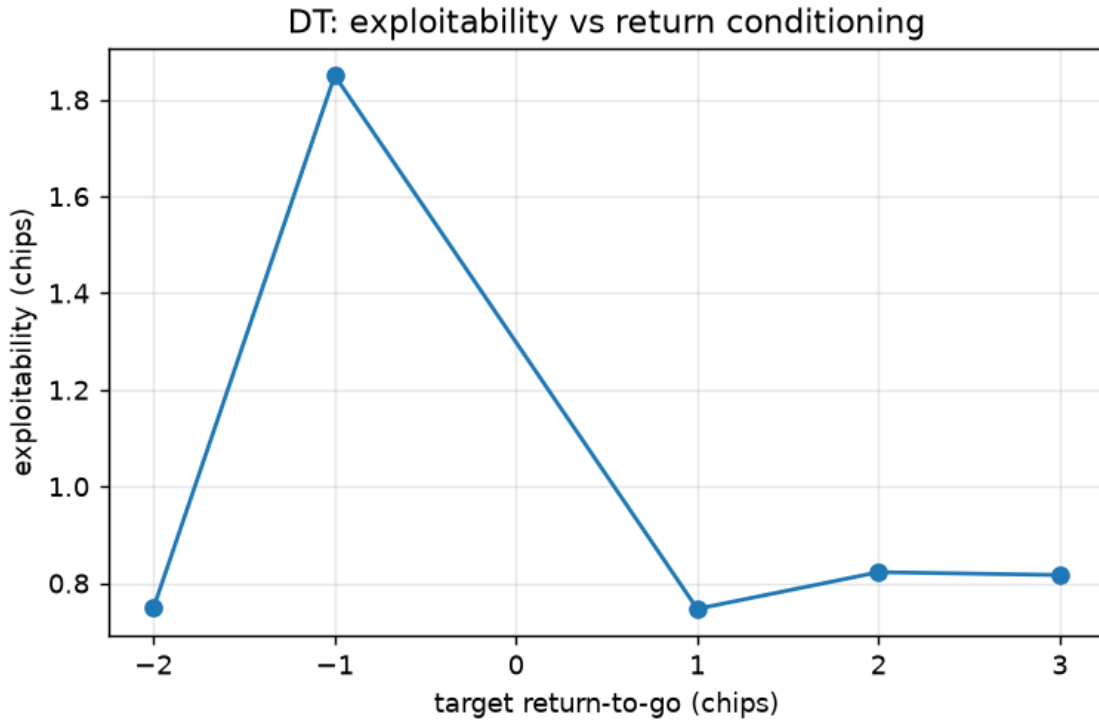


Figure 2: DT exploitability against the conditioned target return, including an impossible +3. The curve is flat across the real range with a sharp collapse at $R = -1$, the modal payoff and the payoff of folding.

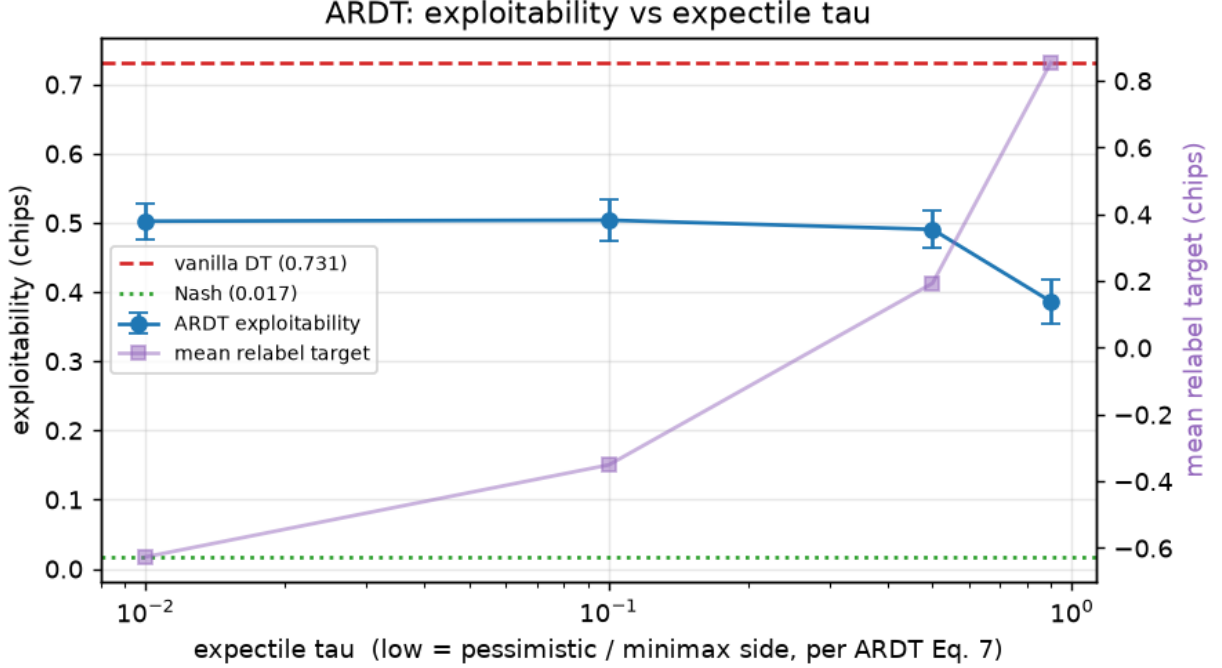


Figure 3: ARDT exploitability against the expectile tau, with the mean relabel target on the right axis. The relabel target moves monotonically as theory requires, but exploitability is lowest on the optimistic side.

against sampling on identical prompts: mean $|P(\text{bet})|$ gap **0.027** against a binomial SE of 0.102 (consistent). It is *not* valid with a chain-of-thought prompt — the prefill contradicts “reason first”, giving an out-of-distribution policy — a boundary documented in the code.

1.5.2 Where the loss actually is

Information set	P(bet)	Nash	deviation	share of leak
2p (Queen, opponent checked)	1.000	0.000	1.000	41.4%
2b (Queen, facing a bet)	0.253	0.347	0.094	13.1%
3 (King, root)	1.000	0.561	0.439	0.1%

This **corrects a headline stated earlier in this step**. “Every LLM value-bets the King at 1.00 where Nash mixes at 0.68” was reported as a signature failure — it costs **0.1%** of the leak. Over-betting a hand that is never behind is nearly free. The damage is one Queen node: always betting the Queen after the opponent checks is **41.4%** of total exploitability, and repairing that single decision alone moves the model from 0.357 to 0.209 chips. **Deviation magnitude and cost are nearly uncorrelated**, which is precisely what a single number hides.

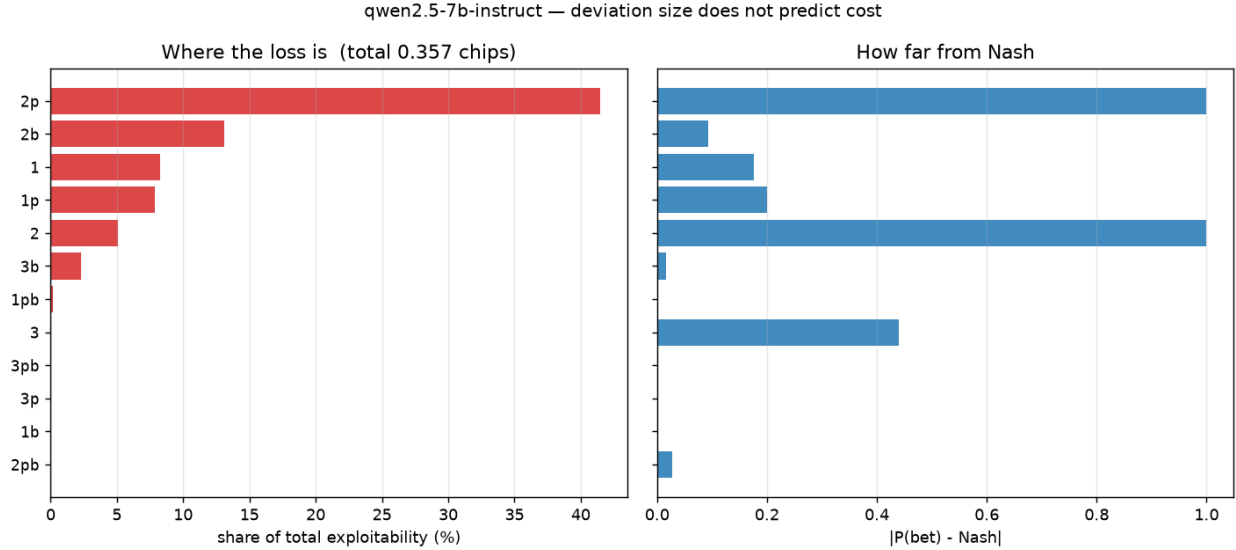


Figure 4: Each information set’s share of total exploitability, beside how far it deviates from Nash. The two orderings barely correspond.

1.5.3 The models play better than they can explain

Asked, at the same information sets and with identical situation text, what percentage of the time they should bet:

	MAE vs Nash (stated)	MAE vs Nash (executed)	expl. if it played what it says	expl. as played
Qwen2.5-7B	0.353	0.246	0.921 chips	0.357
gpt-oss-20b	0.434	0.328	1.576 chips	0.392

Both models’ **stated** strategies are substantially worse than their **played** ones — $2.6\times$ and $4.0\times$ more exploitable respectively. The failure modes differ (Qwen answers “50%” at 11 of 12 information sets; gpt-oss answers $\sim 0\%$ at 8 of 12, including all three King nodes where Nash bets 100%), which strengthens the shared conclusion: **extracting a strategy from an LLM by asking it yields something worse than probing its behaviour.**

1.5.4 Exploitative by default, not adaptive

Sessions of 120 hands against a fixed archetype, with the observed history of previous hands in context. Baselines are computed **exactly** on the agent’s own realised deals, so neither deal luck nor opponent randomness enters the comparison.

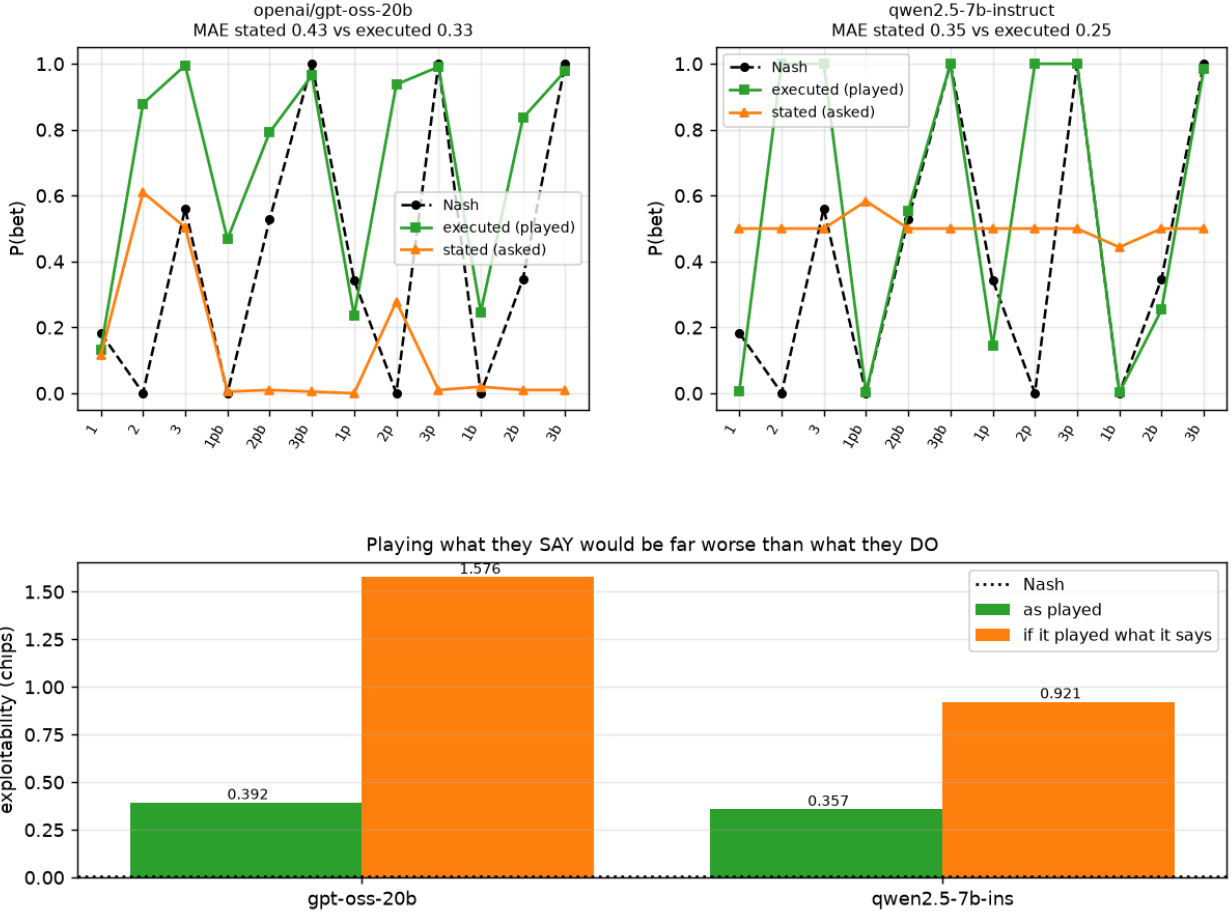


Figure 5: Stated versus executed betting frequency at each information set for both models, with Nash for reference, and the exploitability of each model’s played strategy against a strategy built from its own stated frequencies.

Opponent	Nash	best-response		gap closed	learning (2nd – 1st half)
		ceiling	agent $\pm$ SE		
AlwaysPass	+0.229	+0.981	+0.850 $\pm$ 0.056	+0.83 $\pm$ 0.07	+0.00
AlwaysBet	+0.319	+0.530	+0.383 $\pm$ 0.169	+0.31 $\pm$ 0.80	−0.52
TightPassive	+0.088	+0.316	+0.092 $\pm$ 0.122	+0.02 $\pm$ 0.53	−0.15

Mean learning **−0.22**. Against the one well-powered cell the agent captures 83% of the available exploitation **from the first half onward and never improves** — a fixed loose-aggressive prior, not opponent modelling.

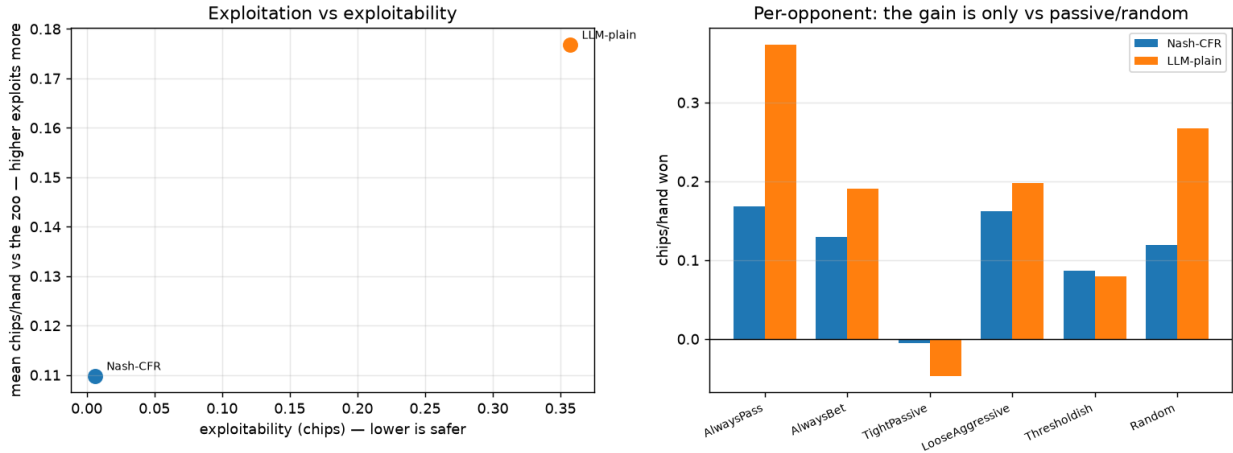


Figure 6: Exploitation against exploitability, and the per-opponent breakdown showing the gain is confined to passive and random opponents.

The exploitation/exploitability frontier corroborates it: the LLM **exploits 61% harder than Nash while being 59 $\times$ more exploitable** (0.177 vs 0.110 chips/hand), but the entire gain is against *passive or random* opponents (AlwaysPass 0.374 vs 0.168). Against the two most competent archetypes it is **worse** than Nash. This is the safe-exploitation trade-off of Contribution #2, measured.

1.5.5 Head-to-head

20,000 hands per pair, seats alternated. Nash versus Nash is 0.0000 exactly and Nash loses to nobody, which validates the tournament.

Row player	vs Nash	vs gpt-oss	vs Open-Thinker3	vs Qwen	mean	exploitability
Nash-CFR	0.0000	+0.1061	+0.1245	+0.0806	+0.104	0.0061
Qwen2.5-7B	-0.0806	+0.1618	+0.1817	0.0000	+0.088	0.3571
gpt-oss-20b	-0.1061	0.0000	+0.1535	-0.1618	-0.038	0.3917
OpenThinker3-7B	-0.1245	-0.1535	0.0000	-0.1817	-0.153	0.8940

The **7B beats the 20B**, and the ordering is strictly transitive and matches the exploitability ordering exactly — in this population, exploitability predicts head-to-head results.

1.6 Leduc Hold'em — a complexity check

A narrow port (encoder, dataset, DT training, and a return-conditioning sweep scored by chips/hand against a near-Nash opponent) plus an LLM scouting pass. Leduc has **936** information sets versus Kuhn’s 12, **15** distinct payoff values versus 4, two betting streets and a board card.

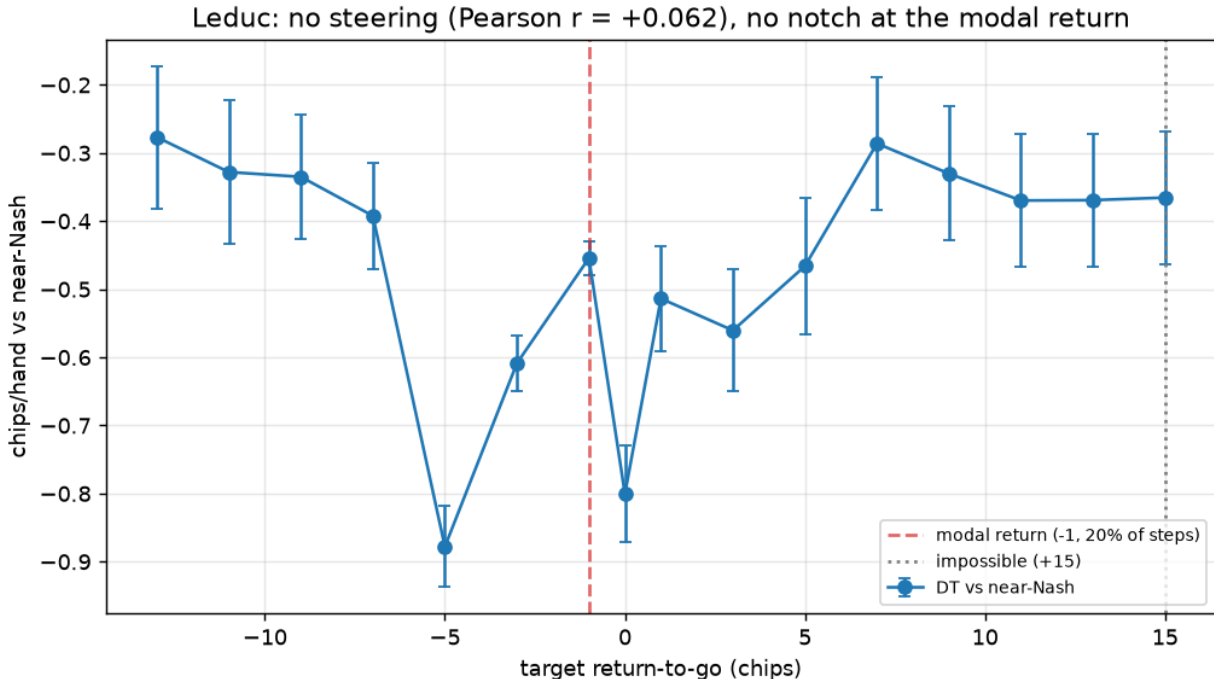


Figure 7: DT performance against the conditioned target return on Leduc, with standard errors. There is no collapse at the modal return and no monotone trend.

The Kuhn notch does not reproduce — at the modal return the DT is 0.2 SE from the mean of the other targets. **But conditioning still does not steer:** Pearson $r = +0.062$, Spearman $r =$

-0.054. It is not inert (the spread across targets is 0.60 chips/hand, many times the per-point SE), it is simply not ordered by target quality. The impossible target saturates, the one Kuhn prediction that reproduces cleanly.

LLM competence degrades sharply. On Leduc the LLM scores -0.463 ± 0.132 chips/hand against near-Nash, statistically indistinguishable from the DT’s -0.454; against the exploitable zoo it manages -0.071 where Nash makes +0.582, losing even to **Random**. Only **31–54%** of the 936 information sets were ever reached, so reach probability makes full enumeration unnecessary.

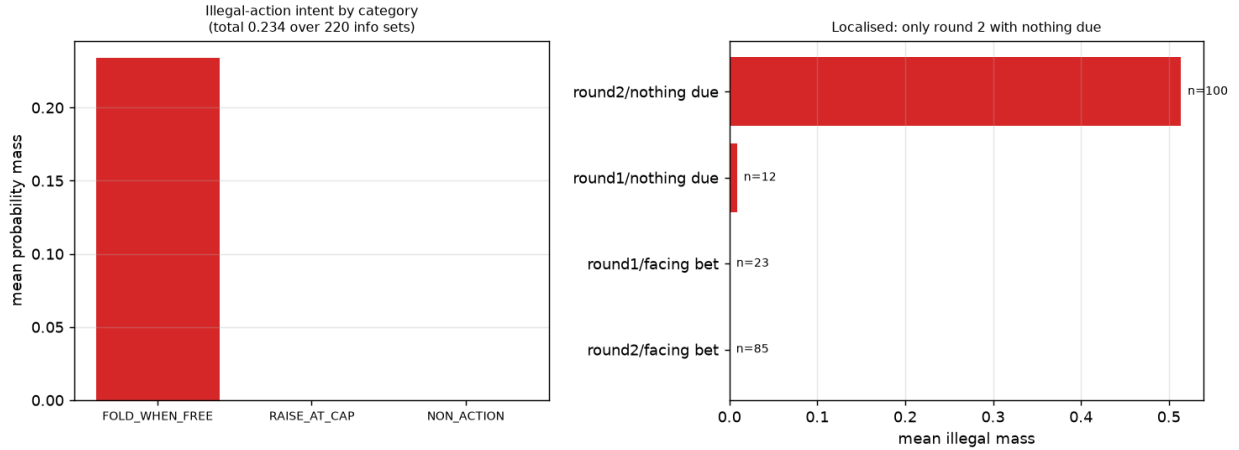


Figure 8: Illegal-action intent by category and by situation. A single category accounts for all of it, concentrated in one situation.

Its 23.4% illegal-action intent is **one systematic misconception**, not diffuse confusion: FOLD_WHEN_FREE is **100%** of the illegal mass, while RAISE_AT_CAP is 0.0000 and non-action (format failure) is 0.0001. It is concentrated in round 2 with nothing due (0.5138, up to 0.99), on weak unpaired hands against a high board — the model conflates “my hand is weak” with “I should fold”, forgetting that checking is free. The near-zero round-1 rate shows this is **board-texture-triggered rather than a rules gap**, making it a prompting finding rather than a capability ceiling.

1.7 Prediction vs reality reconciliation

Pre-run predictions are kept; what actually happened is appended (WORKFLOW §0.1).

R1 — Return conditioning never steers, and the proposed mechanism was half right. The Kuhn explanation (magnitude encodes the betting line, sign is card luck, so the modal fold payoff selects “the folding line”) predicted the effect should weaken with a richer payoff alphabet. Leduc removed the notch exactly as predicted **but steering did not appear in its place**. The mechanism explains the *notch*, not the *failure*, which survives 15 payoff values, two streets and a board card.

R2 — MATH FLAG B was correct about and understated the structural gap. Confirmed against Eqs. 6–7; the real gap is $V(s)$ versus the paper’s $Q(s, a)$ (Algorithm 1 line 7). Target #4 stays red.

R3 — Behavioural cloning wins, and the King “failure” was nearly free. See *The headline comparison* and *Where the loss actually is*.

R4 — The going-in hypothesis was refuted. “Knows the frequency but cannot sample it” predicted stated Nash and executed Nash. Measured: stated is *worse* than executed on both models.

R5 — An in-context-learning result of +1.59 was retracted. A gap closed above 1.0 means beating the exact best response, which is impossible; the ceiling was verified by hand ($K \rightarrow +2$, $Q \rightarrow 0$, $J \rightarrow -1$ +0.333, matching the computed +0.3387), identifying the agent’s figure as the artefact. Cause: a 60-hand session compared against baselines sampled over *different* deals, where Kuhn’s per-hand standard deviation of ~ 1.2 gives an SE comparable to the entire Nash $\rightarrow$ BR span. With exact paired baselines the conclusion **reverses** to no learning.

R6 — A Leduc LLM score of -0.83 was retracted. The tokenizer splits RAISE into " RA" and FOLD into " F" while CALL/CHECK stay whole, so whole-word matching discarded 70% of the probability mass and *inverted* the policy — a model raising at $p=0.953$ was scored as calling $\sim 97\%$ of the time. Corrected with prefix-aware matching to -0.463 ± 0.132 .

R7 — LLM mixing cannot be measured at temperature 0. A real model plays a *pure* strategy there, so every measured frequency degenerates to 0 or 1: bluff(J) read 0.75, 0.25 and 1.00 across three runs of the same configuration, and raising the sample count from 4 to 24 did not help. At temperature 0.7 the frequencies become intermediate and the illegal-move rate becomes nonzero.

1.8 Trustworthiness and sample adequacy

Four results in this step looked like discoveries and were measurement artefacts (R5, R6, a seat-0 return that was 1.7 SE of sampling noise, and a Leduc trend that vanished at $20\times$ the hands). Each was caught by a cheap consistency check against something exactly computable — an exact game value, an exact best-response ceiling, a standard error, or a probability-mass conservation diagnostic.

Separately, `plotting.py` originally contained **only** a self-test that plotted hard-coded numbers into the results directory, and it was the sole figure-producing path in the step; following the runbook literally would have committed a fabricated figure. It was replaced with a renderer that reads exclusively from committed JSON, which is how every figure in this report was produced.

Sample adequacy: head-to-head cells use 20,000 hands; zoo matches 4,000; the Leduc sweep 4,000 per target. The opponent-modelling cells for AlwaysBet and TightPassive are **underpowered**

(SE ± 0.80 and ± 0.53 , because their Nash $\rightarrow$ BR spans are narrow) and only the AlwaysPass cell is conclusive.

Training variance in the neural rows. The dataset, CFR and the offline stub are seeded and reproduce exactly, but each invocation of the comparison table **retrains** the neural models, so DT/BC/ARDT carry run-to-run variance. Measured across two runs of an identical configuration, DT exploitability moved **0.671 $\rightarrow$ 0.799** (a $\sim 19\%$ swing) while the two deterministic rows (Nash-CFR 0.0162, LLM-stub 0.8333) reproduced to four decimal places. The numbers quoted here are those of the committed artifact `results/comparison_SMOKE_stub.json`; the qualitative ordering is unaffected, but **these rows should not be compared at three decimal places across runs.**

1.9 Limitations (ranked by how much they affect the conclusions)

1. **Every Leduc LLM conclusion rests on one model, one prompt style, 600 hands** (SE ± 0.13). Enough to say the Kuhn advantage does not transfer; not enough to rank models on Leduc.
2. **ARDT is a documented simplification of the published method** — single-sided, state-only $V(s)$ instead of the coupled state-action $Q(s, a)$. Targets #3–4 test the proxy, not ARDT.
3. **No exact Leduc exploitability.** Leduc’s metric lives in Step 03, whose package is also named `cfr` and collides with Step 02’s; chips/hand against near-Nash was used instead.
4. **Kuhn results are SMOKE-profile** (5,000 trajectories, 5,000 CFR iterations, small networks). The qualitative ordering is robust, but absolute values would shift at scale.
5. **Logprob extraction is validated for plain prompts only**; CoT figures come from sampling.
6. **OpenThinker3 was measured on the CoT row only** ($n=12$); a full pass is ~ 9 hours at ~ 38 s per decision.
7. **B5’s two underpowered cells** (above) — tightening them needs roughly $30\times$ more hands.

1.10 Conclusions and research directions

Conclusions. Return conditioning does not steer a Decision Transformer in poker, on either of two games, and the reason is not the size of the payoff alphabet: in a zero-sum imperfect-information game the realised return is dominated by factors the agent does not control. Plain behavioural cloning beats both of the methods this step is named after. LLMs get hand ranking right and mixing frequencies wrong, cannot verbalise the strategy they actually play, do not learn opponents from observed play, and lose their apparent competence when the game grows by one street.

Research directions (each tied to a measured effect):

- *Replace ARDT’s state-only relabeling with the paper’s coupled state-action estimators* (Eqs. 8–11 plus the Algorithm-1 warm-up). This is the named, evidence-backed reason the proxy

underperforms and the change to make before Step 13’s fixed logs.

- *Treat return-conditioned DT as the wrong instrument for offline poker* — the value of ARDT’s relabeling is precisely that it swaps an uncontrollable conditioning target for a controllable one.
- *Report per-decision decompositions rather than a scalar* in the Step 14 evaluation framework; deviation magnitude is not a proxy for cost.
- *Probe behaviour rather than asking* in any LLM-based opponent-modelling component.
- *Test the one-line Leduc prompt fix* (“you may check for free”) before drawing any conclusion about LLM capability at multi-street poker.

1.11 Reproduction

from the repository root, with the venv active

`cd implementation/step12/implementation`

```
python validate.py                # 5 validation targets -> 3 PASS / 2 FAIL / 0 SKIP
python comparison_table.py        # headline table -> results/comparison_SMOKE_<backend>
python tau_sweep.py --seeds 3     # MATH FLAG B, empirical half
python exploitability_decomposition.py --style plain --method logprob
python frequency_elicitation.py --repeats 3 --temp 0.7 --exec-style plain
python opponent_modeling.py --hands 120 --style cot
python exploitation_vs_zoo.py --style plain --hands 4000
python llm_head_to_head.py --tournament --hands 20000
python leduc_stage0.py           # Leduc DT return-conditioning sweep
python leduc_llm.py --hands 600  # Leduc LLM scouting
python leduc_illegal_taxonomy.py # illegal-action breakdown
python plotting.py               # all 12 figures, from committed JSON only
```

Real-model runs require a local OpenAI-compatible server. Protocol that matters: **load exactly one model** (`lms unload --all` then `lms load <model>`), restart the server after any killed run (orphaned connections hold all parallel slots and later runs hang indefinitely), and set `STEP12_LLM_TEMP=0.7` — at temperature 0 the measured frequencies are meaningless (R7).

Determinism: datasets, CFR and network initialisation are seeded; the offline LLM stub is deterministic, so the SMOKE table and validation harness reproduce exactly without a GPU or network. Real-model numbers are **not** bit-reproducible (MoE routing under batched serving), which is why frequency estimates use 24 samples and why the temperature protocol is stated explicitly.